



Infor Education
Master Infor products.
Maximize your potential.

CloudSuite Industrial: v10 Using the Application Event System Training Workbook (10.18)

CloudSuite Industrial

September 20, 2018

Course code: 01_0011018_IEN3538_SYT

Legal Notice

Copyright © 2018 Infor. All rights reserved.

Important Notices

The material contained in this publication (including any supplementary information) constitutes and contains confidential and proprietary information of Infor.

By gaining access to the attached, you acknowledge and agree that the material (including any modification, translation or adaptation of the material) and all copyright, trade secrets and all other right, title and interest therein, are the sole property of Infor and that you shall not gain right, title or interest in the material (including any modification, translation or adaptation of the material) by virtue of your review thereof other than the non-exclusive right to use the material solely in connection with and the furtherance of your license and use of software made available to your company from Infor pursuant to a separate agreement, the terms of which separate agreement shall govern your use of this material and all supplemental related materials ("Purpose").

In addition, by accessing the enclosed material, you acknowledge and agree that you are required to maintain such material in strict confidence and that your use of such material is limited to the Purpose described above. Although Infor has taken due care to ensure that the material included in this publication is accurate and complete, Infor cannot warrant that the information contained in this publication is complete, does not contain typographical or other errors, or will meet your specific requirements. As such, Infor does not assume and hereby disclaims all liability, consequential or otherwise, for any loss or damage to any person or entity which is caused by or relates to errors or omissions in this publication (including any supplementary information), whether such errors or omissions result from negligence, accident or any other cause.

Without limitation, U.S. export control laws and other applicable export and import laws govern your use of this material and you will neither export or re-export, directly or indirectly, this material nor any related materials or supplemental information in violation of such laws, or use such materials for any purpose prohibited by such laws.

Trademark Acknowledgements

The word and design marks set forth herein are trademarks and/or registered trademarks of Infor and/or related affiliates and subsidiaries. All rights reserved. All other company, product, trade, or service names referenced may be registered trademarks or trademarks of their respective owners.

Table of contents

About this workbook.....	9
Course overview.....	11
Course description and agenda.....	12
User interface	16
Navigation pane menus	16
Main toolbar.....	16
Keyboard shortcuts.....	16
Field types	20
Icon name differences.....	20
Lesson 1: Introduction to the Application Event System	23
About the Application Event System.....	24
Basic components	25
Events	25
Event handlers.....	25
Event actions.....	26
Advantages	28
Reduce or eliminate the need for procedural code.....	28
Prevent situations where event handlers could be inadvertently changed	28
Provide an easier way to gain control of the application flow.....	28
Examples.....	29
Predefined workflow event handlers.....	30
Check your understanding	31
Lesson 2: Understanding how the system works.....	32
When, where, and how events are generated and handled	33
When events are generated	33
Where events are generated	33
How events are handled	34
Framework events	35
Controlling the sequence of event handlers and event actions.....	36
Event handler sequence numbers.....	36
Event action sequence numbers	36
Restricting which handlers run	37
Using the Event Handler form	37
Include or exclude using the Session Access As form	37
Synchronicity	39
Synchronous events	39
Asynchronous events.....	39
Generating synchronous or asynchronous events	39
Synchronous and asynchronous event handlers	40
Suspension.....	41
Suspend-validating mode.....	42
Suspend-committing mode	43
When an event is not suspended	44
Payload	44
Adjournment and resumption	45
Success, failures, and retries	46
When a failure occurs	46
Ignoring failures	46
Retrying event handlers	47

Transactions.....	49
Transactions with synchronous events.....	49
Event handlers marked transactional	50
Event handler not marked transactional	50
Rolling back transactions	50
The Infor Framework Event Service	52
Setting up the service	52
Processing order.....	52
Administrative details.....	53
Troubleshooting	53
Demo: Utility services and verifying the event service setup	53
Check your understanding	55
Lesson 3: Using basic system elements.....	56
System overview.....	57
Exercise 3.1: Log in and explore the Application Event System menu	58
Access As identifier	59
Classifications for Access As	59
Access As form.....	60
Events	61
Event types.....	61
Events form	62
Modifying events.....	62
Deleting events.....	62
Event triggers	63
Event Triggers form	63
Conditions	64
Retesting triggers.....	65
Event handlers and event actions	66
Event handlers.....	66
Event actions	66
Event Handlers form	66
Other options for event handlers	67
Event Actions form.....	67
Create a simple handler for a framework event: IdoPostItemInsert	68
Exercise 3.2: Notification event – Add a customer	69
Event action parameters	72
Syntax for parameters.....	72
Function types	73
Nesting function calls.....	74
Passing parameters from actions	74
Setting variable and parameter values	75
Event Action Parameters forms.....	75
Using parameters to tailor message content.....	76
Exercise 3.3: Notification event - Refine the message	76
Event variables and initial states	78
Event Variable Groups form	78
Event global constants.....	79
Event Global Constants form	79
Exercise 3.4: Notification event - Create an event global constant.....	80
Other forms	82
Check your understanding	83
Lesson 4: Designing events and handlers.....	84

Setting up custom events and handlers.....	85
Designing a custom event.....	86
Event handling and order.....	88
Event handler initial sequence	88
Resequencing event handlers.....	89
Ordering event actions.....	90
Determining names of IDO collections and components.....	91
Determining the name of an IDO.....	91
Determining the name of an IDO property	92
Discarding the metadata cache	93
Unload global objects.....	93
Restart the IDO Runtime Service	94
Manually from the IDO Runtime Development Server.....	94
New Workflow Wizard form.....	95
Creating workflows with the wizard.....	95
Exercise 4.1: Using the New Workflow Wizard	97
Notification of changes to an existing record	99
Exercise 4.2: Changes to a credit limit.....	99
Notification that includes an old and new value	103
Exercise 4.3: Changes to an item lot size.....	103
Check your understanding	108
Lesson 5: Viewing event messages	109
About message-related forms	110
Inbox form	111
Variables in event messages	112
Setting variable access.....	112
Setting translatable captions for variables	112
Prompts and responses.....	113
Prompts and external email.....	113
Setting up a prompt with choices.....	113
Saved Messages form	114
Moving messages between folders	114
Send Message form.....	115
Check your understanding	116
Lesson 6: Tracking event system status.....	117
Tracking status forms.....	118
Event Status form	119
Demo 6:1: Event Status form	119
Event Handler Status form.....	120
Demo 6:2: Event Handler Status form	120
Event Queue form.....	121
Demo 6:3: Event Queue form.....	121
Event and event handler revisions	122
Event Revisions form.....	122
Event Handler Revisions form.....	122
Event and event handler revisions	122
Demo 6:4: Event Revisions form and Event Handler Revisions form	123
Suspended Updates form	124
Review of suspended records	124
Demo 6:5: Suspended Updates form	124
Check your understanding	125
Lesson 7: Requesting approvals - basic.....	126

About requesting approvals.....	127
Exercise 7.1: Approving a purchase order.....	127
Event handler diagrams.....	131
Accessing the Event Actions form to modify event actions.....	131
Adding event actions.....	131
Deleting actions.....	132
Editing the diagram display.....	132
Copying the diagram to the system clipboard.....	132
Printing a diagram.....	132
Saving a diagram.....	132
Check your understanding.....	133
Lesson 8: Adding information.....	134
About adding information to a record.....	135
Exercise 8.1: Providing a credit limit for a new customer.....	135
Check your understanding.....	139
Lesson 9: Requesting approvals - advanced.....	140
About multiple and complex approvals.....	141
Approval by multiple levels of management.....	143
Exercise 9.1: Approving a PO by up to three management levels.....	143
Check your understanding.....	154
Lesson 10: Generating events from within a form.....	155
Framework events vs. developer-defined events.....	156
Framework events.....	156
Developer-defined events.....	156
Using the Application Event System within a form.....	158
Exercise 10.1: Calling an application event from a form.....	158
Check your understanding.....	162
Lesson 11: Setting up external email.....	163
Sending messages to external email.....	164
Format of external email.....	165
Troubleshooting email problems.....	165
Setting up external email.....	166
Demo 11.1: Setting up external email accounts.....	166
Check your understanding.....	168
Lesson 12: Voting.....	169
Voting rules.....	170
Dealing with indeterminate voting results.....	173
Quorums.....	174
Check your understanding.....	175
Course summary.....	176
Check your understanding.....	177
Appendices.....	179
Appendix A: User accounts.....	180
Appendix B: Reference tables.....	182
Firing events.....	182
Summary of synchronous functionality.....	184
Framework events.....	185
Framework event attributes.....	187
Application events.....	193
Event action types.....	193
Event action parameters.....	200
Expression functions.....	219

Pre-parser functions.....	236
Complex expansions.....	237
Nested expansions	238
Expression operators.....	238
Appendix C: Expression grammar.....	241
Restrictions.....	241
Start symbol	242
Character sets	242
Terminals.....	243
Rules: Variable, constant, and event parameter references.....	244
Rules: Expressions	244
Rules: Boolean rules.....	244
Rules: Typeless rules.....	248
Rules: String rules	249
Rules: Numeric rules.....	253
Rules: Date rules	255
Rules: Restricted arguments.....	264
Keyword paren lists	264
Enumerations	264
Appendix D: Sample stored procedures	267
Passing parameters to a synchronous event.....	267
Calling a synchronous event within a transaction and handling failure	267
Appendix E: Synchronization of metadata	269
Overview and rationale	269
How sequencing and synchronization work.....	271
Performing upgrades	282
Overriding others' handlers	286
Dealing with obsolete handlers.....	291
Appendix F: Event flow options	295
Appendix G: Sample scenarios	297
Scenario 1: Voting for various choices.....	297
Scenario 2: Localizing message contents by translating captions in a purchase request.....	299
Scenario 3: Cross-site event firing - Adding a message to another site's inbox	301

About this workbook

Welcome to this Infor Education course! We hope you will find this learning experience enjoyable and instructive. This Training Workbook is designed to support the following forms of learning:

- Classroom instructor-led training
- Virtual instructor-led training
- Self-directed learning

This Training Workbook is not intended for use as a product user guide.

Activity data

You will be asked to complete some practice exercises during this course. Step-by-step instructions are provided in this guide to assist you with completing the exercises. Where necessary, data columns are included for your reference.

Your instructor will provide more information on systems used in class, including server addresses, login IDs, and passwords.

Self-directed learning

If you are taking this course as self-directed learning, there may be instructor-recorded presentations and/or simulations available to assist you.

If instructor-recorded presentations are available, a hyperlink to the recording will be included on the first page of each corresponding Lesson.

If simulations are available, the demos and exercises throughout this Training Workbook will include hyperlinks that allow you to view and/or practice the execution of the demo or exercise in a simulated training environment.

Learning Libraries

Learning Libraries in Infor Campus include learning materials that are available to you online, anytime, anywhere. These materials can supplement instructor-led training, providing you with additional learning resources to support your day-to-day business tasks and activities.

Please note that if you accessed this Training Workbook directly via a Learning Library, you will not have access to the Infor Education Training Environment that is provided with all instructor-led and most self-directed learning course versions, as referenced above. Therefore, you will not be able to practice the exercises in the specific Training Environment for which the exercises in this Training Workbook were written.

Symbols used in this workbook



Exercise



Your notes



Question



Demo



Important note



Answer



Scenario or Discussion



Critical note



Task simulation



For your reference



Navigation back



Navigation down



Course overview

Estimated time

.25 hours

Learning objectives

Upon completion of this course, you should be able to:

- Describe the Application Event System (AES).
- Describe the functionality of events and event handlers.
- Define framework events.
- Identify the forms used for the Application Event System.
- Define the purpose of an event global constant.
- Describe key concepts for creating custom events and handlers.
- Define the New Workflow Wizard.
- Discuss how event messages work with the Inbox form.
- Identify the forms used to track event system status.
- Explain how to set up an event that uses a prompt to request approval.
- Explain how to set up an event to add information to a record.
- Explain how to set up an event to request multiple levels of approval.
- Explain how to generate an event from within a form.
- List the steps for setting up external email.
- Match voting rules to their definitions.

Topics

- Course description and agenda

Course description and agenda

This course covers the infrastructure and functionality of the Application Event System. It includes hands-on practice designing and setting up typical system events such as email notifications, approval requests, and voting. This training is for versions: 10.18.00 and all previous versions. Previous course name: Infor Mongoose Guide to the Application Event System.

Prerequisite courses

- User Interface Part 1: Finding and working with forms
- User Interface Part 2: Finding and displaying records
- User Interface Part 3: Adding, editing, and deleting records
- User Interface Part 4: Additional record management
- Optional: CloudSuite Foundation 101

Course duration

16 hours

Prerequisite knowledge

To optimize your learning experience, Infor recommends that you have the following knowledge prior to attending this course:

- Basic knowledge of database query syntax

Audience

- Customer User
- Pre-Sales Consultant
- Business Consultant
- Technical Consultant
- Support
- System Administrator

System requirements

- Infor CloudSuite Training Environment

Course agenda

The agenda below details the contents of this course, including lesson-level learning objectives and supporting objectives.

Lesson	Lesson title	Learning objectives	Time required (hours)
Course overview		Review course expectations.	.25
1	Introduction to the Application Event System	Describe the Application Event System (AES).	.5
2	Understanding how the system works	- Describe the functionality of events and event handlers. - Define framework events.	1.5
3	Using basic system elements	- Identify the forms used for the Application Event System. - Define an event global constant.	2
4	Designing events and handlers	- Describe key concepts for creating custom events and handlers. - Define the New Workflow Wizard.	2
5	Viewing event messages	Discuss how event messages work with the Inbox form.	.5
6	Tracking event system status	Identify the forms used to track the event system status.	.5
7	Requesting approvals - Basic	Explain how to set up an event that uses a prompt to request approval.	.5
8	Adding information	Explain how to set up an event to add information to a record.	.5
9	Requesting approvals - Advanced	Explain how to set up an event to request multiple levels of approval.	1
10	Generating events from within a form	Explain how to generate an event from within a form.	.5

Lesson	Lesson title	Learning objectives	Time required (hours)
11	Setting up external email	List the steps for setting up external email.	.5
12	Voting	Match voting rules to their definitions.	.5
Course summary		Debrief course.	.5

Appendices

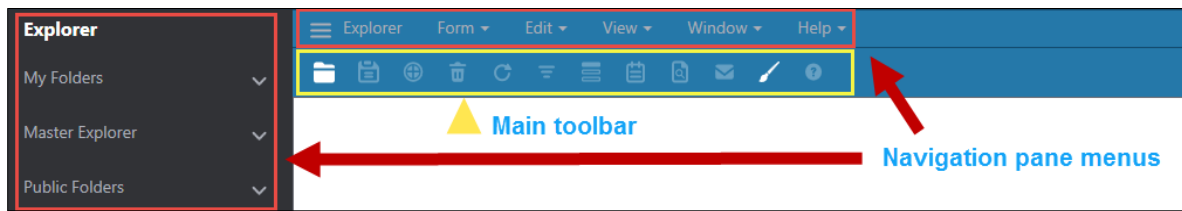
This section contains information that is not part of the instructional content of this course, but provides additional related reference information.

Appendix	Appendix title	Content description
Appendix A	User accounts	This appendix provides a reference for student and instructor login credentials.
Appendix B	Reference tables	This appendix contains detailed information about the following Application Event System topics: • Firing events • Summary of synchronous functionality • Framework events • Framework event attributes • Application events • Event action types • Event action parameters • Expression functions Expression operators
Appendix C	Expression grammar	This appendix contains the complete grammar available for constructing expressions for event action parameters.
Appendix D	Sample stored procedures	This appendix contains sample code and function usage for the following stored procedures: • Passing parameters to a synchronous event • Calling a synchronous event within a transaction and handling failure
Appendix E	Synchronization of metadata	This appendix covers the various concepts and strategies for synchronizing metadata belonging to different owners.

Appendix	Appendix title	Content description
Appendix F	Event flow options	The flow diagram in this appendix illustrates some of the possible ways an event can be generated and the types of flows that can result. It highlights the differences between functionality with and without the Application Event System.
Appendix G	Sample scenarios	This appendix provides sample scenarios.

User interface

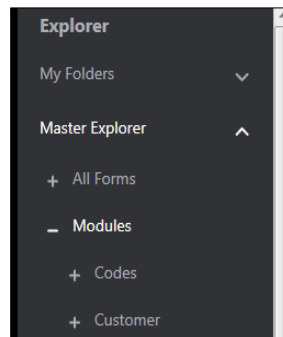
Standard user interface elements include the navigation pane menus and the main toolbar.



Navigation pane menus and main toolbar

Navigation pane menus

You can browse all available folders through the navigation pane menus and submenus. Click the down arrow for each of the navigation pane menu, this allows you to explore the submenus contained beneath.



Navigation pane menus and submenus

For information on how to pin submenus to the navigation pane, please refer to the Creating Form Personalizations workbook or ILT. For example, if you know that you will be using the Customer Service submenus frequently, you can display the menus at the top level, by pinning them to the navigation pane.

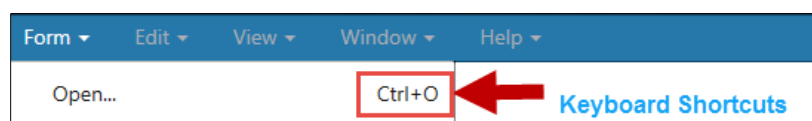
Main toolbar

The main toolbar contains icons that allow you to quickly complete commonly used tasks, such as saving data or closing a form. You can position your mouse pointer over an icon on the toolbar to reveal its description and functionality.

To view a list of all toolbar icons and for steps on how to add/remove icons from the default toolbar, See Appendix B. **Note:** Availability to make hiding buttons visible is not available for Web users.

Keyboard shortcuts

Some navigation pane submenus display available keyboard shortcuts that can be used to save multiple steps when completing certain tasks.



The following tables display the available shortcut keys and their actions.

Navigating forms

Action	Menu Path	Smart Client Shortcut	Web Client Shortcut	Icon	Button Default
Close current form	Form > Close and Cancel Changes	Ctrl + F4	N/A		Hidden
Open Select Form	Form > Open	Ctrl + O	Ctrl + O		Visible
Open workspaces	Form > Workspaces	Ctrl + W	N/A		
Switch view to next open form		Ctrl + Tab	N/A		

Navigating records

Action	Menu Path	Smart Client Shortcut	Web Client Shortcut	Icon	Button Default
Move to first editable field of form	View > Home Cursor	Ctrl + Home	Ctrl + Home		
Move to the next field		Tab	Tab		
Move to the previous field		Shift + Tab	Shift + Tab		
Move to the next record	Actions > Next	F8	F8		Hidden
Move to the previous record	Actions > Previous	F7	F7		Hidden
Switch the next collection on the form to become the current collection in a form that contains multiple collections.	View > Activate Next Collection	Ctrl + F8	N/A		Visible
Hide/Show 1 st Splitter Pane. Toggles back and forth between form and dual view.	View > Hide/Show 1 st Splitter Pane	Ctrl + 1	Ctrl + Shift + 1		
Hide/Show 2 nd Splitter Pane. Toggles back and forth between grid and dual view.	View > Hide/Show 2 nd Splitter Pane	Ctrl + 2	Ctrl + Shift + 2		

Editing records

Action	Menu Path	Smart Client Shortcut	Web Client Shortcut	Icon	Button Default
Activate the drop-down list with an implied asterisk (i.e. wildcard) after the entered text in the field to then populate the list with data entries that begin with the entered text.		F2	F2		
Add a new record Note: The Actions menu is only available when a form is open.	Actions > New	Ctrl + N	N/A		Visible
Add value for current field	Edit > Add	Ctrl + A	N/A		Hidden
Copy (only works to copy a record with web client)	Actions > Copy	Ctrl + C	Ctrl + C		
Cut		Ctrl + X	Ctrl + X		
Paste		Ctrl + V	N/A		
Undo	Edit > Undo	Ctrl + Z	N/A		
Delete record	Actions > Delete	Ctrl + D	Ctrl + D		Visible
Find value for current field (only available for certain drop-down fields)	Edit > Find Value for Current Field	Ctrl + F	N/A		Hidden
Display details for current field drill-down (only available for certain drop-down fields)	Edit > Details for Current Field	Ctrl + L	N/A		Hidden
Save changes (only available when a form is open)	Actions > Save	Ctrl + S	Ctrl + S		Visible

Filtering records

Action	Menu Path	Smart Client Shortcut	Web Client Shortcut	Icon	Button Default
In Filter Place mode, cancel the filter in place and return to the collection previously displayed	Actions > Filter > Cancel in Place	F3	F3		
In Refresh/Run mode, begin filter in place (i.e. clear the collection and go back to Filter In Place mode) In Filter In Place Mode, execute in place (i.e. run the filter)	Actions > Filter > Begin in Place Actions > Filter > Execute in Place	F4	F4		Visible
In Refresh/Run mode, refresh the current collection. In Filter In Place Mode, clear the filter in place.	Actions > Refresh Actions > Filter > Clear in Place	F5	F5		Visible
In Refresh/Run mode, refresh the current record.	Actions > Refresh current	Ctrl + F5	Ctrl + F5		Visible
In Refresh/Run mode, repeat Find Value in Collection	Edit > Repeat Find	Ctrl + F2	Ctrl + F2		
In Refresh/Run or New mode, open associated query form to specify filter criteria.	Actions > Filter > By Query	Ctrl + Q	N/A		
Open field level help topic	Help > Current Field	F1	N/A		
Toggle design mode on and off.	Edit > Design Mode	Ctrl + E	N/A		Visible
Print	Print	Ctrl + P	N/A		

Field types

The following tables display the field types available with a brief description:

Field type	Description
Required, system-generated, and read-only fields	
Order Date:  7/20/2013  15	- Any field that is required will have a red star - A system-generated field will have a green star - A field with no border around the data is read-only
Standard GUI fields	
 Control Point	Check box indicating Yes or No
 Female  Male	Radio button to select one option from multiple choices
Order Type:  Regular	Drop-down list to select one option from multiple choices

Icon name differences

Here are the names of the different icons in the smart client and web client.

Smart Client	Web Client
Open a form	Open
Save	Save
Create a new object in the current collection	New
Delete an object in the current collection	Delete
Refresh the current collection	Refresh
Filter in Place	Filter in Place
Get more rows in the current collection	Get more rows
Display notes for the current object	Notes
Display documents for the current object	Documents
Send email for the current object	Send email for the current object
Export the data in the current collection to excel	Export to excel
Enter or exit design mode on this form	Open designer in a new window
Translate	Translate

Smart Client	Web Client
Get help on the current form	Help Current Form



Lesson 1: Introduction to the Application Event System

Estimated time

.5 hours

Learning objectives

After completing this lesson, you will be able to:

- Describe the Application Event System.

Topics

- About the Application Event System
- Basic components
- Advantages
- Examples
- Predefined workflow event handlers

About the Application Event System

The Application Event System is a powerful and flexible set of tools designed to help extend and customize any business process in CloudSuite. You can use these tools to define and monitor events that take can place anywhere in the system, on any form.

Think of an event occurrence as a “package” that contains three parts: a condition, a set of rules, and a list of actions. The rules will begin to execute the actions whenever the condition becomes true.

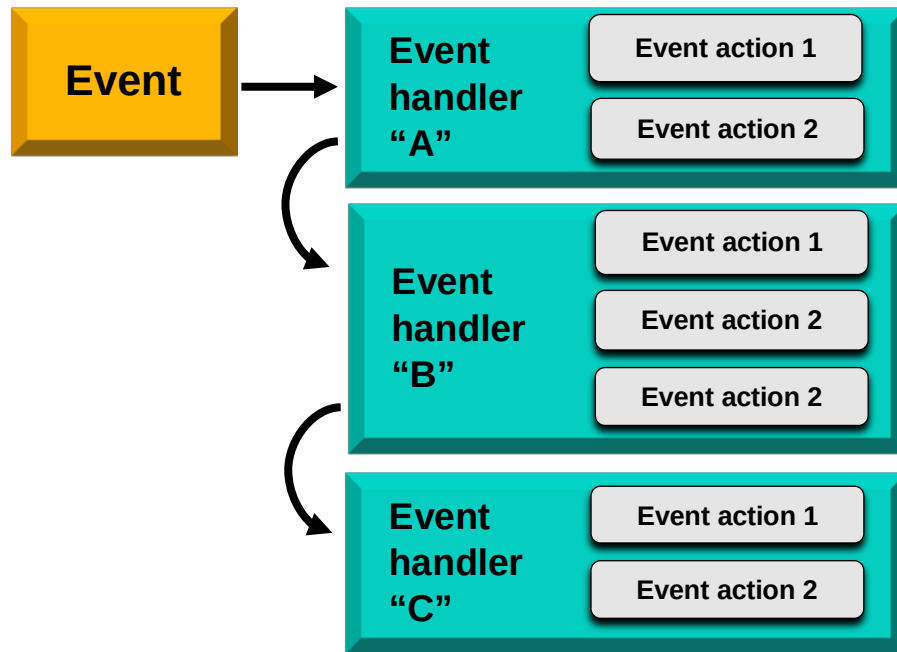
Let's say you want to be alerted when a new product is added to the system. You can use the Application Event System to write simple rules to be notified automatically. The trigger condition becomes true when any system user adds a new item record. When that condition is met, your rules define what actions the system should take, in this case sending you a message about the new product.

The Application Event System can support highly complex conditions too. It can support event actions that can update any data in the application, automate any process provided by the application, prompt a user or group of users to approve a change, integrate to an external application, or simply notify users of a change.

Anyone can use the Application Event System because it allows you to define the rules without writing any code. In this course, we will provide a good grounding in the system basics first. Later, we will guide you step-by-step through a range of typical scenarios that you can modify and use in your own organization.

Basic components

In its most basic form, the Application Event System consists of three interrelated components: events, event handlers, and event actions. The diagram below illustrates the relationship between these components.



Events, event handlers, and event actions

Let's explore each component separately.

Events

Events are uniquely named incidents that can occur during the use of an application. Events can have multiple triggers and can be generated by user actions, conditions in a database, other events, or other situations.

Event handlers

Event handlers are associated with one event and one or more event actions. When an event occurs, an event handler evaluates the event and determines what actions, if any, to initiate. Essentially, an event handler is a set of instructions that describes how the system should respond to a particular event. An event handler consists of data that specifies:

- The event to which it is associated.
- Any conditions, situations, or attributes that determine when and why it should execute.
- One or more event actions to take place when it executes.

An event can have multiple handlers but each handler can be associated with only one event.

Event actions

Event actions consist of data that specifies the individual tasks or bits of work that are performed by the event handler. Each event handler must have at least one event action and can have multiple actions.

Now let's look at an example.

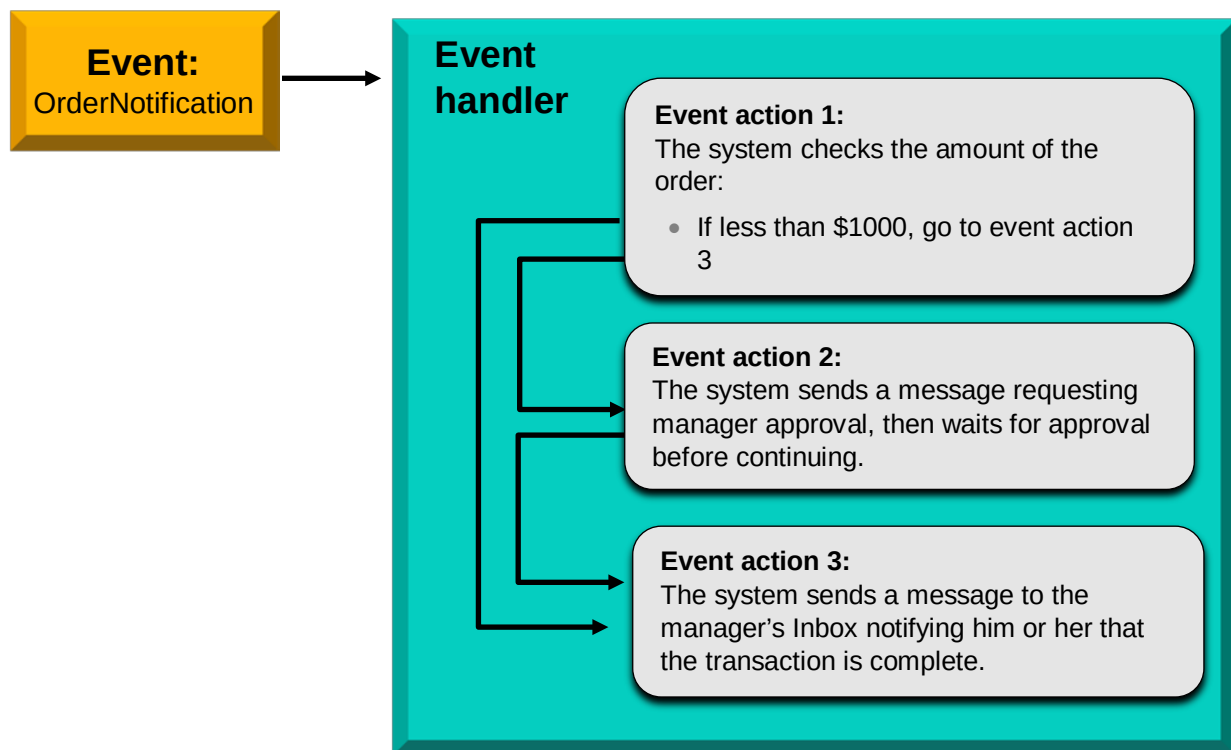
Example ► Order approval

Let's say you want the system to automatically notify you whenever someone adds a customer order to the system and to request your approval if the order is \$1000 or more.

Here are the components as they apply to this example:

- The **event** gets triggered whenever someone creates a new customer order. In this example, we have named the event OrderNotification.
- The **event handler** is set up to execute when the event occurs and any situational conditions have been met.
- The **event actions** that you want to take place are to notify you that an order has been placed and to request your approval for orders of \$1000 or more.

Conceptually, the event, event handlers, and event actions for this example could be illustrated as shown in the following diagram.



Conceptual diagram for new customer order approval

Only a single event handler is needed for this example, but that event handler requires several actions to complete all the requirements.

- The first action is to check the amount of the order and determine, based on the amount, what additional actions are required. If the order is \$1000 or more, then it must direct the flow to another action that requests a manager's approval. If the order is less than \$1000, then it directs the flow to yet another action that simply sends out a notification that the order has been placed.
- The second action is used only if the order is \$1000 or more. If this action is used, the system generates a prompt message to the manager, requesting approval for the order. The system then waits until the manager responds. If the manager approves the order, the system then proceeds to the next action.
- The third action is used regardless of the amount of the order. This action sends out a notification to let the manager know that the order has been completed.

End of example



In the example, what happens if the manager does not approve the order? Please consider your response before reading the answer.

Answer: If the manager does not approve the order, the event handler fails and no further action is taken. The example was specifically kept simple, so it does not provide for the case where the order is not approved. Please note that as a good general practice, an event handler should address all possibilities.



Advantages

The Application Event System offers several significant advantages over hard-coded systems, by helping you to:

- Reduce or eliminate the need for procedural code.
- Prevent situations where event handlers could be inadvertently changed.
- Provide an easier way to gain control of the application flow.

Let's review each in more detail.

Reduce or eliminate the need for procedural code

The Application Event System provides a flexible way to implement event functionality, often without having to modify any procedural code. Metadata is used to define event handlers and actions which eliminates (or greatly reduces) the need for procedural code.

The AES infrastructure allows both the framework and application code to generate events, and for developers and system administrators to create handlers for those events. Event handlers can augment or replace the default framework or application behavior associated with the event. You can design event handlers so that maximal work can be performed without requiring new procedural code.



In this context, the following definitions apply:

- Procedural code refers to compiled code.
- Metadata refers to the practice of using uncompiled code and information about data formats that are interpreted during run time.

Prevent situations where event handlers could be inadvertently changed

System upgrades will not change or overwrite any of the event functionality that you have created with AES. Because event handlers are defined using metadata, any potential upgrade problems are prevented.

Additionally, there are no collision problems if multiple developers have created event handlers for the same event. Events and event handlers are tagged at the time of their creation with a special identifier (Access As) that prevents them from being modified or deleted by other development organizations. Among other things, this prevents your custom events and handlers from being overwritten when Infor or another developer issues updates to their events and handlers.

Provide an easier way to gain control of the application flow

You can use your event handlers with events created by others to gain control of the application flow, augmenting it and even altering default system behaviors. This allows you to take appropriate actions, without being forced to call into the application using application program interfaces (APIs).

Examples

The following examples show how to use the Application Event System to address a variety of common business needs. Later in this course, you will define event handlers and actions for similar scenarios.

Example 1 ► Send notification when a new record is added

Currently, you must rely on everyone who adds customers, trusting that they will remember to send an email notifying the sales manager of each new customer.

The Application Event System can fully automate compliance of the sales manager's request. You can set up an event that is generated whenever anyone adds a new customer into the system, and use event handlers and actions to automatically generate a notification that is sent directly to the sales manager.

Example 2 ► Get approval for a credit limit change

Your company requires that any change to a customer's credit limit be approved by a designated credit manager. Currently, you rely on customer representatives to contact that credit manager whenever a change to a credit limit is requested.

The Application Event System can be set up to automatically send the credit manager a message requesting approval of the change. To speed up the process, you also send an email to the credit manager with links the manager can click to approve or deny the request. When the credit manager responds to the message and approves the request, the system can then automatically change the credit limit amount.

Example 3 ► Complex approval of a purchase order status change

Suppose your company has a multi-tiered business approval process in place that functions as follows:

- Approval is requested from a purchasing manager when the status of any purchase order (PO) is changed to an "ordered" status.
- If approved, and if the request and the total cost of the PO exceeds a certain amount, majority approval is required from a group of mid-level managers on whether the order should be sent to the vendor.
- If those managers also approve and the total cost of the PO exceeds another, higher amount, then a request is sent requiring unanimous approval from a group of top-level directors.
- If the request is approved at all the required levels for the total cost of the PO, then the transaction is approved and completed.

The process described in this example could all be taken care of manually. However, it would involve actions by many employees and the risk of something getting missed would increase with each tier of the approval process.

With the Application Event System, you could use events and handlers to automate this entire process, evaluating the PO at each step of the way, requesting only the necessary approvals, and completing the transaction.

Example 4 ► Automatically ship a customer order

Suppose you have a system that, by default, requires someone to manually set the system to ship a customer order line when the status of the line is set to "filled." This can cause delays in orders, if/when the responsible employee does not set the order to ship in a timely manner.

You can use the Application Event System to "cut out the middle-man" and automatically set the order to ship as soon as the status is set to "filled."

End of examples

Predefined workflow event handlers

The Application Event System is extremely powerful and flexible. However, for some non-developer users, the system can be somewhat overwhelming at first.

To help you with the learning curve, Infor provides a set of predefined workflow event handlers. You can access the full set from the **Workflow Event Handler Activation** form.

These handlers are almost ready-to-use. In most cases, all you need to do is define the user names or email addresses of the people who should be notified when a certain event happens, and activate the handler.

Here are some examples of the more than fifty predefined handlers:

- Alert on shipping order
- Alert users of a new vendor document
- Alert when order shipment is late
- Alert when account's actual > budget
- Approve customer credit limit change
- Approve PO line amount
- Notify customer created
- Notify invoice due date changed
- Prospect interaction follow-up alert
- Send portal question/comment email

For example, if a customer's credit limit is changed, the "Approve customer credit limit change" workflow event handler can automatically send an email to the Accounts Receivable manager and request approval of the change. To set this up, just specify the manager's email address and activate the handler on the **Workflow Event Handler Activation** form.

Check your understanding



Name the three basic components of the Application Event System.

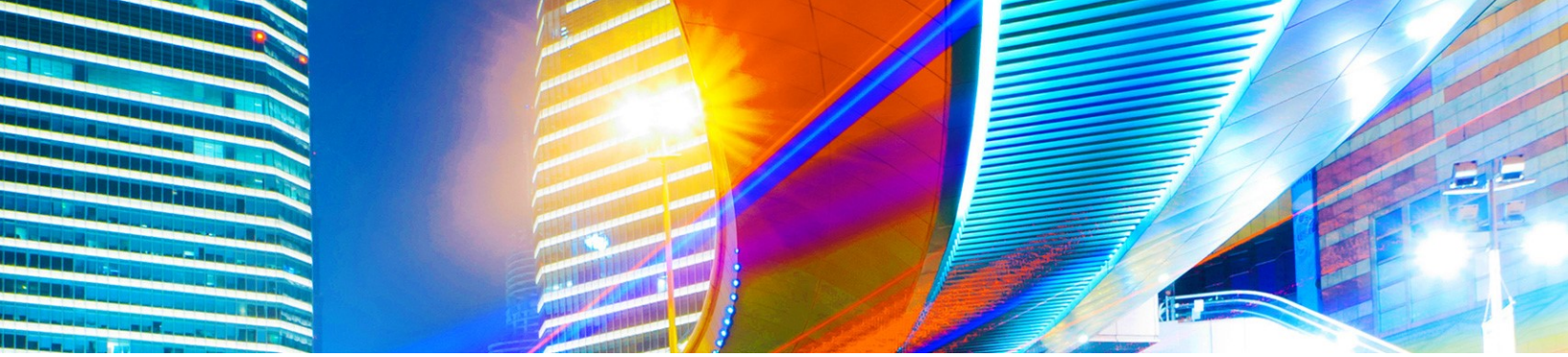


1. _____
2. _____
3. _____



Which of the following is an advantage of the Application Event System over hard-coded systems? Select all that apply.

- a) It reduces the need for writing procedural code
- b) It prevents situations where event handlers could be inadvertently changed
- c) It eliminates the need for metadata
- d) It limits control of the application flow



Lesson 2: Understanding how the system works

Estimated time

1.5 hours

Learning objectives

After completing this lesson, you will be able to:

- Describe the functionality of events and event handlers.
- Define framework events.

Topics

- When, where, and how events are generated and handled
- Framework events
- Controlling the sequence of event handlers and event actions
- Restricting which handlers run
- Synchronicity
- Suspension
- Adjournment and resumption
- Success, failures and retries
- Transactions
- The Infor Framework Event Service

When, where, and how events are generated and handled

Events are generated (also called fired) in response to an action or condition that occurs in the system.

When events are generated

An event can be generated under the four situations shown in the following table:

Event generation situations	Examples
A person using the system performs a particular action, perhaps only on a given form and/or when a particular business process is involved.	A sales representative saves a record in the Customers form. OR A manager changes the credit hold status of a customer.
A database calculation is performed, perhaps resulting in a certain value.	The balance of a certain customer's order becomes greater than \$10,000.
Another event occurs, acting as a trigger.	The replenishment severity level for an item rises due to over allocation of that item.
A certain amount of time passes.	The first day of the month arrives.

Where events are generated

With the Application Event System, events can be generated from any tier.

In this tier...	An event is generated by...
Client tier	Using a form that has a form event handler with a response type of Generate Application Event
Middle tier	Invoking the appropriate .NET method
Database tier	Using the appropriate stored procedure
All tiers	Using the GenerateEvent action type



For details on the tiers and how to generate an event from any of these tiers, see "Firing Events" in CloudSuite's Online Help System.

How events are handled

When the event is generated, it causes the event handlers to evaluate the conditions and fire any associated event actions that are called for by those conditions.

Essentially, when an event is generated, it requires an event handler to do something in response. Otherwise, the event serves no practical purpose. In other words, if there is no handler for an event, the event actually does nothing when it is generated.

Some events, such as framework events, intentionally do not have an event handler.

Framework events

Framework events (or core events) are events predefined in the system that have been designed to be generated only about certain “framework” occurrences. These events—which include system actions like adding a record, modifying a record, deleting a record, and so on—cannot be generated programmatically. That is, you cannot define framework events; they simply get generated every time the system does something.

Now, what this means is that events are constantly being generated in CloudSuite applications. Most of the time, you are not aware of them and the system does nothing in response to them. That is because, if no event handler has been defined to respond to an event, no response is generated. The event occurs, and that is that.

We can, however, define event handlers to respond to these framework events, and that is a large part of the operations that occur in the system.



Framework events were created as a starting point for developers to use with custom event handlers.

Infor includes a set of framework events as part of the system. These events deliberately do not include handlers. Examples of framework events are:

- IdoOnItemInsert
- IdoOnItemUpdate
- IdoPostItemDelete



A list of all framework events is located in Appendix A.

Controlling the sequence of event handlers and event actions

Sequence number settings are used to control the order in which event handlers and their actions will execute.

Event handler sequence numbers

Each event handler has a handler sequence number assigned by the system. Handlers execute in the order of their numeric sequence numbers unless you make modifications.

There are two methods for modifying the default handler sequence:

- Using the **Keep With** and **Chronology** fields on the **Event Handlers** form
- Using the **Event Handler Sequence** form



This topic is covered in more detail in Lesson 4 of this Training Workbook under the topic “Event handling and order.”

Event action sequence numbers

The event actions associated with each event handler also have their own action sequence numbers. These actions will always execute in numeric order, unless at least one of the following is true:

- You use the Initial Action option to designate that a particular action should execute first.
Note: This option is found on the **Event Handlers** form.
- You use certain action types to modify the flow.



Restricting which handlers run

There might be times when you need to disable the event handlers created by one or more development organizations, including yours, at least temporarily. This would occur typically when you are troubleshooting problems with the event system. The system offers several ways to disable event handlers.

Using the Event Handler form

When you want to disable an individual handler with your Access As value, do both of the following:

- Open the **Event Handler** form and for each event handler that you do not want to include in testing or debugging.
- Clear the **Active** check box.

When you are finished testing, and want to enable the handlers, return to the **Event Handler** form and select the **Active** check box for each handler that you previously disabled.

Include or exclude using the Session Access As form



Access As identifies who owns and can modify an event or event handler in the system.

When you want to enable or disable all the event handlers created by a particular developing organization all at once, you can use the **Session Access As** form. To accomplish this, specify which Access As identifiers should have their event handlers execute (included) and which should be ignored (excluded).

The table that follows provides additional information on using the **Session Access As** form to enable or disable handlers:

If you are going to ...	Then ...
Include one or more Access As identifiers	Type the name of the identifier in the Include Access As field. If multiple identifiers are to be included, separate multiple names only by commas (no spaces).
Include all Access As identifiers	Leave the Access As field blank.
Exclude one or more Access As identifiers	Type the name of the identifier in the Exclude Access As field. If multiple identifiers are to be included, separate multiple names only by commas (no spaces).
Exclude event handlers for which the Access As identifier is null (blank)	Select the Exclude Blank Access As check box.



You are not obligated to use both the **Include Access As** and the **Exclude Access As** fields. You can use any combination of the options available on this form.

Example ► Troubleshooting an issue by restricting handlers

Judy, a customer, is having a problem that she suspects is being caused by something she did in the event system, but she is not sure what. She places a call to Infor Xtreme, and the technical support representative wants to verify that Judy's custom event handlers are not causing the problem.

In this case, the technical support representative asks Judy to temporarily disable all custom event handlers so that the operation can be tested with only standard functionality in place. Isolating certain handlers can help to pinpoint where the problem lies.

The technical support representative might also instruct Judy to use the **Session Access As** form to perform one of the steps listed in the table below to disable some of the custom event handlers.

If you are going to ...	Then ...
Allow only the Infor core (framework) and base application event handlers to execute Note: You want all custom event handlers created by business partners and Judy to be disabled (i.e., they will not execute).	In the Include Access As field, specify Core,BaseAppAccessAs where BaseAppAccessAs = Access As identifier associated with the base application installed on the system.
Allow all Infor and business partners event handlers to execute. Note: You want only Judy's custom event handlers to be disabled (i.e. they will not execute).	Leave the Exclude Access As field blank, and select the Exclude Blank Access As check box.

End of example



Synchronicity

Synchronous events

A synchronous event is expected to execute and run through to completion in the generating thread (unless the event is designed to suspend) before the thread continues. Characteristics of a synchronous event are:

- The event runs in the same thread that generated it.
- The event's synchronous event handlers will execute in sequence.
- The event handlers will block the generating thread until they have all been executed or until a handler exits with a failure status.

Asynchronous events

An asynchronous event runs in a different thread than the one that posted it, usually on a utility server. In this case, the event does not block the generating thread, running independently of it. As soon as an acknowledgment is received that the event was successfully queued, the thread that generated the event continues.

Generating synchronous or asynchronous events

The following table shows how an event is synchronous or asynchronous depending on the choices you make when generating the event.

If you generate an event ...	Then it is...
Using a framework (core) event	Synchronous
Using a form event handler with a Generate Application Event response type AND the Synchronous check box is selected	Synchronous
Using a form event handler with a Generate Application Event response type AND the Synchronous check box is cleared	Asynchronous
Calling the FireApplicationEvent() .NET method AND the Synchronous parameter set to True	Synchronous
Calling the FireApplicationEvent() .NET method AND the Synchronous parameter set to False	Asynchronous
Calling this stored procedure: dbo.FireEventSp	Synchronous
Calling this stored procedure: dbo.PostEventSp	Asynchronous
Using an event action of the type Generate Event AND the parameter of Synchronous is set to True	Synchronous
Using an event action of the type Generate Event AND the parameter of Synchronous is set to False	Asynchronous

Synchronous and asynchronous event handlers

A synchronous event handler is one that must complete before the system continues to the next event handler in a sequence. All synchronous handlers in the sequence must complete successfully for the entire event to be handled successfully.

There is an exception to this rule. If one event handler fails for a reason other than for an illegal operation, and the system is set to ignore failures for that handler, the system can continue to the next synchronous event handler. Otherwise, the system returns a failure error, and no more event handlers in the sequence will execute.

By contrast, an asynchronous event handler runs independently of other event handlers.



Event handlers can be designated as synchronous or asynchronous at the time they are created, selecting (or clearing) the **Synchronous** check box on the **Event Handlers** form.

Any event can execute an asynchronous event handler; that is, an event handler for which the **Synchronous** check box is cleared. The system sends the asynchronous event handler to the event handler queue, a process called queueing. The system does this by means of the PostEventHandlerSp stored procedure, to which it passes the configuration name from the event state.

Two outcomes can occur:

- If queueing was successful, the event's thread continues to the next event handler or, if no subsequent handlers are defined, completes the event.
- If queueing was unsuccessful, the event stops with a failure condition.



Suspension

Suspension is when a requested operation is sent to the event system for completion later. In simpler terms, a suspended operation is one that is paused and waiting for something to occur before it can continue. A common reason for suspending an operation is when the system sends a message that requires a response; until the response is received, the system pauses while it waits for a response—and it remains in suspension until it receives a response.

You can define an event handler to suspend by selecting the Suspend option and defining an adjourning action.



An adjourning action, such as receiving a response to a request, causes the system to stop the suspension and continue to the next action.

Suspension is possible only with certain framework events. Currently, there are only three events that can be suspended:

- IdoOnItemInsert
- IdoOnItemUpdate
- IdoOnItemDelete



You cannot create custom events that can be suspended.

For suspension to occur, the following must be true:

- The event must be one of the three framework events that can be suspended.
- You must select the **Suspend** check box for at least one handler (on the **Event Handler** form) that applies to the generated event's object and initiator.

When both above conditions are met, the system passes control of the requested update (insertion/update/deletion) to the event system. The event system then uses a step-by-step process to try to make sure that all the event handlers can execute successfully before committing the system (and the data) to execution of the event.

Suspend-validating mode

When an event is suspended, in what is known as suspend-validating mode, the system takes the actions outlined in the chart below.

Step	The system...	Explanatory details
1	Begins a database transaction	N/A
2	Performs an update to the database, to validate pertinent data against any SQL constraints or triggers	N/A
3	Validates the data by running all effective event handlers that are synchronous and not suspended in the current transaction	This step involves both application and event system changes. At the beginning of this process, in a separate event transaction, the system copies the event handler to a new revision, if necessary. • The reason for this is that both the suspend-validating mode and the suspend-committing mode must use the same metadata. However, the validating transaction will be rolled back, and the master transaction might be rolled back. So, the current event handler revision must be available, in case someone edits the handlers between the times that the two modes execute. • During this process, a non-output event parameter named <code>Suspend_ValidatingMode</code> with a value of 1 is made available to handlers. • You can test this on any synchronous, non-suspended handlers that you do not want to execute now (perhaps because its actions would irreversibly affect something not controlled by our database transaction), using the expression <code>CONDITION(E (Suspend_ValidatingMode)<>1)</code>. Note: At this point, the event system treats any failure as an error and exits the execution of the event.
4	Rolls back the transaction	N/A
5	Places a task on the event queue to run all effective handlers in suspend-committing mode	If the collection update is rolled back due to a later error, this suspension is also rolled back with it, resulting in a state identical to one in which this update was never requested. When the event service picks up the queued event, it knows that a suspension is in effect by checking the <code>EventQueue.Suspend</code> attribute.

Step	The system...	Explanatory details
6	Marks the record involved in the update/deletion request to have the InWorkflow property set to 1	This prevents any user from attempting another change to the suspended record. Note: Suspended insertions do not reach the database at all unless and until the suspended event finishes successfully.
7	Continues the thread in which the event was generated as if the change request succeeded	This occurs even if the thread originated from an event handler executing an UpdateCollection or ExecuteIDORRequest type event action. In other words, suspension affects only one level of event execution.

Suspend-committing mode

Once the suspended event completes the suspend-validating mode successfully and is placed on the event queue, the system enters suspend-committing mode and processes the event using rules noted in the following table:

If this occurs...	Then this results...
A failure or error condition	• Subsequent event handlers are prevented from executing. • Standard processing (that is, the requested insertion, update, or deletion) is prevented from occurring. • The InWorkflow attribute is cleared.
An event handler finishes successfully	The next event handler in sequence executes.
An asynchronous event handler is queued	The next event handler in sequence executes.
The final event handler finishes or is queued successfully	• The standard processing is committed, meaning that the requested insertion, update, or deletion occurs. • For an update request, this also includes clearing the InWorkflow attribute. • The event finishes.

When an event is not suspended

When an event that can be suspended is not suspended (that is, no event handler that applies to that event's object and initiator has the **Suspend** check box selected), the system executes the event using the steps outlined in the chart that follows.

Step	The system ...
1	Performs a database update
2	Executes all effective event handlers, returning either success or failure
3	Handles failures as errors and exits the event

Each insertion/update/deletion request in the process—which could be from an external XML request or from a Save action in WinStudio—participates in the overall transaction. In this case, all complete or nothing completes. So, each handler chain (the same chain executes for each insertion/ update/deletion in the group) also participates in that overall transaction.



A failure of any handler chain results in the entire transaction being rolled back.

Payload

An event that can be suspended carries a payload. This payload represents the requested change (that is, insertion, update, or deletion) and can be accessed and modified by event handlers. For example, if an approval request is sent, the response contains the payload—yes or no.

The value of each named property that forms part of the requested change is passed into and out of the event system as a parameter. The parameter uses the naming convention "Row.Property," where Property is the name of the IDO property. Any modified property values are accompanied by another parameter using the naming convention "Row.Property.Modified" and having value of "1."

During notify and prompt actions, these parameters are:

- Temporarily converted to event variables, which overrides any value that might have been set in preceding actions or specified on the **Event Variable Groups** form.
- Available for display (and update, for prompts) on the **Variables** tab of the **Inbox** form (and for display at the end of an external email prompt).
- Converted back to parameters for subsequent actions.

During other actions such as Set Values, these parameters can be modified to affect the result of the change when it is applied to the database. This can be done by using this syntax:

- SETPROPVALUES("Property"=expression) – If the new value is different than the original framework event parameter received by the event, the PROPERTYMODIFIED(PropertyName) framework event parameter is also set to TRUE automatically
- SETPARMVALUES(Row.Property=expression, Row.Property.Modified="1") – Note that you should also indicate to the framework that the property is now modified

Adjournment and resumption

Some chains of event actions might need to be temporarily stopped while waiting for some condition to be met. A temporary halt in executing the event handler chain is called adjournment.

Example ► Adjournment for manager approval

Company XYZ requires a manager's approval of a customer order before it can be entered into the system. They have set up the event handlers for adding a new customer order so that the system temporarily halts the execution of the event handler chain until the condition is met; that is, until the manager responds with an approval (or disapproval). The system sends a message requesting approval to the manager, and when a response is received, will resume executing the event handler chain.

End of example

Certain event actions, called adjourning actions, must wait for an external stimulus before the event handler can proceed with the next action. In the example given, the adjourning action was the message sent to the manager asking for approval.

An event handler containing such an adjourning action must be either:

- Asynchronous
- or
- Part of a framework event that can be suspended and is marked to suspend

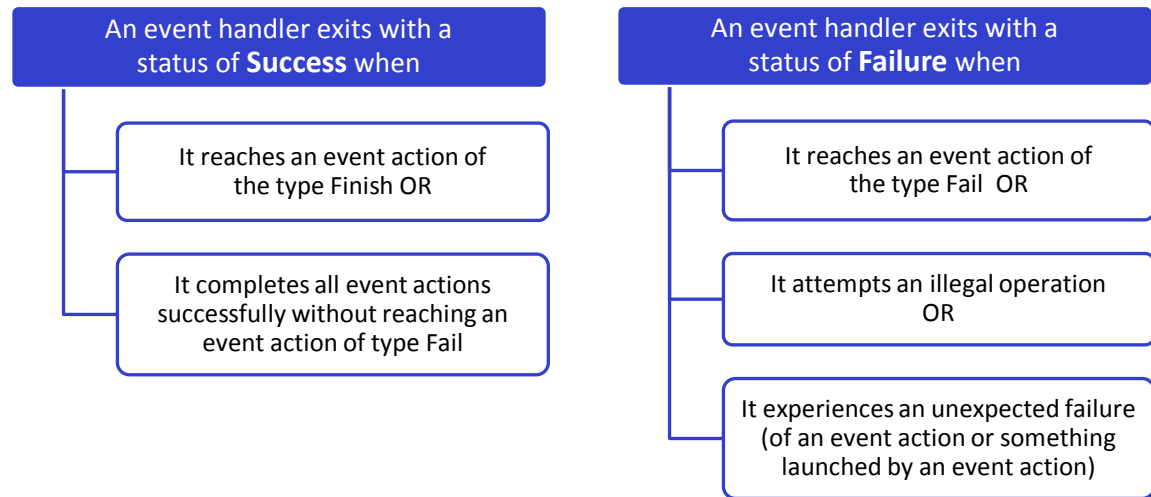
When execution reaches an adjourning action, the event handler state is set to retest or time out at a specified time. At this point, the event becomes an adjourned event.

The event service then processes it at the next opportunity after the Time Out and/or Retest time. This is called resumption.



Success, failures, and retries

Each event handler can exit with a status of Success or Failure. When an event handler fails, if it is not set to ignore failures, and if it has not attempted an illegal operation, the system can retry the event handler's actions.



Causes of an event handler's success or failure

When a failure occurs

Here are the things that take place when a failure occurs:

- The **Event Handler Status** form's **Result** field displays an appropriate error message, which includes an error message passed from a failing stored procedure in a message type parameter (@Infobar) where applicable.
- The current event action state's Times Failed counter is incremented.
- This value displays on the **Actions** tab of the **Event Handler Status** form.
- The event handler state's Last Activity Date is set to the date and time of the failure.
- This value displays on the **Handlers** tab of the **Event Handler Status** form.
- Any event variables are maintained at their current values for inspection on the **Variables** tab of the **Event Handler Status** form.

Ignoring failures

By default, if any synchronous event handler reports a failure, the system skips any remaining event handlers for that event and exits with a status of Failure.

You can override this default behavior by using the Ignore Failure option for the event handler (on the **Event Handlers** form). If you select this option, the event handler is always treated as successful unless it deliberately attempts an illegal operation. The **Status** field on the **Event Handler Status** form, however, still shows a status of Failure, even though the **Any Handlers Failed** check box remains cleared.

The table that follows shows what conditions can result in failures for various action types and whether the failure can be ignored.

Action type	Condition	Ignore Failure option selected	Result (event status)
Set Values	Illegal operation: any attempt to set a non-output event parameter	N/A	Failure
Some	Illegal operation: any attempt to use an action type that is not supported on the current tier	N/A	Failure
Any	Illegal operation: syntax error in parameter	N/A	Failure
Generate Event	Illegal operation: any attempt to generate a framework event	N/A	Failure
Any	Unexpected error	No	Failure
Any	Unexpected error	Yes	Success

For an event handler for which the Ignore Failure option is selected, a Fail action whose condition evaluates to True is equivalent to a Finish action.

An asynchronous event handler's failure does not affect any remaining event handlers for that event. However, setting the Ignore Failure option in this case avoids affecting the AnyHandlersFailed() attribute of that event, unless the failure was caused by deliberately attempting an illegal operation, as shown in the table above.

Retrying event handlers

In the **Event Status** and **Event Handler Status** forms, you can retry a failed event handler, if it can be tried. Whether a handler can be retried or not is determined by the cause of the failure and whether the event handler is marked transactional, as shown in the table.

Cause of failure	Event marked transactional	Handler	Can be retried	Retry point	Event variables
Illegal operation	N/A	N/A	No	N/A	N/A
Any legal operation	Yes	N/A	No	N/A	N/A
Any legal operation	No	Yes	Yes	InitialAction	Re-initialized
Unconditional Fail Event action	No	No	Yes	InitialAction	Re-initialized
Conditional Fail Event action	No	No	Yes	CurrentAction	Maintained
Unexpected error	No	No	Yes	CurrentAction	Maintained

When retrying a single handler, whether the handler runs synchronously or asynchronously depends on the parameter passed to the API, not the handler's definition. When retrying an event, the handlers are run as shown in this table:

Restart type	Failed synchronous handler	Failed asynchronous handler
Restarted synchronously	Synchronous	Asynchronous
Restarted asynchronously	Asynchronous	Asynchronous

An event can have a synchronous handler that is not retrievable, but it will still retry any asynchronous handlers that can be retried. In this case, any synchronous handlers that would have come after the failed handler will not be run.

The Event or EventHandler Result must be cleared when retrying the handler. However, the previous handler result is saved in a handler variable called PreviousResult, and the event results are saved in an event parameter also called PreviousResult.



Transactions

A transactional event is an event or set of event handlers in which all must complete successfully before any data is committed in the system. If any handler fails, then the system rolls back (reverts) all data to its initial state.

If the transactional event contains any adjourning event actions, the system modifies this behavior somewhat: Any actions taken up to the first adjournment, between each resumption of an adjournment and the beginning of the next adjournment, and after the last resumption, are each committed separately.

The system treats any asynchronous event handlers as if they belong to an asynchronously generated non-transactional event. In other words, because they are outside the synchronous flow, these event handlers are not treated as part of the transaction.

Transactions with synchronous events

An event generated synchronously (without the **Transactional** check box selected on the **Event Handlers** form) does one of the following:

- Runs in the current transaction state of the firing thread (if no lower-level transactioning is specified)
- Handles transactions at the event handler or event action level, as explained in these points:

Level	Explanation
Database tier Note: This occurs naturally with no special syntax.	• When FireEventSp is called within an SQL transaction, any changes to the non-event system portions of the database (using Audit, Call Database Method, and Run Background Task event actions) become a part of that transaction, which can be rolled back after FireEventSp returns. • When FireEventSp is called outside an SQL transaction, each event action that might change non-event system portions of the database is wrapped in its own transaction, which is rolled back in case of failure, except when the containing event handler is marked Transactional. Changes to event system state data are performed on a separate connection, so they survive any rollback of the encompassing or wrapping transactions.
Middle tier Note: Any execution methods for database-related event actions must enlist in an existing transaction or create a new one.	• To include the entire event in a single transaction, call the FireApplicationEvent method from a hand-coded IDO method that is marked Transactional in the IDO metadata. In this case, the execution methods for the database-related event action enlist in the transaction created by the IDO Runtime for that transactional IDO method. • When the FireApplicationEvent method is called from a hand-coded IDO method that is not marked Transactional and whose caller is not in a transaction state, each event action execution method creates a new transaction, which is rolled back in case of failure, except when the containing event handler is marked Transactional. Changes to event system state data are performed in a separate

Level	Explanation
	connection (except in suspend-validating mode), so they survive any rollback of the encompassing or wrapping transactions.
Client tier Note: The client is not permitted to control transactions that span form event handler calls.	Client-tier event-firing requests are passed directly on to the MGCORE.Events.FireEvent IDO method.

A failure of a synchronous event returns an error condition to the firing thread. For framework (core) events, this causes the entire current transaction to be rolled back. For application events, whatever generated the event (stored procedure/trigger, .NET method, or VB.NET script) is responsible for trapping the error condition, accomplishing a rollback of the current transaction, and throwing execution to an appropriate recovery point in or out of that code.

Event handlers marked transactional

An event handler marked "Transactional" is wrapped in:

- A new SQL transaction when executed in the database tier.
- A new transaction when executed from the middle tier, if no encompassing transaction is active at the trigger point of the outermost enclosing event.

This event handler cannot contain any adjourning event action types (for example, Prompt, Wait, or Sleep), but it can be marked Asynchronous itself. If this event handler ends in a failure, the wrapping transaction is rolled back (assuming it was created).

Event handler not marked transactional

An event generated asynchronously without the Transactional setting does not run in a transaction state. Event handlers that are not marked Transactional, for a synchronous event generated from a middle-tier non-transactional custom IDO method, are handled this way:

- An event can be generated asynchronously with a Transactional setting that indicates that a new transaction is begun and the entire database-related effects of its synchronous event handlers are committed if they all succeed or rolled back if any fail.
- Any asynchronous event handlers are treated as if they belong to an asynchronously generated non-transactional event.

Rolling back transactions

These conditions cause the system to set the Transactional setting on a new event state row:

- FireEventSp or FireApplicationEvent() detects an enclosing transaction
- FireEventSp, FireApplicationEvent(), PostEventSp, or PostEvent() receives a Transactional setting

This event state setting signals the system administrator that when this event state is finished and not rolled back, everything happened that was expected to happen.

In a synchronous multi-event situation, to encode the proper information in the Rolled Back setting, an enclosing transaction that contains one or more FireEventSp or FireApplicationEvent() calls must follow the certain steps.

The following is a high-level overview of how to encode the proper information in the Rolled Back setting:

Step	How to encode information in the Rolled Back setting
1	Call EventBeginTransactionSp or EventBeginTransaction() Note: This occurs only once, before calling FireEventSp or FireApplicationEvent(). It uses a separate transaction to clear a storage area to record the row pointers of the event state rows to be created and returns a transaction identifier to be passed to FireEventSp or FireApplicationEvent().
2	Pass that transaction identifier to FireEventSp or FireApplicationEvent() Note: This, in a separate transaction, records the row pointer of the new event state row.
3	Call EventEndTransactionSp or EventEndTransaction() and pass the transaction identifier and whether a commit or rollback will be/was performed Note: Perform this step upon rolling back or committing the transaction (it does not matter whether before or after). This uses a separate transaction to set the Rolled Back setting on the stored event state rows (if rolling back), and clears the storage area.

If this is neglected, the system administrator cannot determine why database actions that appear to have finished successfully (according to the status forms) are not reflected in the database.

In asynchronous situations, and when a failure is detected by the event system, the Rolled Back setting for the failing transactional event is set by the event system.



The Infor Framework Event Service

The Infor Framework Event Service is an independent process that can run on any utility server. Each instance of the Infor Framework Event Service monitors the event queue for any number of configurations defined locally.

For load-balancing purposes, this service safely acquires work from the database queue of each configuration's application, so that multiple event services can run on one or multiple servers. The event service monitors the event queue for:

- New events that have been queued by means of asynchronous generation methods or suspending framework events, as found in the Queued Event table.
- New event handlers that have been queued by means of the firing of asynchronous event handlers, as found in the Queued Event Handler table.

Setting up the service

To set up the Infor Framework Event Service to monitor specified configurations, you must specify each configuration individually on the **Event Service** tab of the Service Configuration Manager utility. **Note:** If you do not do this setup, any event handler you create that requires the event service will not run.



For more information, including the procedure, see CloudSuite online help for the Service Configuration Manager utility.

If you want to use the IDO Runtime Development Server (IDORuntimeHost.exe) in your development work, you must also temporarily remove the dependency that the event service has on the IDO Runtime service.



For more information, including the procedure to do this, see the online help for WinStudio developers: From the **Help** menu, select **Customizing Forms**; then, when the help window opens, select the topic "Running IDO Runtime Development Server."

Processing order

The Infor Framework Event Service processes any queued events in "first in, first out" (FIFO) order. Because the event service could receive a request to run something while it is executing a prior request, all requests are queued for execution in the order requested. If the new request happens to be the only one in the queue and the event service is not busy, the request is executed at the next polling.

When the event queue is empty, and immediately after processing each queued event or event handler, the event service checks the Event Handler State and Event Trigger tables for any items whose Retest At time setting has arrived or passed.

The event service then checks the Event Handler State table for any items for which the timeout has expired, that is, the Times Out At time setting has arrived or passed. The event service processes these in order (that is, oldest first), using the Retest At time or Times Out At time to determine the order.

After this, the event service checks the event queue again for any incoming items.

Administrative details

To eliminate the possibility of deadlocks when manipulating event data, a second database connection is used for event data modifications, that is, separate from the one used by the event action execution methods. Also, any event data modifications that do not need to be available to other processes—that is, any tier of any interactive user session or other event service instances—are made in memory and written only when required.

To avoid overtaxing the utility server, the Framework Event Service administrator has control over the minimum interval for these attributes:

- RetestInterval parameter on Wait event actions
- Interval parameter on Sleep event actions
- RetestInterval attributes on event triggers

When any of these attributes is referenced, if the designer-specified value is greater than zero but lower than the minimum interval for the Infor Framework Event Service, the minimum value is used.

The Infor Framework Event Service logs various levels of messages to the Infor Framework Log Monitor.

Troubleshooting

The Infor Framework Event Service must be running for events and event handlers to execute. If you want to explore why your events are not executing as you expect, try these troubleshooting actions:

- First, investigate is whether the Infor Framework Event Service has started and is running.
- Next, verify is that your configuration has been added to the event service so that you can use the event service. This is done in the Event Service Configuration Manager.

Note: Your configuration is the CloudSuite instance that you select from the drop-down list when you log in to the application.



Demo: Utility services and verifying the event service setup

This demonstration shows how to access utility services. This is a demonstration only since it requires a special login.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.

Demo steps

Part 1: Check that the service is running

1. Open the **Landing Server** desktop.
2. Select **Start > Control Panel > Administrative Tools > Services**. The **Services** window displays.
3. Click the **square** in the upper-right corner of the **Services** window. The window expands to full screen.
4. Click and drag the right edge of the **Name** column. The **Name** column expands.
5. Click and drag the right edge of the **Description** column. The **Description** column expands.

6. Scroll down until the **Infor Framework Event Service** record is visible on the screen.
7. Verify that the value in the **Status** column for that record is **Running**.
8. Click **X**. The **Services** window closes.

Part 2: Check configurations

1. Select **Start > All Programs**.
2. Click the **Infor CSI** folder. The folder opens and a list of subfolders are displayed.
3. Right-click **Service Configuration Manager** and select **Run as administrator**. The **Service Configuration Manager** displays.
4. Click the **Event Service** tab.
5. Scroll through the and verify that the **CSI_Demo_DALS** and the **Trainxx** configurations are in the list of configurations. If not, proceed with step 6. Else go to step 11.
6. Click **Add**. The **Event Service** window displays.
7. Select **CSI_Demo_DALS** from the **Configuration** drop-down list.
8. Click **OK**. A dialog box appears with the message: "Selected configuration already exists in the list [Demo]."
9. Click **OK**.

Note: If you added a configuration that did not already exist, the new configuration would appear in the Configurations list.

10. Carry out steps 6 – 9 for any configuration/s you need to add.
11. Click **Save**.

Note: If you add a new configuration, you must restart the Infor Framework Event Service for the new configuration to take effect.

12. Click **X**. The **Event Configuration Manager** window closes.
13. Select **Start > Log off**. The **Remote Desktop Connection** closes.

Check your understanding



Which of the following statements about events and event handlers is true? Select all that apply.

- a) You can temporarily disable an event handler.
- b) Events are either synchronous or asynchronous.
- c) Any event can be suspended.
- d) If an event handler attempts an illegal operation, the system will retry.



Match each event-generating situation with its corresponding event. The possible situations are: **Time passes, Database calculation is performed, User performs an action, and Another event occurs, acting as a trigger.**



Event

Event generating situation

The balance of a customer's order becomes more than \$10,000	
The first day of the month arrives	
A salesperson adds a new customer record	
You run out of an item and all new purchases have a backordered status	



Lesson 3: Using basic system elements

Estimated time

2 hours

Learning objectives

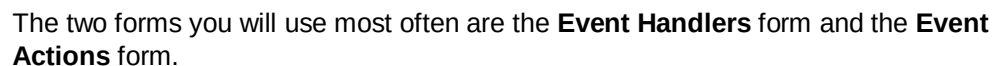
After completing this lesson, you will be able to:

- Identify the forms used for the Application Event System.
- Define an event global constant.

Topics

- System overview
- Access As identifier
- Events
- Event triggers
- Event handlers and event actions
- Event action parameters
- Event variables and initial states
- Event global constants
- Other forms

Here is a quick guide to the most-used forms of the Application Event System. The chart below shows how the forms relate to the system.



© 2018 Infor Education. All rights reserved. No part of this Training Workbook may be reproduced or transmitted in any form, without written permission.



Exercise 3.1: Log in and explore the Application Event System menu

In this exercise, you will log in and locate commonly used forms in the Application Event System menu.

Before you begin:

- Ensure you have completed Exercise [X.X] because it provides data or configurations for this exercise.
- Ensure you are logged in to Configuration Console and it is open to the Configure application page. If not, refer to Exercise 1.2, part 1.
- [Add other prerequisite or note, as needed].

Exercise steps

1. Open the **Landing Server** desktop.
2. Double-click the **CloudSuite Smart Client** icon. The login screen displays.
3. Log in using the information provided by the instructor. The **Infor CloudSuite** application opens.
4. Select **Master Explorer > Modules > System**. The **System** menu displays.
5. Locate the following form in the list, but do not open it:
 - **Access As** form
6. Select **Master Explorer > Modules > System > Event System**. The **Event System** menu displays.
7. Locate the following forms in the list, but do not open them:
 - **Events** form
 - **Event Handlers** form
 - **Event Actions** form
 - **Event Global Constants** form
 - **Session Access As** form
8. Select **Master Explorer > Modules > System > Messages**. The **Messages** menu displays.
9. Locate the following forms in the list, but do not open them:
 - **Inbox** form
 - **New Workflow Wizard** form
10. Select **Master Explorer > Modules > System > Utilities**.
11. Locate the following form in the list, but do not open it:
 - **Workflow Event Handler Activation** form
12. Click anywhere in an empty area of the CloudSuite **Home** screen. The **Master Explorer** closes.

Access As identifier

When first learning about the Application Event System, it is important to understand the Access As identifier and how and why the system uses it.

The Access As identifier:

- Indicates who (what organization) created and "owns" an event or related event system object.
- Prevents unauthorized developers from modifying or deleting event system objects that they do not own.

On many forms in the Application Event System, the **Access As** field indicates who has ownership of the pertinent object metadata. You cannot modify or delete metadata for event system objects that you do not own. However, you can use without changing them.

For instance, you will often use events created by Infor and shipped with the Infor CloudSuite Application Event System, called framework or core events. You can attach your own event actions and handlers to these events—you just cannot directly modify or delete the event system objects.

Classifications for Access As

Generally, the Access As identifier falls into one of three classifications:

- **Core** – Indicates that the object is one that Infor created and owns. These event system objects are used for the framework forms and operations.
- **Any other name** – Indicates that the object was created by and belongs to an application created either by Infor or by one of Infor's business partners or other authorized vendors.
- **Blank** – Indicates that the object was created by and belongs to the end customer.



Which of the three classifications do you think will apply to you as you create events and event system objects during this course? Please consider your response before reading the answer below.



Answer: The Access As identifier will be blank because you are using the system as an end customer.

Access As form

The Access As form is a display-only form and is not directly used to create or customize application events and handlers. The form displays the current Access As setting for your user, which indicates which event system elements you are authorized to modify and/or delete. **Note:** In most cases, this field should be left blank.



If you are an Infor business partner or authorized vendor developer and you need to change the Access As identifier, contact your Infor Consulting Services (ICS) representative for help.



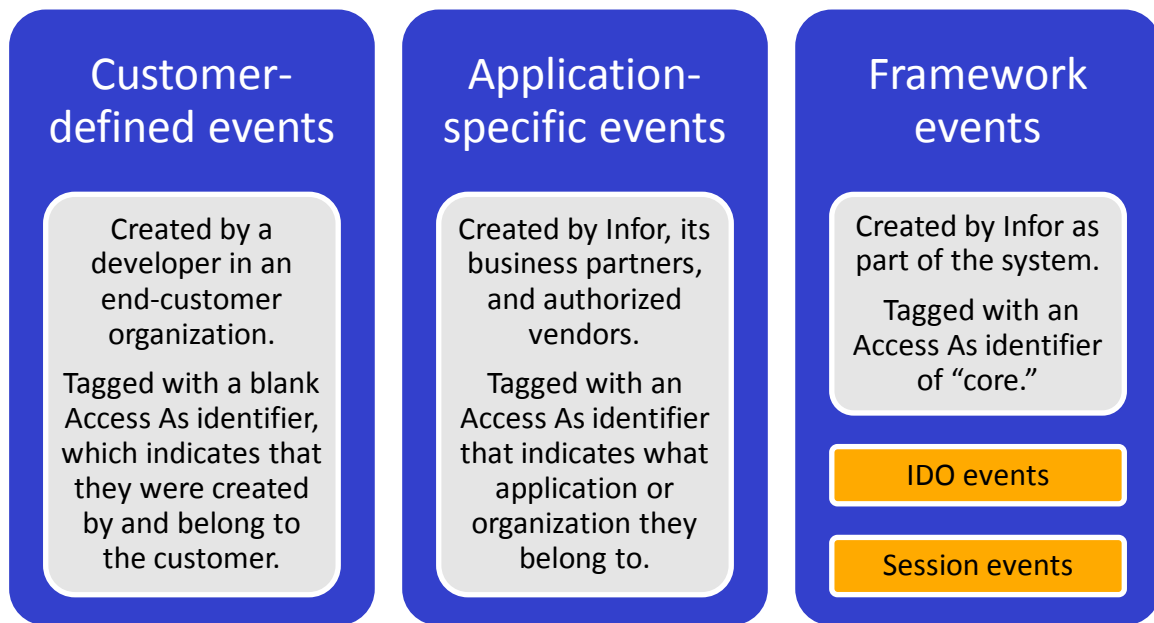
Events

An event, as we learned earlier, is defined as a uniquely named situation that can be triggered by:

- An action performed by somebody working in the system.
- A condition that occurs while the system is running.
- A certain value in a database record that exceeds a set threshold level.
- Another event's handler.
- Other, similar occurrences.

Event types

Events can be classified as one of three general types shown in the following diagram:



Event classifications

Framework events generally fall into one of two categories:

- **IDO events:** These framework events are business process-related and are generated when certain IDOs are invoked. You can identify these events easily by their names, which begin with the letters "Ido."
- **Session events:** These framework events are generated when certain session activities take place. These include SessionOnLogin, SessionOnLogout, and SessionOnVarChanged. **Note:** These events are always synchronous and transactional. Some can be optionally suspended to await user responses.



A list of all framework events can be found in Appendix A.

Events form

The **Events** form is used to name events. Once named here, events are available on other forms as well, including the **Event Triggers** and **Event Handlers** forms.

An event named on the **Events** form and is just that: a name. Until you define a way for it to be triggered or initiated, the event remains just a name. This can be done using either the **Event Triggers** form, the **Event Handlers** form, or both.

Note: An event can be named on other forms (**Event Triggers** or **Event Handlers** form), but be aware that an event name defined on another form will not display on the **Events** form.

Modifying events

Once an event has been created and saved, the only thing you can modify is the event's description. The event name and other attributes are locked.

Here is how to modify an event:

If you are going to ...	Then ...
Modify an event's description	• Open the Events form. • Select the event you want to modify. • Type a new descriptive name in the Description field. • Click Save.

Note: You can modify an event's description only if its **Access As** field has the same value as the current Access As value, as displayed on the **Access As** form.

Deleting events

If you are certain you no longer need an event and you want to delete it, you can.

Here is how to delete an event:

If you are going to ...	Then ...
Delete an event	• Open the Events form. • Select the event you want to delete. • Select Delete from the Actions drop-down list. • Click Save.

Note: You can delete an event only if its **Access As** field has the same value as the current Access As value, as displayed on the **Access As** form.

Event triggers

An event trigger is defined as a condition that causes a particular event to fire, more or less independent of anything that might be happening in the user interface. The event trigger carries a set of event trigger parameters for use when the event fires.

You can set an event trigger to fire the event only once, or you can set it to retest for its condition (and fire again) at certain time intervals after:

- The trigger last successfully fired the event.
- The trigger last tested unsuccessfully for its condition.

In both cases, you have control over the interval to wait, both for the successful firing of the trigger and for the unsuccessful test for the trigger conditions (using separate settings). Testing and retesting is accomplished by means of polling; this is not a true interruptive trigger.

An event trigger carries with it the user name and configuration in effect at the time it was defined. This data is passed on to the event state when the trigger fires the event.



You cannot define a trigger for a framework event.

Event Triggers form

The **Event Triggers** form is used to define event triggers. Use this form to determine the condition that will fire the event, set the parameters to be passed to the event when it fires, and specify retest intervals.

Here is how to create an event trigger with the **Event Triggers** form:

If you are going to ...	Then ...
Create an event trigger	• Open the Event Triggers form. • Click Filter in Place. • Click Create a new object in the current collection. • Select an event from the Event Name drop-down list. • Type the conditions at which the event is to fire. • Type the name and values for any event parameters for which you need to pass values to the event handlers when the event fires. • Click Save.



For more information on other form options, please see the online help for each field.

Conditions

Each event trigger must contain a condition consisting of one of these:

- A Boolean expression
- Two non-Boolean expressions separated by a comparison operator

Let's look at three examples of how to write condition statements.

Example 1 ► Fire event every seven days

This example causes the event to fire when seven days have elapsed since the current result of the database function `dbo.LastEntryDate()`:

```
DATEDIFF(day, DBFUNCTION("LastEntryDate"), CURDATETIME()) > 7
```

Example 2 ► Fire event for customer orders over \$10,000

This example causes the event to fire when the balance on a certain customer's order is greater than \$10,000:

```
DBFUNCTION("OrderBalance", GC(BigCustNum)) > 10000
```

Example 3 ► Fire event at the beginning of each month

This example causes the event to fire on the first day of each month:

```
DATEPART(day, CURDATETIME()) = 1
```

End of examples

The event trigger conditions you create should include an element that changes, which will generally be either:

- A time operation
 - A database calculation
- Or
- Both a time operation and a database calculation

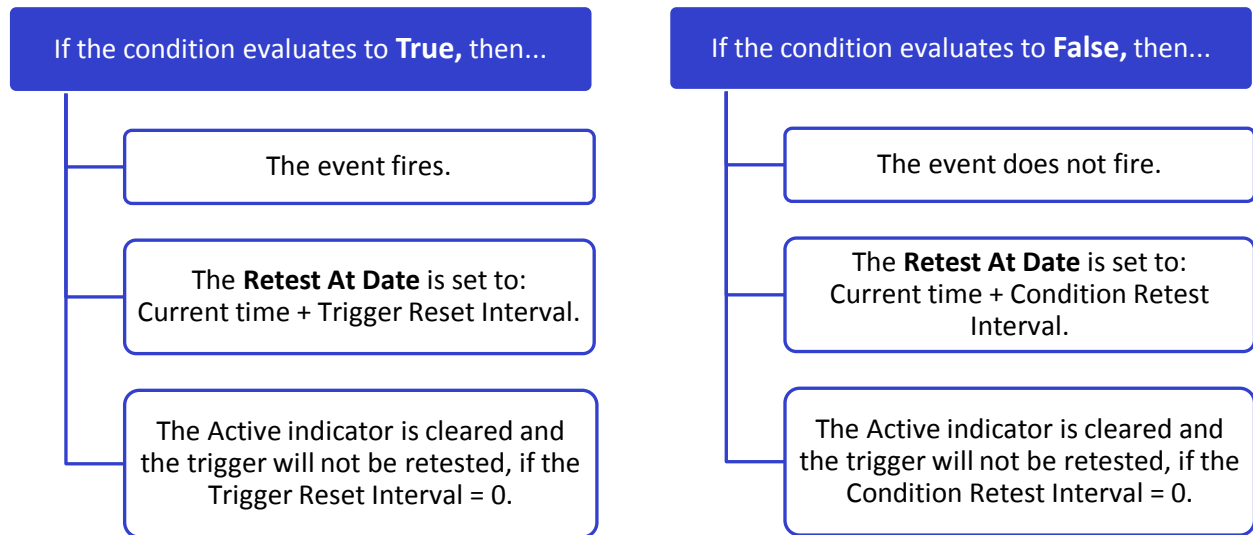
This is because time and the database are the only known factors that can undergo change from external stimuli. Time changes from the forward movement of time; database calculations change because of the actions of application users.



Retesting triggers

When the event trigger's Retest At Date setting becomes older than the current system clock time, the event trigger is available to be processed by the event service. This happens when the event service is free from processing any waiting queued events and handlers and any already-waiting triggers that need to be tested or retested.

To process the event trigger, the system parses the condition and evaluates it as either true or false.



Event handlers and event actions

Event handlers and event actions work closely together and we will cover them both in this lesson.

Event handlers

An event handler defines what actions are to be taken upon the firing of a particular event. Each event handler is comprised of one or more event actions and, optionally, an initial state.

Each event can have multiple event handlers that execute when the event fires. In such cases, the handler sequence number and other factors determine the order in which event handlers are processed.

Event actions

An event action is a unit of work to be performed during the execution of an event handler. A single event handler can have multiple event actions, but each action is assigned to a single event handler. Depending on its action type, an event action can do such things as:

- Evaluate and compare expressions, using the results to select which event action of its event handler to perform next
- Affect the event's visual state
- Complete the event handler
- Set event variables
- Call methods or web services
- Perform other predefined tasks



Event Handlers form

The **Event Handlers** form is used to display and define event handlers which determine the work to be done when an event fires. Each event handler is uniquely defined in the system by the combination of an event name and a handler sequence number. Event handlers also must have one or more associated event actions.

Here is how to create an event handler:

If you are going to ...	Then ...
Create an event handler	• Open the Event Handlers form. • Click Filter in Place.

If you are going to ...	Then ...
	- Click Create a new object in the current collection. - Select an event from the Event Name drop-down list. Note: You may type in a name if you want to define a new event. - Click Save.
Add event actions	- Click the Event Actions button. - Create the actions to be performed when this event handler executes. Note: See the next section for instructions. - Close the Event Actions form.
Set other options	- Set the other options on the Event Handlers form if desired. - Save the Event Handlers form.

Other options for event handlers

You can restrict an event handler so that it executes only in relation to a specific set of conditions. For example, a particular event handler might be defined to execute only when the associated event is triggered by an action on a particular form or when a particular IDO is involved.

You can set an event handler to execute synchronously or asynchronously, or as part of a transactional event or set of event handlers.

You can control the order in which the event handler executes by using the **Keep With** and **Chronology** fields, or the **Event Handler Sequence** form.

Event Actions form

The **Event Actions** form is used to display and define the actions to be performed by a particular event handler during its execution. These are the individual tasks accomplished by the event handlers.



You can open the **Events Action** form associated with a particular event handler by clicking the **Event Actions** button on the **Event Handlers** form.

Here is how to create event actions:

If you are going to ...	Then ...
Create an event action from the Event Handlers form	- Open the Event Handlers form. - Create and save a new event handler, or select the event handler for which you want to create the action. - Click the Event Actions button. - Click New.

If you are going to ...	Then ...
	- Type a number in the Action Sequence field. Note: This number determines the order in which actions for a particular event handler are processed. - Select the type of action to be performed from the Action Type drop-down list. Note: A list of action types can be found in Appendix A.
Add parameters to the action	- Click the Edit Parameters button. - Define the parameters for this action. - Click Check Syntax.
Use a variable and want to set restrictions on the target form	- Click the Variable Access tab. - Define the parameters for this action.
Save the event handler for this action	- Click Save. - Close the Event Actions form. - Save the Event Handlers form.



For more information on other options on the **Event Handlers** form or the **Event Actions** form, please see the online help for each field.

Now that you have been introduced to some of the components of the Application Event System, the next exercise will help to reinforce the concepts. In addition to using the forms we've covered (Events, Event Handlers, and Event Actions), you will also see a new form called the **Edit Action Parameters** form. The exercise will walk you through making simple selections on that form, and in the lesson that follows you'll learn further details and more complex usage of the **Edit Action Parameters** form.

Create a simple handler for a framework event: IdoPostItemInsert

In the exercise that follows, you do not need to create an event, because Infor provides a framework event named IdoPostItemInsert that you can use. All you need to do is create an event handler for that event and assign an action to it that generates and sends the message to the credit manager.

We could use the IdoOnItemInsert framework event instead of the IdoPostItemInsert event. The advantage of using the IdoPostItemInsert event is that, if you allow the **Customers** form to auto-assign the customer number (instead of specifying the customer ID number yourself), the system waits until the ID number has been assigned before filling in the "CustNum" data in the message. If we use the IdoOnItemInsert event, the system does not wait, which means that, if you auto-assign the customer ID number, the resulting message has "TBD" in place of the actual customer number.



Exercise 3.2: Notification event – Add a customer

In this exercise, you will use a framework event. You will set up a simple handler with actions to specify that you are to be notified when a new customer is added. Then, you will test the event; you will add a new customer record, and then verify that a notification message was sent.

Exercise steps

Part 1: Set the Unload IDO Metadata With Forms option

1. Select **System > View > User Preferences**. The User Preferences dialog box opens.
2. Select the **Unload IDO Metadata With Forms** check box on the **Runtime Behaviors** tab.
3. Click the **Runtime Layout** tab.
4. Verify the **Tabbed Form Layout** check box is cleared. If not, clear the check box.
5. Click **OK**.

Part 2: Create an event handler

1. Open the **Event Handlers** form.
2. Press **F3**. The filter is removed.
3. Click **Create a new object in the current collection**. A blank record displays.
4. Select **IdoPostItemInsert** from the **Event Name** drop-down list.
5. Type *Notify Me* in the Handler **Description** field.
6. Type *SLCust** in the **Applies to Objects** field.
7. Select **SLCustomers** from the Applies to Objects drop-down list.
8. Clear the **Synchronous** check box.

Note: The handler can run asynchronously because the notification does not require any response from the credit manager.

9. Click **Save**.

Part 3: Define the event action

1. Select the Event handler you created in Part 1 of this exercise.
2. Click the **Event Actions** button. The **Event Actions** linked form displays.
3. Type *10* in the **Action Sequence** field.

Note: It may already be in the field.

4. Select **Notify** from the **Action Type** drop-down list.
5. Click **Edit Parameters**. The **Event Action Notify (Modal)** form displays.
6. Click the **To** button. The **Event Action Parameter Recipients (Modal)** form displays.
7. Select **<your user ID>** from the **Recipients** list.

Note: Your user ID is the username you used to log in to CloudSuite, for example “train11.”

8. Click **Update**.
9. Click **OK**. The **Event Action Parameter Recipients (Modal)** form closes.
10. Type *New Customer!* in the **Subject** field.
11. Type *Change Notification* in the **Category** field.
12. Type *We have a new customer!* in the **Body** field.
13. Select the **Save In Sent Items** check box.
14. Click **OK**. The **Event Action Notify (Modal)** form closes.
15. Click **Show Details**. The **Parameters** tab displays.
16. Verify that the details show the following information:
 - CATEGORY("Change Notification")
 - TO("UserID")
 - SUBJECT("New Customer!")
 - BODY("We have a new customer!")
 - SAVEMESSAGE(TRUE)
17. Click **Check Syntax**. A dialog box opens with the message: "No syntax errors were found."
18. Click **OK**.
19. Click **Hide Details**. The **Parameters** tab closes.
20. Click **Save**.
21. Close the **Event Actions** form.

Note: To close any form in this application click **X** in the upper-right corner.
22. Click **Save**.
23. Click **Ctrl+U** to discard the metadata cache.
24. Close the **Event Handler** form.

Part 4: Test the event handler

1. Open the **Customers** form.
2. Click **Filter in Place**.
3. Click **Create new object in the current collection**.
4. Type *MyTest1* in the **Customer Name** field.

Note: This field is to the right of the **Customer** field.
5. Select **BK1 Universal Savings** from the **Bank** drop-down list.
6. Click **Save**.
7. Close the **Customers** form.
8. Open the **Inbox** form.
9. Select the message with the subject **New Customer!** The message opens in the lower part of the screen.
10. Click the **Variables** tab. The **Variables** tab displays.

11. Scroll to view the variables.

Event action parameters

Event action parameters are settings that further define certain types of event actions. There are two ways to specify event action parameters:

- Type parameters directly into the **Event Actions** form.
- Use the Event Action Parameters form to define the parameters.

You can specify parameters in any order for an action. To list multiple parameters, specify them one after another, specifying either a space or nothing between them.



When you use the **Event Action Parameters** form, the parameters are returned to the **Event Actions** form properly formatted and free of syntax errors.

Syntax for parameters

The syntax for each event action parameter is as follows: FUNCTION(value).

The value enclosed in parentheses can consist of:

- A constant number
- A literal string enclosed in quotation marks
- A Boolean value: TRUE or FALSE
- An event function call

Note: These can be nested.

- An expression consisting of a number of these elements combined using operators

Although it is not a requirement that function names be specified using all uppercase letters, we recommend the practice, as it leads to greater ease of recognition and readability.

Example ► Simple event parameters

In exercise 3.2, you created several simple event parameters. The parameters are noted in the following table:

Event parameters	Value in the parentheses
CATEGORY("Change Notification")	Literal string
TO("userID")	Constant
SUBJECT("New Customer!")	Literal string
BODY("We have a new customer!")	Literal string
SAVEMESSAGE(TRUE)	Boolean value

End of example



For a list of all acceptable event action parameters, including their meaning and examples, please see Appendix A. For complete expression grammar for constructing event action parameters, please see Appendix B.

Function types

Functions can be any of three basic types:

- Parameter functions
- Value functions
- Word functions

Let's look further at each type.

A **parameter function** is one where the parentheses wrap a parameter to the event action. These functions must always appear at the root level and can never be nested inside any other type of function.

Here are some typical parameter functions:

- SETVARVALUES
- METHOD
- INTERVAL
- EVENTNAME



Parameter functions are identified in this documentation generically as PARAMS(...).

A **value function** is one that calls an event value. These types of functions can never appear at the root level but must be nested within another function construct, either a parameter or another function.

Here are some typical value functions:

- SUBSTITUTE
- DATE
- ABS
- CEILING



Value functions are identified in this documentation generically as FUNCTION(...).

A **word function** is verbatim words used inside certain event function calls. Note that some of these can also be used as event function calls, but word functions always appear within an event function call.

Here are some typical word functions:

- AS
- STRING
- NUMBER
- DAY
- DATE



Value functions are identified in this documentation generically as ...WORD...

Nesting function calls

When building event parameters, you can also use the parentheses following the function to wrap expressions that signify operations to be performed on the results of the expression.

When defining event action parameters, keep these nesting rules in mind:

Function type	Nesting rules
PARAMS(...)	Cannot be nested
FUNCTION(...)	Must be nested within either: • PARAMS(...) functions • FUNCTION(...) functions
...WORD...	Must be nested ONLY within: • FUNCTION(...) functions

This is an example of an event action parameter that uses nested functions:

PARAMS(...FUNCTION1(...FUNCTION2(...WORD1...)...WORD2...)...)

Passing parameters from actions

Parameter lists to methods, scripts, Web services, and generated events are always enclosed in a PARAMS(...) function and delimited by commas.

Each parameter is specified as shown in the following table:

Syntax	Direction	Meaning
V(<i>var</i>)	Input	Pass in the value of the variable <i>var</i>
<i>expression</i>	Input	Pass in the value obtained by evaluating the expression
RV(<i>var</i>)	Input and output	Pass in the value of the variable <i>var</i> , and place the output value into the same variable

Setting variable and parameter values

You can set values for event variables and parameters using these syntaxes:

Event action type	Type of storage	Syntax
Call Database Method Call IDO Method	Variable	PARMS(RV(<i>var</i>))
	Parameter	PARMS(RE(<i>param</i>))
Generate Event Load IDO Row	Variable	SET(RV(<i>var</i>)= <i>name</i>)
	Parameter	SET(RV(<i>param</i>)= <i>name</i>)
RV(<i>var</i>)	Input and output	Pass in the value of the variable <i>var</i> , and place the output value into the same variable

Event Action Parameters forms

Event **Action Parameters** forms are used to define the event action parameters for each action type.



There is a set of **Event Action Parameters** forms, not a single form. Each event action type has its own corresponding form for defining parameters.



To open the **Event Action Parameters** form, click the **Edit Parameters** button found on the **Event Actions** form.

To make it easier to create event action parameters that are properly defined and formatted, each event action type has a corresponding **Event Action Parameter** form. These forms allow you to build appropriate sets of parameters for each action type by providing options that pertain to that action type.

Very few options on any given form are considered required. As a rule, only those fields that you populate return parameter values to the **Event Actions** form. Where strings require quotation marks, the system inserts them automatically. Where expressions are nested inside other expressions or parameter statements, the system automatically places parentheses where they are required.

The result is that, by using the **Event Action Parameter** forms, you can eliminate most, or more likely all, syntax errors in your parameter statements.



Using parameters to tailor message content

Suppose you now want to refine the message sent by an event handler to make it even more informative and useful to the recipient. You may want that message to include the customer number and name for the new customer so that the recipient can look up the customer profile more easily.

In the next exercise, we will revise the event actions we created in exercise 3.2.



Exercise 3.3: Notification event - Refine the message

In this exercise, you will use revise the content of the message you created in exercise 3.2 to include markers for customer name and customer number.

Exercise steps

Part 1: Create message text that includes markers for values

1. Open the **Event Handlers** form.
2. Select *IdoPostItemInsert* from the **Event Name** drop-down list.
3. Type *Notify Me** in the **Handler Description** field.
4. Click **Filter in Place**. The event handler you created in exercise 3.2 displays.
5. Click **Event Actions**. The **Event Actions (Linked)** form displays.
6. Click **Edit Parameters**. The **Event Action Notify (Modal)** form displays.
7. Click **Body**. The **Event Action Expression Editor (Modal)** form displays.
8. Select **SUBSTITUTE** from the **Select a function** drop-down list. Additional fields associated with the SUBSTITUTE function are displayed.
9. Type *We have added a customer, {0}, customer ID {1}, to our family of customers.* in the **Argument 1** field.

Note: The numbers enclosed in curly brackets are the replacement markers for which values will be substituted at run time. They must begin with 0 and increment sequentially.

Part 2: Create the expression that will be used to supply the values for markers {0} and {1}

1. Click in **row 1** of the **Arguments** grid. The row becomes active.
2. Click **Build Expression**. A second **Event Action Expression Editor (Modal)** form displays.
3. Select **PROPERTY** from the **Select a function** drop-down list.

4. Select **Name** from the **Argument 1** field drop-down list.
5. Click **OK**. The second **Event Action Expression Editor (Modal)** form closes.
6. Click in **row 2** of the **Arguments** grid.
7. Click **Build Expression**.
8. Select **PROPERTY** from the **Select a function** field drop-down list.
9. Select **CustNum** from the **Argument 1** field drop-down list.
10. Click **OK**. The **PROPERTY("Name")** and **PROPERTY("CustNum")** are displayed in the **Arguments** grid.
11. Click **OK**. The **Event Action Expression Editor (Modal)** form closes.
12. Click **OK**. The **Event Action Notify (Modal)** form closes.

Part 3: Review the changes and save the event handler

1. Click **Show Details**.
2. Click **Check Syntax**. A dialog box is displayed with the message, "No syntax errors were found."
3. Click **OK**. The dialog box closes.
4. Click **Save**.
5. Click **Ctrl+U** to discard the metadata cache.

Part 4: Test the event handler

1. Open the **Customers** form.
2. Click **Filter in Place**.
3. Click **Create new object in the current collection**.
4. Type *MyTest2* in the **Customer Name** field.
Note: This field is to the right of the **Customer** field.
5. Select **BK1 Universal Savings** from the **Bank** field drop-down list.
6. Click **Save**.
7. Close the **Customers** form.
8. Open the **Inbox** form.
9. Select the message with the subject **New Customer!** The message opens in the lower part of the screen.
10. Review the body of the message on the **Message** tab.
 Note that it now shows the new customer's name and ID in the text of the message body.
11. Click the **Variables** tab.
12. Scroll to view the variables.

Event variables and initial states

If an event handler has variables associated with it, those variables can be assigned an initial state. An initial state is a set of values assigned when the event handler starts to execute.

Each initial state consists of:

- A name that identifies it in the system.
- Any number of event variables with the initial values they are to have when the event handler starts to execute.

Example ► Initial states

For example, you might want to use variables with these initial values with your event handlers:

Variable	Initial value	Comments
Increment	250	None
ItemTypes	AB	None
TimesRemaining	GC(MaxTimes)	In this case, the value is determined by the value of the MaxTimes global constant.

End of example

Event Variable Groups form

The **Event Variable Groups** form is used to display and define initial states for sets of event variables. The initial states are the values to be passed to the event handler when it starts to run.

You can use an initial state you have already created with any other event handler. To use an existing initial state, select it in the **Initial State** field on the **Event Handlers** form.

Event global constants

An event global constant is defined as a named static value that event expressions can reference during processing of the associated event handlers. In simple terms, an event global constant is a placeholder for a stored value. The advantage of using event global constants is that a value is not hard-coded in the handler. This means you can easily change the value of the constant in one place without having to revise all the event handlers that reference that value.

Event Global Constants form

Event global constants are defined using the **Event Global Constants** form. There are only two editable fields on this form:

- **Name** – the name you choose for the global constant
- **Value** – the value or values you want to assign to this global constant

The system references a global constant using a function mechanism that allows dynamic evaluation at each reference.

Now we will look at two examples for using an event global constant.

Example 1 ► Event global constant for a job title

Let's say you have one individual in your organization, Pat, who holds the title of credit manager. In this role, Pat is authorized to review and set customer credit limits. You can create an event global constant named **CreditManager** to refer to Pat and then use it in a variety of ways. You can set up event handlers to:

- Email the **CreditManager** when an order exceeds a customer's credit limit.
- Notify the **CreditManager** when a customer has been placed on credit hold for some reason.
- Prompt the **CreditManager** for a response to a request for an increase to a customer's credit limit.

Later, then, if Pat no longer has the credit manager role, all you have to do is replace Pat's name with the name of the individual who is in the credit manager role, and all the **CreditManager** actions will reference the new individual.

Example 2 ► Event global constant for a yes/no prompt

Your manager wants you to set up a series of events to ask various people to vote yes or no on every new proposal that comes in. You decide event global constants will be useful for defining a set of choices to offer the recipients of a prompt message for each new proposal.

You create a new global constant with the name **PromptChoicesYesNo** and define it with the value 1,sYes,0,sNo. You then reference this constant for all prompt event action, using the expression: **CHOICES(GC(PromptChoicesYesNo))**.

End of examples



Exercise 3.4: Notification event - Create an event global constant

In this exercise, you will create a global constant for the credit manager and then edit the event handler parameters to replace the recipient with the global constant.

Exercise steps

Part 1: Create an event global constant for Credit Manager

1. Open the **Event Global Constants** form.
2. Click **Filter in Place**.
3. Click **Create a new object in the current collection**.
4. Type *CreditMgr* in the **Name** field.
5. Type *<your user ID>* in the **Value** field.
6. Click **Save**.
7. Close the **Event Global Constants** form.

Part 2: Incorporate the event global constant in the event handler action

1. Open the **Event Handlers** form.
2. Select **IdoPostItemInsert** from the **Event Name** drop-down list.
3. Type *Notify Me** in the **Handler Description** field.
4. Click **Filter in Place**. The event handler you created in exercise 3.2 and modified in exercise 3.3 displays.
5. Click **Event Actions**.
6. Click **Edit Parameters**. The **Event Action Notify (Modal)** form displays.
7. Click **To**. The **Event Action Parameters Recipient** form displays.
8. Click **Recipients**. The **Event Action Expression Editor** form displays.
9. Select **GC** from the **Select a function** drop-down list.
10. Select **CreditMgr** from the **Argument 1** drop-down list.

Note: Notice that the event global constant name is not enclosed in double quotation marks. Generally, only literal strings and property names must be enclosed in double quotation marks.

11. Click **OK three times**. You will return to the **Event Actions** form.

Part 3: Review the changes and save the event handler

1. Click **Show Details**.
2. Click **Check Syntax**. A dialog box is displayed with the message, "No syntax errors were found."
3. Click **OK**. The dialog box closes.
4. Click **Save**.
5. Click **Ctrl+U** to discard the metadata cache.

Part 4: Test the event handler

1. Open the **Customers** form.
2. Click **Filter in Place**.
3. Click **Create new object in the current collection**.
4. Type *MyTest3* in the **Customer Name** field.
Note: This field is to the right of the **Customer** field.
5. Select **BK1 Universal Savings** from the **Bank** drop-down list.
6. Click **Save**.
7. Close the **Customers** form.
8. Open the **Inbox** form.
9. Select the message with the subject **New Customer!** The message opens in the lower part of the screen.
10. Click the **Variables** tab.
11. Scroll to view the variables.

Other forms

Several other forms related to the Application Event System are described in the table below.

Form name	Usage
Event Handler Diagram form	This form is used to present a graphical representation of an event handler flow. You can also use this form to access the Event Actions form for selected event actions, to view or modify them. You can also add event actions to an event handler flow using this form. The Event Handler Diagram form is found at this path: Master Explorer > Modules > System > Event System > Event Handler Diagram Note: We will view and use this form in Lesson 7.
Event Handler Sequence form	This form is used to change the order of any handlers that have the current Access As identifier (as indicated on the Access As form). The Event Handler Sequence form is found at this path: Master Explorer > Modules > System > Event System > Event Handler Sequence
System Configuration Parameters form	This form is used to define certain parameters that affect the system configuration. Although not directly used in the creation and customization of application events and handlers, you can use this form to control some aspects of how events and handlers behave on the system. These aspects are concerned mostly with retest and reset intervals that globally govern when and how often various conditions can be retested or events can retry. The System Configuration Parameters form is found at this path: Master Explorer > Modules > System > System Configuration Parameters
Workflow Event Handler Activation form	This form is used to activate predefined event handlers that represent common workflows. You must specify some information, such as users or email addresses that will be notified when an event occurs. You can also copy and modify these event handlers. The Workflow Event Handler Activation form is found at this path: Master Explorer > Modules > System > Event System > Workflow Event Handler Activation

Check your understanding



Match each form to its description. The possible forms are: **Access As**, **Event Actions**, **Event Handlers**, and **Events**.



Form description	Form
This form is used to name events.	
This form contains a button that links to the Edit Parameters form.	
This form displays details on the work to be done when an event fires.	
This form displays who is authorized to modify system elements.	



Which statement best defines an event global constant?

- a) A system-wide event action parameter
- b) A reusable event action
- c) A form for defining global event handlers
- d) A placeholder for a stored value



Lesson 4: Designing events and handlers

Estimated time

2 hours

Learning objectives

After completing this lesson, you will be able to:

- Describe key concepts for creating custom events and handlers.
- Define the New Workflow Wizard.

Topics

- Setting up custom events and handlers
- Designing a custom event
- Event handling and order
- Determining names of IDO collections and components
- Discarding the metadata cache
- New Workflow Wizard form
- Notification of changes to an existing record
- Notification that includes an old and new value

Setting up custom events and handlers

We have already learned about the predefined events that Infor has built in to the Application Event System. There are predefined event handlers as well that are available for your immediate use. In addition, if you have an add-on product distributed by one of Infor's business partners, they might have also added their own events and handlers that you can use.

If you have special requirements, you can design and create your own custom events and handlers to automate tasks for your needs. This lesson provides information about the procedures for creating and using custom events and handlers.

To create and use your own custom application events, there are several important steps and considerations to keep in mind. You must:

- Design and define your custom event.
- Carefully plan and set up the order in which event handlers and actions execute.
- Periodically discard the metadata cache, to ensure that you are working with the most current version of the event metadata.

We will look at each in further detail.

Designing a custom event

When you create a custom event, you must also define what will fire the event. In the current WinStudio system, there are four ways to generate a custom event:

- Create an event trigger using the **Event Trigger** form.
- Use an action of type Generate Event in another event handler.
- Use a form (WinStudio) event handler with a Response Type of Generate Application Event. (Do not confuse the form, or WinStudio events with the Application Event System handlers described in this guide.)
- Use the WinStudio API to write a custom script that generates an event.



The easiest and most common way to generate a custom event is to create a trigger using the **Event Trigger** form.

The following is a high-level overview of a typical process for creating custom application events:

Step	How to create a custom application event
1	Name the event • If you want to name the event before defining how it should be triggered and/or handled, use the Events form. • Names will appear on the Name drop-down lists on the Event Triggers and Event Handlers forms. You can name events named on those forms, however, the name will not display on the Events form.
2	Define the triggers (optional) To define one or more triggers that will fire the event, use the Event Triggers form.
3	Define the event handlers • To define one or more event handlers that will execute when the event fires, use the Event Handlers form. • Each event can have multiple handlers that execute when the event fires. The order in which multiple handlers execute is controlled by several factors.
4	Define the event actions To define one or more event actions for each event handler, use the Event Actions form.
5	Define an initial state (optional) If required to name and define an initial state for the event handler to use, use the Event Variable Groups form.
6	Define global constants (optional) If required to name and define any global constants for the event handler to use, use the Event Global Constants form.

7	Test the event Test the event and its triggers and handlers on a test system before implementing them on your live system.
---	--

Event handling and order

There are two basic ways to arrange the order in which events that have multiple handlers and actions execute those handlers and actions.

Event handler initial sequence

When an event handler is first defined and saved, the system automatically assigns it a handler sequence number. When the event handler is first saved, the system checks to see if there are already any other handlers associated with the named event. Depending on the results of that check, the system then assigns 1 to the handler (if there are no other handlers associated with the event) or the next available integer (if there are other handlers associated with the event).

In general, then, if an event uses multiple event handlers, by default, the system uses the handler sequence numbers to determine the order in which the handlers execute.

It is possible, however, to indirectly alter this default order. This is done either by:

- Using the **Keep With** and **Chronology** fields on the **Event Handlers** form.
- Or
- Moving your own adjacent event handlers up or down in the sequence, using the **Event Handler Sequence** form.

The following is a high-level overview of a typical process for altering the default order in which event handlers execute using the **Keep With** and **Chronology** fields on the **Event Handlers** form:

Step	How to alter the order of a handler using the Keep With and Chronology fields
1	Select an event handler In the grid view, select an event handler that has your Access As value.
2	Determine the event handler you want to use as a reference point and anchor for the selected handler. Select that reference event handler from the Keep With drop-down list. Note: This step is not required if you are using the First or Last option in the Chronology field.
3	Select the option you want the current handler to use from the Chronology drop-down list. This sets what the current handler will use with respect to the handler you selected in Step 2. Options include: • First – Executes the current handler before any other handlers. • Before – Executes the current handler just before the referenced handler. • Instead – Executes the current handler in place of the referenced handler. In this case, the referenced (original) handler does not execute at all. • Exclusively Instead – Executes the current handler instead of the referenced handler and any other handler that may be referenced to execute instead of that same handler. • After – Executes the current handler just after the referenced handler. • Last – Executes the current handler after all other handlers.



For more information about reordering event handlers, including some detailed examples, see Appendix D, Synchronization of Metadata.

Resequencing event handlers

Use the **Event Handler Sequence** form to change the sequence in which your adjacent event handlers execute for a specified event.



The **Event Handler Sequence** form is intended to be accessed as a linked form, from the **Event Handlers** form only, using the **Resequence** button. If you open it in standalone mode, the results can be unpredictable.

The Access As identifier must be considered when resequencing handlers. Here are some tips that restrict when you can use the **Event Sequence** form:

- Handlers that have the same Access As value:
 - Yes, you can use the **Event Handler Sequence** form to change the order of event handlers.
 - The handlers must be grouped together (that is, adjacent to one another in the sequence), and within the group.
- Handlers that have a different Access As value:
 - No, you cannot use this form to change the sequence of event handlers.
 - To change the order of your event handlers with respect to those of others (that is, with different Access As values), use the **Keep With** and **Chronology** fields on the **Event Handlers** form.

The following is a high-level overview of a typical process for changing the sequence in which event handlers execute using the **Event Handlers Sequence** form:

Step	How to change the sequence of a handler using the Event Handlers Sequence form
1	Open an event handler In the grid view, select an event handler that has your Access As value.
2	Click the Resequence button on the Event Handlers form
3	Move the handler up or down in the sequence Click the Up or Down button to move the selected handler up or down in the sequence, keeping in mind the restrictions mentioned above. Note: If you try to violate those restrictions, the system generates an error and you cannot complete the move.
4	Save the new sequence

Ordering event actions

A single event handler can contain multiple event actions. When you define an event action, you must assign an action sequence number to it (in the **Action Sequence** field of the **Event Actions** form). You can assign any number you want in that field, and the system automatically sorts the actions in the correct sequence on the **Event Handlers** form. When the event handler executes, the system uses this Action Sequence number to determine the order in which the actions execute.

The only exception to this rule is that, if you select an action (other than 1) in the **Initial Action** field of the **Event Handlers** form, then processing of the event actions begins with the designated action and proceeds from there.

Example ► Using the Initial Action field

Let's say you have four event actions associated with a handler, and have numbered them 1, 2, 3, and 4.

You later decide you want action number 3 to be the starting point. You select 3 in the **Initial Action** field of the **Event Handlers** form. In this case, only actions 3 and 4 would execute. The system would skip over actions 1 and 2.

Note: You could later execute these actions by using an action type of Branch or Goto, with one of these actions as the destination.

End of example

Determining names of IDO collections and components

To create custom event handlers and actions, you are often required to know the internal names of the collections (known internally as "IDO collections") and the components you want to refer to.

For example, to set up a handler, you often need the name of the IDO collection associated with a particular form. To include dynamic content in the subject or body of a message, you often must know the internal name of a property within that IDO.

But what if you do not know those names? How can you find what you need?

Determining the name of an IDO

The following is a high-level overview of a typical process for determining the name of an IDO for an event handler definition:

Step	How to determine the name of an IDO
1	Open the form that uses the IDO collection you need For example, if you are setting up an event handler to work with customer records, you would open the Customers form.
2	Enter design mode for that form
3	Locate the Form properties sheet If it is not showing, from the View menu, select the Form Properties option.
4	Select the Collections tab The names of all IDO collections associated with that form are displayed in the Collections list at the top of the form. Usually, there is only one collection, which makes it easy to figure out. For example, the Customers form uses the SL.SLCustomers IDO. The name of the IDO collection is SLCustomers. If more than one collection is listed, you might have to take further steps to determine which collection is the one you need.



The internal name of the IDO is that part of the IDO name that displays after the period.

Determining the name of an IDO property

The following is a high-level overview of a typical process for determining the name of an IDO property:

Step	How to determine the name of an IDO property
1	Open the form that has the field or other component that displays the data to which you want to refer. For example, to create a subject line that includes a customer's ID number and name, you would open the Customers form.
2	Select design mode for that form
3	Locate the Component properties sheet If it is not showing, select the Component Properties option from the View menu.
4	On the form, select the component for which you require the property name. For example, to determine the customer ID number, select the drop-down list box to the right of the Customer field label
5	On the Component properties sheet, locate the Binding field in the Data Source section. The component's internal name displays in that field, after the period. For example, for the customer ID number, the Binding field displays object.CustNum. Thus, the customer ID field's IDO property name is CustNum.



Discarding the metadata cache

Metadata is a set of data that describes and gives information about other data.

As you use the Application Event System, certain summary event metadata is cached in memory for faster performance. You should discard the metadata cache periodically. This should be done, at a minimum, after doing development work, before testing, and after synchronizing the metadata on your system. Ideally, you should discard the IDO metadata cache after making changes to events, handlers, actions, triggers, or global constants.

Here are some considerations for discarding the cache:

- Any event metadata that is not referenced within two minutes is automatically discarded from the cache. That is why you might notice that things work the way you expect, even without manually discarding the cached metadata. We still recommend that, as a precaution, you manually discard the cached metadata to be sure.
- If you have multiple utility servers in your system, you must discard cached metadata for each utility server on which metadata might have been cached. The best way to do this is to unload the global objects.

There are four ways you can discard the cached metadata:

- Unload global objects by pressing **Ctrl+U**
- Restart the IDO Runtime Service on the utility server
- Manually from the IDO Runtime Development Server
- Use the **Discard Cache** button on the **Utilities** tab of the Configuration Manager

We'll look at the first three methods in more detail.

Unload global objects

With the correct system settings, you can have the system discard the cached metadata automatically every time you unload all global objects.

Here is how to change the settings and the shortcut for unloading global objects:

If you want to...	Then do this...
Change settings to be able to discard the cached metadata when you unload global objects	Select User Preferences from the View menu in WinStudio. Click the Runtime Behaviors tab. Select the Unload IDO Metadata With Forms check box. Click OK .
Unload global objects (and discard cached metadata)	Click Ctrl+U .

While working on event metadata, periodically unload global objects, and the cached metadata is automatically discarded with the global objects.

Restart the IDO Runtime Service

You can discard the cached metadata by manually stopping and restarting the IDO Runtime Service on the utility server. This works because the metadata cache is not saved when the service is stopped.

Here is how to discard cached metadata by restarting the IDO Runtime Service:

If you want to...	Then do this...
Stop the IDO Runtime Service	Open Control Panel on the utility server. Select Administrative Tools > Services . Double-click Infor Framework IDO Runtime Services . Click Stop in the Infor Framework IDO Runtime Services Properties dialog box.
Restart the IDO Runtime Service	Verify that the IDO Runtime Services Properties dialog box indicates that the service has stopped. Click Start .

Manually from the IDO Runtime Development Server

If you are using a local installation of the IDO Runtime Development Server, you can discard the cached metadata manually.

Here is how to manually discard cached metadata in the IDO Runtime Development Server:

If you want to...	Then do this...
Discard cached metadata in the IDO Runtime Development Server	Open the IDO Runtime Development Server . Select the configuration for which you want to discard the cached IDO and event metadata. Select Discard IDO Metadata Cache from the Configuration menu.



If you are creating and using your own custom events and handlers, we recommend that you discard the metadata cache periodically. This should be done after doing development work, before testing, and after synchronizing your metadata on your system.

New Workflow Wizard form

The New Workflow Wizard is new to CloudSuite 9. You can use this wizard in place of the **Event Handlers** form when setting up simple notification events. The wizard cannot support events for complex notifications or for those that use prompt-type actions such as approval notices.

Creating workflows with the wizard

Users such as purchasing managers or executives can use the New Workflow Wizard to quickly create workflows (event handlers and actions) for some basic notification scenarios. For example, a purchasing manager can build a workflow to "Notify me when a PO line is created for more than \$1000."

As you make simple choices in the wizard, behind the scenes the system is creating event handler and event actions metadata that can later be viewed or edited in the standard **Application Event System** forms.

Here is how to build a simple notification workflow:

On the first page of the New Workflow Wizard, specify one of these options:

Option	Explanatory details
Notify Me When	Select this radio button if you want to assign the people who should receive a notification when the specified change is made to the system. For example, if the sales manager and the credit manager should receive notification when anyone adds a new customer, use this option.
Notify You When	Select this radio button if the originator (the person who makes or causes the specified change) should receive a notification in their CloudSuite Inbox. For example, if a salesperson should receive notification when they add a new customer to the system, use this option.
Build from Scratch	Select this option and click Finish to exit the workbench and use the standard forms to build the workflow.
Customize Resulting Workflow	Select the Customize Resulting Workflow check box if you want to open the event handler you created in the standard forms when you finish building the workflow through the wizard. Note: This option is enabled only if you selected one of the Notify options.
Next	Click Next to accept your selections and move to the next page of the wizard.

On the second page of the wizard, specify these options:

Option	Explanatory details
Description	Type a description of the workflow. Note: You can change the description on later pages.
IDO	Type a name in the IDO field to specific the IDO that you want associate with the event handler. Note: This is a required field
Form	Type a name in the Form field (or select from the drop-down list) to restrict the triggering conditions by specifying the form where the changes are made in addition to the IDO. If you leave the Form field blank and/or select the All Forms check box, the workflow will apply to all forms that use the specified IDO. Note: If you specify a form name before specifying an IDO, the IDO field is automatically filled in. This can be helpful if you know the form but not the IDO.
All Forms	If you select the All Forms check box, the workflow will apply to all forms that use the IDO you specified in the IDO field.
All IDOs	If you select the All IDOs check box, the workflow will apply to all IDOs associated with form you specify in the Forms field.
What to track	Select a radio button for Additions, Changes, or Deletions to set the type of operation on that IDO that will trigger the notification.
Next	Click Next to accept your selections and move to the next page of the wizard.

On the third page of the wizard, specify these options:

Options	Explanatory details
All Rows	In the Filter area, select the All Rows radio button to trigger the notification on any change to the IDO.
Where	In the Filter arear, select the Where radio button and then specify IDO property and value filter criteria if you only want a notification triggered when the change is for a row that matches the filter criteria.
Clauses	To specify multiple filter clauses, specify whether to use And or Or to join the new clause with the preceding one, and click Add . For each clause, you can specify one of these options: - Property, operator, and value - Property, operator, and another property, with an optional date-time adjustment Property, operator, and the date/time when the workflow runs, with an optional adjustment.

Options	Explanatory details
Finish	Click Finish to complete the workflow. Note: If you selected Customize Resulting Workflow , the Event Handlers form is launched for this workflow's event handler.



Exercise 4.1: Using the New Workflow Wizard

In this exercise, you will use the New Workflow Wizard to create a workflow that sends a notification to your Inbox when a new purchase order line is added where the cost is more than \$1000.

Exercise steps

Part 1: Create the event handler

1. Open the **New Workflow Wizard** form. The first page of the wizard displays.
2. Select the **Notify Me When** radio button.
Note: It may already be selected.
3. Click **Next**. The second page of the wizard displays.
4. Type *Notify me when new PO line cost >\$1000* in the **Description** field.
5. Type *SLP** in the **IDO** field.
6. Select **SLPoltms** from the **IDO** drop-down list.
7. Select the **Additions** radio button.
8. Click **Next**. The third page of the wizard displays.
9. Select the **Where** radio button.
Note: The caption for the IDO you selected on the previous page is displayed, i.e., Purchase Order Items.
10. Select **DerExtItemCostConv** from the **Purchase Order Items** drop-down list.
11. Select **>** from the **Operator** drop-down list.
12. Type *1000* in the **Value** field.
Note: Do not specify separators or currency symbols.
13. Click **Add This Clause**. The text "Extended cost > 1000" is displayed.
14. Click **Finish**. A dialog box displays with the message, "At least one Workflow was created."
15. Click **OK**.

Part 2: View the event handler details

1. Open the **Event Handlers** form.
2. Type *Notify me when new PO** in the **Handler Description** field.
3. Click **Filter in Place**.

4. Select the **IdoOnItemInsert** record that you just created. The record opens for the handler you just created.
5. Right-click **IdoOnItemInsert** in the **Event Name** field in the record. A menu displays.
6. Select **Details**. The **Events** form opens.
7. Verify the description of the IdoOnItemInsert event is “Occurs when an IDO inserts a row.”
8. Close the **Events** form.
9. Verify that your student ID appears in line 3 of the Event Actions grid, as you are the person set up to be notified when the event fires. If you do not see your student ID, tell your instructor.

Part 3: Test the workflow to see if it triggers the event

1. Click **Ctrl+U**.
2. Open the **Purchase Order Lines** form.
3. Select **Planned** from the **Status** drop-down list.
4. Click **Filter in Place**. The record for Bicycle Parts Company (PO 51) opens.
5. Click **Create a new object in the current collection**.
6. Select **FA-10000 Bicycle, Model – 30-26** from the **Item** drop-down list. The cost of 173.17511 populates the **Item Cost** field.
7. Type **10** in the **Ordered** field.
8. Verify that 1731.75 is displayed in the **Extended Cost** field.
Note: This value is > 1000 and should trigger the event handler to fire.
9. Click **Save**.
10. Open the **Inbox** form.
11. Verify that a message with the subject “A new row was added to Purchase Order Items” appears in the grid.
12. Close all forms.



It is good practice to also test that an event handler does not fire when a new row is added that does not meet the value.



Notification of changes to an existing record

We have seen how to build events and handlers for simple notifications. Now we will practice making a minor change to a notification.

This exercise is similar to exercise 3.4, except that, instead of notifying the credit manager when a customer is added to the database, we want to create an event and handler that notifies the credit manager whenever a customer's credit limit is changed.

As with the first scenario, we can use an existing framework event. This time we will use `IdoOnItemUpdate` and create our own handler for it. And again, because we are simply sending out a notification and the system is not waiting for a response from the credit manager, we can make it an asynchronous event handler.



Exercise 4.2: Changes to a credit limit

In this exercise, you will create an event handler with two actions: one to check whether the Credit Limit field has been changed and one to send the notification message to the credit manager.

Exercise steps

Part 1: Create the event handler

1. Open the **Event Handlers** form.
2. Click **Filter in Place**.
3. Click **Create a new object in the current collection**.
4. Select **IdoOnItemUpdate** from the **Event Name** drop-down list.
5. Type *Notify of Credit Limit Change* in the **Handler Description** field.
6. Type *SLCust** in the **Applies to Objects** field.
7. Select **SLCustomers** from the **Applies to Objects** drop-down list.
8. Clear the **Synchronous** check box.
9. Click **Save**.

Part 2: Create the first event action to check the credit limit

1. Select **<the Event Handler>** you created in Part 1.
2. Click **Event Actions**. The **Event Actions (Linked)** form displays.
3. Select **Finish** from the **Action Type** drop-down list.
4. Click **Save**.
5. Click **Edit Parameters**. The **Event Action Finish (Modal)** form displays.
6. Click **Condition**. The **Event Action Parameter Condition (Modal)** form displays.
7. Click **Expression 1**. The **Event Action Expression Editor (Modal)** form displays.

8. Select **PROPERTYMODIFIED** from the **Select a Function** drop-down list. Additional fields associated with the function are displayed.
9. Select **CreditLimit** from the **Argument 1** drop-down list.
10. Click **OK**. The **Event Action Expression Editor (Modal)** form closes.
11. Select the **Not** check box.

Note: This tells the system that if the credit limit is not modified, then finish, otherwise proceed to the next action.

12. Click **OK**. The **Event Action Expression Editor (Modal)** form closes.
13. Click **OK**. The **Event Action Parameter Condition (Modal)** form closes.
14. Click **OK**. The **Event Action Finish (Modal)** form closes.
15. Click **Save**.

Note: You may need to first click in a text box to activate the form so it can be saved.

Part 3: Create the second event action to send the notification

1. Click **Create a new object in the current collection**.
2. Select **Notify** from the **Action Type** drop-down list.
3. Click **Save**.
4. Click **Edit Parameters**. The **Event Action Notify (Modal)** form displays.
5. Click **To**. The **Event Action Parameter Recipients (Modal)** form displays.
6. Select **<your user ID>** from the **Recipients** list.
7. Click **Update**.
8. Click **OK**. The **Event Action Parameter Recipients (Modal)** form closes.
9. Type *Credit Limit Change* in the **Subject** field.
10. Type *Financial* in the **Category** field.
11. Click **Body**. The **Event Action Expression Editor (Modal)** form displays.
12. Select **SUBSTITUTE** from the **Select a function** drop-down list. Additional fields display.
13. Type *The credit limit has been changed to \${0} for customer {1}, customer number {2}.* in the **Argument 1** text box.

Note: Be sure to use the curly brackets.

14. Click in **row 1** of the **Arguments** grid.
15. Click **Build Expression**. The **Event Actions Expression Editor (Modal)** form displays.
16. Select **FILTERPROPERTY** from the **Select a function** drop-down list.
17. Select **CreditLimit** from the **Argument 1** drop-down list.
18. Click **OK**. The **Event Actions Expression Editor (Modal)** form closes.
19. Click in **row 2** of the **Arguments** grid.
20. Click **Build Expression**.

21. Select **FILTERPROPERTY** from the **Select a function** drop-down list.
22. Select **Name** from the **Argument 1** drop-down list.
23. Click **OK**.
24. Click in **row 3** of the **Arguments** grid.
25. Click **Build Expression**.
26. Select **FILTERPROPERTY** from the **Select a function** drop-down list.
27. Select **CustNum** from the **Argument 1** drop-down list.
28. Click **OK**.
29. Click **OK**. The full expression including the substitution properties are now displayed in the **Body** text box.
30. Click **OK**.

Part 4: Save the event actions

1. Click **Show Details**.
2. Click **Check Syntax**. A dialog box displays with the message "No syntax errors were found."
3. Click **Save modified data and close form**.
4. Click **OK**.
5. Click **Ctrl+U** to discard the metadata cache.

Part 5: Test that the event fires when a credit limit is changed

1. Open the **Customers** form.
2. Click **Filter in Place**. The **Coordinated Bicycles** record displays.
3. Click the **Credit** tab.
4. Type **200000** in the **Credit Limit** field.
5. Type **200000** in the **Order Credit Limit** field.
6. Click **Save**.
7. Open the **Inbox** form.
8. Verify that a message was received with the subject "Credit Limit Change" and that the body of the message text indicates that the credit limit was changed to \$200,000.

Part 6: Test that the event does not fire when something other than credit limit is changed

1. Click the **Main** tab to display the **Coordinated Bicycles** record.
2. Type **99999** in the **Postal/ZIP** field.
3. Click **Save**.
4. Click the **Inbox** tab to display the inbox.
5. Click **Refresh**.
6. Verify that no new messages were received.

Note: This second test shows that the event does not send messages for other changes, only for credit limit changes.



Notification that includes an old and new value

Notifications can range from the very simple, to increasingly complex. In the next exercise, we will practice making a moderately complex change to a notification.

In this scenario, we want to notify the inventory stock clerk automatically whenever an item's lot size changes. In the message that is sent, we want to include both the previous lot size and the new lot size. We also want to let the stock clerk know who initiated the change.

Once again, we will use an existing framework event, `IdoOnItemUpdate`, and create our own handler for it. Because this handler needs to retrieve the "before" property value, we must make it synchronous, so that during handler execution we can retrieve from the database the original row that is being updated, before it is updated by the IDO request.

This event handler requires three actions:

- One to check whether the **Lot Size** field has been changed and, if not, finish.
- One to retrieve the row being updated and both the original and new values for the **Lot Size** field.
- One to send the notification to the inventory stock clerk.



Exercise 4.3: Changes to an item lot size

In this exercise, you will create a new event handler with three event actions. The first action checks for changes in lot size, the second retrieves both the old and new lot sizes, and the third sends a notification message.

Before you begin:

- Ensure you have completed Exercise [X.X] because it provides data or configurations for this exercise.
- Ensure you are logged in to Configuration Console and it is open to the Configure application page. If not, refer to Exercise 1.2, part 1.
- [Add other prerequisite or note, as needed].

Exercise steps

Part 1: Create the event handler

1. Open the **Event Handlers** form.
2. Click **Filter in Place**.
3. Click **Create a new object in the current collection**.
4. Select **IdoOnItemUpdate** from the **Event Name** drop-down list.
5. Type *Notification of Item Lot Value* in the **Handler Description** field.
6. Select **SLItems** from the **Applies to Objects** drop-down list.

Note: You may want to narrow the choices in the drop-down list by first typing *SLItem** in the **Applies to Objects** field.

7. Click **Save**.

Part 2: Create the event global constant for stock clerks

1. Open the **Event Global Constants** form.
2. Click **Filter in Place**.
3. Click **Create a new object in the current collection**. A new **Event Global Constants** record displays.
4. Type *StockClerk* in the **Name** field.
5. Type *<your user ID>* in the **Value** field.
6. Click **Save**.
7. Close the **Event Global Constants** form.

Part 3: Create the first event action to check the lot size

1. Open the **Event Handlers** form.
2. Select the Event Handler created in Part 1.
3. Click **Event Actions**. The **Event Actions (Linked)** form displays.
4. Select **Finish** from the **Action Type** drop-down list.
5. Click **Save**.
6. Click **Edit Parameters**. The **Event Action Finish (Modal)** form displays.
7. Click **Condition**. The **Event Action Parameter Condition (Modal)** form displays.
8. Click **Expression 1**. The **Event Action Expression Editor (Modal)** form displays.
9. Select **PROPERTYMODIFIED** from the **Select a Function** drop-down list.
10. Select **LotSize** from the **Argument 1** drop-down list.

Note: Alternatively, you may type *LotSize* in the text field.

11. Click **OK**. The **Event Action Expression Editor (Modal)** form closes.
12. Select the **Not** check box.

Note: This tells the system that if the lot size is not modified, then finish, otherwise proceed to the next action.

13. Click **OK**. The **Event Action Expression Editor (Modal)** form closes.
14. Click **OK**. The **Event Action Parameter Condition (Modal)** form closes.
15. Click **OK**. The **Event Action Finish (Modal)** form closes.
16. Click **Save**.

Note: You may need to first click in a text box to activate the form so it can be saved.

Part 4: Create the second event action to send the retrieve the old and new lot size values and store it into a variable

1. Click **Create a new object in the current collection**.
2. Select **Load IDO Row** from the **Action Type** drop-down list.
3. Click **Save**.

4. Click **Edit Parameters**. The **Event Action Load IDO Row (Modal)** form displays.
5. Type *SLItems* in the **IDO** field.
6. Type *LotSize* in the **Properties** field.
7. Click **Filter**. The **Event Action Expression Editor (Modal)** form displays.
8. Select **SUBSTITUTE** from the **Select a function** drop-down list.
9. Type *Item = {0}* in the **Argument 1** field.
Note: Be sure to use the curly brackets.
10. Click in **row 1** of the **Arguments** grid.
11. Click **Build Expression**. A second **Event Action Expression Editor (Modal)** form displays.
12. Select **FILTERPROPERTY** from the **Select a function** drop-down list.
13. Select **Item** from the **Argument 1** drop-down list.
14. Click **OK**. The **Event Action Expression Editor (Modal)** form closes.
15. Click **OK**. The full expression including the substitution properties are now displayed in the **Filter** text box.
16. Click **Output**. The **Event Action Output Parameters (Modal)** form displays.
17. Click in **row 1** of the grid under the **Output Type** heading.
18. Select **Return Variable** from the **Output Type** drop-down list.
19. Click in **row 1** of the grid under the **Output Object Name** heading.
20. Type *OldLotSize* in the **Output Object Name** field.
21. Click in **row 1** of the grid under the **Value** heading.
22. Type *LotSize* in the **Value** field.



This action loads the record of the item being changed, takes the lot size of that record, and assigns it to a variable called OldLotSize.

23. Click **OK**.
24. Click **OK**. The **Event Action Load IDO Row (Modal)** form closes.
25. Click **Show Details**. You will see parameters as shown in the following table:

Parameters
IDO("SLItems")
PROPERTIES("LotSize")
FILTER(SUBSTITUTE("Item = {0}", FP("Item")))
SET(RV(OldLotSize) = "LotSize")

26. Click **Save**.

Part 5: Create the third event action to send the notification

1. Click **Create a new object in the current collection**.
2. Select **Notify** from the **Action Type** drop-down list.
3. Click **Save**.
4. Click **Edit Parameters**.
5. Click **To**.
6. Click **Recipients**.
7. Select **GC** from the **Select a function** drop-down list.
8. Select **StockClerk** from the **Argument 1** drop-down list.
9. Click **OK**.
10. Click **OK**. The **Event Action Parameter Recipients (Modal)** form closes.
11. Click **Cc**.
12. Click **Recipients**.
13. Select **ORIGINATOR** from the **Select a function** drop-down list.
14. Click **OK**.
15. Click **OK**. The **Event Action Parameter Recipients (Modal)** form closes.
16. Type *Lot Size Change* in the **Subject** field.
17. Type *Change Notification* in the **Category** field.
18. Click **Body**.
19. Select **SUBSTITUTE** from the **Select a function** drop-down list.
20. Type *The lot size changed for item: {1}. The previous lot size was {2}, the new lot size is {3}. This change was made by {0}.* in the **Argument 1** text box.
21. Click in row 1 of the Arguments grid.
22. Click **Build Expression**.
23. Select **ORIGINATOR** from the **Select a function** drop-down list.
24. Click **OK**.
25. Click in row 2 of the Arguments grid.
26. Click **Build Expression**.
27. Select **FILTERPROPERTY** from the **Select a function** drop-down list.
28. Select **Item** from the **Argument 1** drop-down list.
29. Click **OK**.
30. Click in row 3 of the Arguments grid.
31. Click **Build Expression**.
32. Select **V** from the **Select a function** drop-down list.
33. Select **OldLotSize** from the **Expression 1** drop-down list.

34. Click **OK**.
35. Click in row 4 of the Arguments grid.
36. Click **Build Expression**.
37. Select **P** from the **Select a function** drop-down list.
38. Select **LotSize** from the **Argument 1** drop-down list.
39. Click **OK**.
40. Click **OK**. The full expression including the substitution properties are now displayed in the Body text box.
41. Click **OK**.

Part 6: Save the event actions

1. Click **Show Details**. You will see parameters as shown in the following table:

Parameters
CATEGORY("Change Notification") TO(CG(StockClerks)) CC(ORIGINATOR()) SUBJECT("Lot Size Change") BODY(SUBSTITUTE("The lot size changed for item: {1} The previous lot size was {2}, the new lot size is {3}. This change was made by {0}.", ORIGINATOR(), FP("Item"), V(OldLotSize), P("LotSize"))) SAVEMESSAGE(FALSE)

2. Click **Save**.
3. Click **Check Syntax**. A dialog box displays with the message "No syntax errors were found."
4. Click **Save modified data and close form**.
5. Click **OK**.
6. Click **Ctrl+U** to discard the metadata cache.

Part 7: Test that the event fires when a lot size is changed

1. Open the **Items** form.
2. Click **Filter in Place**. The **30' Hurricane – 30F Floor Plan** item record is displayed.
3. Type 3 in the **Lot Size** field.
4. Click **Save**.
5. Open the **Inbox** form.
6. Verify that a message was received with the subject "Lot Size Change" and that the body of the message text indicates the old and the new lot size.

Check your understanding



What functionality allows you to change the execution order of your event handlers? Select all that apply.

- a) Keep With and Chronology fields
- b) Access As identifier
- c) Queue First field
- d) Event Handlers Sequence form



Sequence the tasks for determining the name of an IDO for a custom event handler in order from 1-4.



Tasks	Step number
Locate the Form Properties sheet	
Enter design mode	
Select the Collection tab	
Open the form that uses the IDO collection you need	



For which type of event can you set up event handlers with the New Workflow Wizard?

- a) All notification events
- b) Simple notification events only
- c) All prompt events
- d) Simple prompt events only



Lesson 5: Viewing event messages

Estimated time

.5 hours

Learning objectives

After completing this lesson, you will be able to:

- Discuss how event messages work with the Inbox form.

Topics

- About message-related forms
- Inbox form
- Variables in event messages
- Prompts and responses
- Saved Messages form
- Send Messages form

About message-related forms

Event message-related forms are used to view, sort, file, respond to, and send messages generated within the Application Event System.

The Application Event System uses these message-related forms:

- Inbox form
- Saved Messages form
- Send Message form



The Infor Application Event System can be configured to use external email systems such as Microsoft® Outlook® in addition to the system message forms.

Inbox form

The **Inbox** form can be used with the Application Event System or it can also be used to communicate with other users in your CloudSuite environment. The **Inbox** form displays system messages that can come from three possible sources:

- Application events that use a Notify action type
- Application events that use a Prompt action type
- Communications initiated and sent by individual users to other users on the system

The Inbox is not a full working email system. For example, you may notice that there is no way to compose a new message from the Inbox. (To send messages, you must go the **Send Message** form.) The Inbox is primarily a way to read and respond to messages.

If you have a new message in the Inbox, an envelope icon will appear in your system tray. You can click the icon to open the **Inbox** form. You can mark messages as read and respond to certain types of messages.

The **Inbox** form can be accessed in several different ways:

- From the **View** menu, by selecting the **Inbox Form** option
- In the Windows® taskbar (notifications area), by double clicking the Inbox notification icon
- In the same ways that any other forms are accessed

This form displays only those messages which are still in the recipient's Inbox folder and not messages that have been moved to other folders. To view those messages, recipients must use the **Saved Messages** form. The **Inbox** form also does not allow recipients to move messages to other folders. To do that, recipients must use the **Saved Messages** form.

When a message is received, if the recipient has set options to be notified, the system alerts the recipient with the selected notifications. **Note:** For more information, see "Notifications Settings" in the online help.

Recipients can mark messages as read and, depending on how and why the message was sent, make other responses.



Variables in event messages

Some messages in the Inbox involve variables. For each variable, recipients might be able to:

- Provide an optional response
- Provide a mandatory response
- Only read the variable value
- Not see the variable value at all

Setting variable access

The effective visibility and writability of each variable displayed on the **Inbox** form is determined by two things:

- What type of action generated the message (Notify or Prompt)
- The (optional) variable access level as specified:
 - On the **Event Actions** form for the action itself and/or
 - On the **Event Variable Groups** form for the event handler's initial state and/or
 - On the **Event Action Notify** or **Event Action Prompt** form for the action about the payload status of the property that corresponds to the variable, when the handler is associated with an IDO event.

For **Notify** type messages, the default for variable access is Read-Only. You can use the variable access options to override this to Hidden for each variable.

For **Prompt** type messages, the default for variable access is Writable. You can use the variable access options to override this to Hidden, Read-Only, or Mandatory for each variable.

To change the value of a variable that might appear with a message in the **Inbox** form, you can do any of these actions:

- Specify the data using the **Variables** tab on the **Inbox** form as part of a response to a Prompt action
- Use a Set Values event action with the function syntax: SETVARVALUES(PropertyName=name)
- Use any of various event actions that can set a variable on output using the function syntax: RV(PropertyName)

Setting translatable captions for variables

In the **Inbox** form's **Variable** tab, the **Caption** column displays the contents of the notify or prompt message's variable captions in the current user's language. This assumes that:

- The caption component attributes are set to interpret the bound contents.
- The caption contains a translatable string name.

For payload variables resulting from an IDO event, this string name might come from an IDO property's label string ID. For non-payload variables that are created by an event action, you define this string name using the **Event Actions** form, **Variable Access** tab.

Prompts and responses

If the message in the **Inbox** form is the result of a system-generated prompt, each recipient can respond to the prompt. In such cases, the system must be set up to do the following:

- Wait for responses
- Handle responses as they are received
- Act when responses are not received within a specified timeout period

Message prompts typically display a set of choices for the recipient, usually by means of a set of voting buttons. These buttons display in the **Response** tab area of the **Inbox** form. For example, a prompt might include buttons labeled:

- Approve / Disapprove (default option)
- Yes / No
- OK / Send More Info / Cancel

Prompts and external email

Prompts that are sent to external email addresses do not contain voting buttons. Instead they contain links that the recipient can use to respond.

If the message was sent both to the recipient's **Inbox** form and to external email, and the recipient responded to the email message, the message in the refreshed Inbox is marked as expired and the buttons are inactive, so the recipient cannot respond twice.



Lesson 11 provides further details on using external email with the Application Event System.

Setting up a prompt with choices

To customize the choices for a prompt, you must include a Prompt action with a Choices parameter as part of the event action definition. This Choices parameter consists of the CHOICES function followed by a string expression that evaluates to a comma-separated list that contains an even number of elements (value/label pairs). For example, if you want the voting buttons to be labeled Yes and No, with corresponding values returned to the action to be 1 and 0, you could include this parameter:

CHOICES("1,sYes,0,sNo")

In this example, the strings "sYes" and "sNo" are WinStudio form strings. These have already been defined for the system as Yes and No, respectively.

If you want your button labels to be localized, for the recipient, you must:

- Use names of existing form strings (as found in the Strings table).

Or

- Add your own form strings, using the **Strings** form, and provide the necessary translations. **Note:** To open the **Strings** form, you must be in Design Mode and, from the **Edit** menu, select **Strings**.

If localization is not an issue, you can also use a literal value that displays on the button verbatim. To specify the string as a literal value here, simply specify it as a list value. If the system does not find the string in the Strings table, the system automatically treats it as a literal value.

Saved Messages form

The **Saved Messages** form displays messages that you have saved or at least have not yet deleted for a selected folder. Within this folder, you can sort messages by any one of a number of criteria.

You can also use this form to:

- Create your own folders.
- Move messages from one folder to another.

You can also view messages on this form for information about any responses you might have made to messages generated by the system.



Folders (e.g., Inbox, Saved Messages, and Send Messages folders) are not represented visually as they typically are in email programs. The only place you can view your personal folders is in the **Folder Name** field on this form.

Moving messages between folders

You can use the **Saved Messages** form to move messages from one folder to another. If the folder does not already exist, you can create the folder at the same time.

Here's how to move a message from one folder to another folder:

If you are going to ...	Then ...
Select a message to move	• Open the Saved Messages form. • In filter in place mode, type the folder name into the Folder Name field. • Click Filter in Place. The system displays in the grid view all messages in that folder. • Select the message you want to move.
Move a message	• In the Folder Name field, do one of these actions: ○ For an existing folder, type the name of the folder or select the folder from the drop-down list. ○ To create a new folder and move the message to that folder, specify the name of the new folder. • Save the message.

Send Message form

The **Send Message** form is used to send system messages, similar to email messages, to other users in the system. You can designate multiple recipients, "carbon-copy" recipients, and instruct the system whether to save a copy in your Sent Items folder.



Check your understanding



Which of the following are sources for messages that display in the Inbox form? Select all that apply.

- a) Application events that use a Notify action type
- b) Application events that use a Prompt action type
- c) Communications sent by one system user to another
- d) Communications sent from external email to system users



Lesson 6: Tracking event system status

Estimated time

.5 hours

Learning objectives

After completing this lesson, you will be able to:

- Identify the forms used to track the event system status.

Topics

- Tracking status forms
- Event Status form
- Event Handler Status form
- Event Queue form
- Event and Event Handler Revisions forms
- Suspended Updates form

Tracking status forms

Some application events can take a considerable amount of time to process, especially if they involve event messages that require responses from the recipients. The system provides several tools and forms that allow you to track the status of application events as they execute and after they have finished executing. Some of these forms also allow you to temporarily adjust the behavior of handler execution.

These forms are located in the Explorer under Master Explorer > Modules > System > Event System.

These forms can all be used to track various aspects of event system status:

- Event Status form
- Event Handler Status form
- Event Queue form
- Event Revisions form
- Event Handler Revisions form
- Suspended Updates form

Initially, these forms can be accessed only by members of the System Administration authorization group.

Event Status form

Use the **Event Status** form to view:

- The status of events that are currently running.
- The history of events that have finished running.

The **Event Status** form has five tabs:

Tab name	Tab description
Event tab	This tab provides options to filter for events and handlers in various states and make it easier to locate specific ones. For example, suppose you want to find data about all events that are currently suspended. With Filter in Place, you would select the Suspended check box and then execute Filter-in-Place.
Handlers tab	This tab shows data about handlers that are currently processing or have finished processing.
Voting tab	This tab shows data about the voting status of a Prompt action type.
Parameters tab	This tab shows data about the parameters associated with an event that is currently running. These include input parameters passed into the event, as well as output parameters created by handlers' actions.
Output Parameters tab	This tab shows data about the output parameters of an event that has finished running.

The **Event Status** form can be helpful when you are not seeing the behavior you expected from an event. Here are things to search for in the Event tab:

- Records with a status of “failure” and have a finish time
- Records for which you are the originator
- Records that have a status of “running” and do not have a finish time



Demo 6:1: Event Status form

This demonstration shows how to open and use the Event Status form.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.

Event Handler Status form

Use the **Event Handler Status** form to view:

- The status of event handlers that are currently running.
- The history of event handlers that have finished running.

The **Event Handler Status** form has four tabs:

Tab name	Tab description
Handlers tab	This tab shows data about the event handler itself.
Actions tab	This tab shows only data about actions of the handler that have started.
Variables tab	This tab shows information about variables associated with the event handler.
Voting tab	This tab shows data about the voting status of a Prompt action type.

The **Event Handlers Status** form can be helpful when you want to review all the actions associated with the handlers as it will show which actions were completed or which were not completed. Look at the **Results** section on the **Handler** tab.



Demo 6:2: Event Handler Status form

This demonstration shows how to open and use the Event Handler Status form.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.

Event Queue form

The **Event Queue** form displays a list of all asynchronous events and event handlers that the system has queued for processing.

Events on the queue are processed in FIFO (first in, first out) order. The ID number on this form indicates the order in which events are queued for processing.

This form also displays other information about events and event handlers that have been queued for processing. Each event or event handler that has been queued is displayed as a separate record in the grid view.



All information on the **Event Queue** form is display only.



Demo 6:3: Event Queue form

This demonstration shows how to open and use the Event Queue form.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.

Event and event handler revisions

We learned that whenever an event or handler revision is revised, the system copies all the event's handlers and actions to a read-only table. The event or handler then uses the data in this read-only table when executing the handlers and actions, until a new revision is created.

There are two forms that contain display-only information about revisions:

- Event Revisions form
- Event Handler Revisions form

Event Revisions form

The **Event Revisions** form displays information about the event revisions associated with events.

Event Handler Revisions form

The **Event Handler Revisions** form shows information about event handler revisions associated with event handlers. All information on this form is display-only. The reason for this is that some running or finished handlers are using the metadata in this revision to complete their processing. To change the data for this event handler, you must use the **Event Handlers** form.



The **Event Handler Revisions** form can be opened by clicking the **Event Handlers Revision** button found on the **Event Revisions** form.

Event and event handler revisions

The system creates a set of event handler revisions the first time:

- The event is generated or any of its handlers executes.
- The event is generated after any of its constituent handlers or their actions have been modified in any way.
- Any of the event's handlers executes after any of its constituent actions have been modified in any way.

Modifications can include additions, changes, or deletions to the handlers or actions associated with the event or handler.

It is important to understand that these forms are display-only forms. You will not perform any action on these forms. They are reference forms that show you all revisions that exist in the system for a particular event or event handler.

When the revision is created, the system copies all the event's handlers and actions to a read-only table. This means that the events and event handlers are saved off and persist in the system. The event or handler then uses the data in this read-only table when executing the handlers and actions, until a new revision is created.

Revisions are used to help ensure that the metadata used by an event and its handlers and actions is not altered while the event is executing. In some cases, it can take the system some time to finish executing an event's handlers, either because of the processing time involved, or because the system is waiting for some input or response before it can continue. It is possible that someone could make changes to the

event's handlers and actions while the event is processing or waiting, and that these changes could affect the execution of the event that is in progress.

Revisions were developed to address this kind of potential problem. When an event fires, the event and its handlers use the revisions that are in effect at the time they start executing until they are finished.

Example ► Changing the person who will give manager approval

Suppose you are using an event to send notices to a manager when a customer order is more than \$10,000. The manager must approve the order before it can be processed.

During a transfer of responsibilities from one manager to another, a new manager is assigned the responsibility of approving such orders. The system administrator makes the change of notice to the appropriate event action.

However, the original manager still has a few orders pending approval. These orders continue to use the metadata for the revision in effect at the time they were created, awaiting that manager's approval. In the meantime, the new manager receives notices for any subsequent orders, because the first new order generates a new revision using the information for the new manager.

End of example



Demo 6:4: Event Revisions form and Event Handler Revisions form

This demonstration shows how to open and use the Event Revisions form and the Event Handler Revisions form.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.

Suspended Updates form

The **Suspended Updates** form is another status form. It displays a list of all the update actions that are currently in a suspended state. This form also allows you to take selected updates out of suspension manually.

Review of suspended records

On the **Event Handlers** form you can suspend a handler and it will wait for an adjourning action, which is an action that causes it to proceed. Typically, this is used for event handlers that require approval. If they choose to not approve a record, then we do not want the disapproved record committed in the system. Any record that has not been approved (i.e., that is either disapproved or has had no response) will be suspended. The **Suspended Updates** form displays all these records.

Example ► New purchase order record in suspense

Suppose you have created an event handler for new purchase orders that is a prompt type (you are requesting approval from a manager). You selected the **Suspend check** box on the **Event Handlers** form.

You then create a new purchase order, and the event handler fires off a request to the manager asking for manager approval. If the manager sends approval, the system will commit the record. But you wait for 24 hours and cannot find any record of the purchase order. You think that the manager either disapproved or did not respond at all, and the record is being held in a suspended state.

You open the **Suspended Updates** form and can locate the record. This is the only place in the system where you can view the record; however, you can also see the event actions that would be running, waiting for a response.

End of example



Demo 6:5: Suspended Updates form

This demonstration shows how to open and use the Suspended Updates form.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.

Check your understanding



Which form shows the history of events that have finished running?

- a) Event Queue form
- b) Event Revisions form
- c) Event Status form
- d) Suspended Updates form



The Event Revisions form and the Event Handlers Revision form are display-only forms.

- a) True
- b) False



Lesson 7: Requesting approvals - basic

Estimated time

.5 hours

Learning objectives

After completing this lesson, you will be able to:

- Explain how to set up an event that uses a prompt to request approval.

Topics

- About requesting approvals
- Event handler diagrams

About requesting approvals

Setting up an event handler to request approval requires using an action type we have not yet used called “prompt.” A prompt action asks (or prompts) and sends a message, but also asks the recipient to send a response to the message. This differs from notification actions that require no response.

In our next scenario, we want to send a message to the purchasing manager whenever a purchase order (PO) is added. This message prompts the purchasing manager for approval of the PO. The purchasing manager can indicate approval (or disapproval) by clicking a button in the message itself. If approved, the system adds the PO; if not, the system does not add the PO.

As with the previous exercises, the system framework has a built-in event, `IdoOnItemInsert`, which we can use with our handler. The handler itself requires two event actions: a “prompt” action to send the approval request to the manager, and a “fail” action to tell the system what to do if approval is denied.

You must tell the system how to evaluate the response received by the manager. This is done by adding an event action parameter condition for the function called `VOTINGRESULT` to the “fail” event action, and assigning values to “approved” or “denied” responses.



Exercise 7.1: Approving a purchase order

In this exercise, you will set up an event handler to request manager approval for all new purchase orders. This event handler will contain two event actions.

Exercise steps

Part 1: Create the global constant for Purchasing Manager

1. Open the **Event Global Constants** form.
2. Click **Filter in Place**.
3. Click **Create a new object in the current collection**.
4. Type *PurchasingMgr* in the **Name** field.
5. Type *<your user ID>* in the **Value** field.
6. Click **Save**.
7. Close the **Event Global Constants** form.

Part 2: Create the event handler

1. Open the **Event Handlers** form.
2. Click **Filter in Place**.
3. Click **Create a new object in the current collection**.
4. Select `IdoOnItemInsert` from the **Event Name** drop-down list.
5. Type *Approval of a New Purchase Order* in the **Handler Description** field.
6. Select `SLPos` in the **Applies to Objects** drop-down list.

Note: You can type *SLPo** in the **Applies to Objects** field to narrow the choices in the drop-down list.

7. Clear the **Synchronous** check box.
8. Click **Save**.

Part 3: Create the first event action to send the approval message

1. Select the Event Handler you created in Part 2.
2. Click **Event Actions**. The **Event Actions (Linked)** form displays.
3. Select **Prompt** from the **Action Type** drop-down list.
4. Click **Edit Parameters**. The **Event Action Prompt (Modal)** form displays.
5. Click **To**. The **Event Action Parameter Recipients (Modal)** form displays.
6. Click **Recipients**. The **Event Action Expression Editor (Modal)** form displays.
7. Select **GC** from the **Select a Function** drop-down list.
8. Select **PurchasingMgr** from the **Argument 1** drop-down list.
9. Click **OK**. The **Event Action Expression Editor (Modal)** form closes.
10. Click **OK**. The **Event Action Parameter Recipients (Modal)** form closes.
11. Type *New purchase order needs your approval* in the **Subject** field.
12. Type *Order Approval* in the **Category** field.
13. Click **Body**. The **Event Action Expression Editor (Modal)** form displays.
14. Select **SUBSTITUTE** from the **Select a function** drop-down list. Additional fields associated with this function display.
15. Type *A new purchase order has been requested for vendor {0}, {1}. Please review the details on the Variables tab and register your approval on the Response tab.* in the **Argument 1** text box.
16. Click in **row 1** of the **Arguments** grid.
17. Click **Build Expression**.
18. Select **P** from the **Select a function** drop-down list.
19. Select **VendNum** from the **Expression 1** drop-down list.
20. Click **OK**.
21. Click in **row 2** of the **Arguments** grid.
22. Click **Build Expression**.
23. Select **P** from the **Select a function** drop-down list.
24. Select **VendorName** from the **Expression 1** drop-down list.
25. Click **OK**.
26. The **Event Action Prompt (Modal)** form opens.
27. Type *Do you approve this new PO?* in the **Question** field.
28. Click **Choices**. The **Event Action Prompt Choices (Modal)** form opens.
29. Click in **row 1** under the **Return Value** heading. The field activates.
30. Type *1* in the **Return Value** text field.

31. Click in **row 1** under the **Button Caption** heading. The field activates.
32. Type sYes in the **Button Caption** text field.
33. Click in **row 2** under the **Return Value** heading. The field activates.
34. Type 0 in the **Return Value** text field.
35. Click in **row 2** under the **Button Caption** heading. The field activates.
36. Type sNo in the **Button Caption** text field.
37. Click **OK**. The **Event Action Prompt Choices (Modal)** form closes.
38. Click **OK**. The **Event Action Prompt (Modal)** form closes.
39. Click **Show Details**. You will see parameters as shown in the following table:

Parameters
CATEGORY("Order Approval") TO(GC(PurchasingMgr)) SUBJECT("New purchase order needs your approval") BODY(SUBSTITUTE("A new purchase order has been requested for vendor {0}, {1}. Please review the details on the Variables tab and register your approval on the Response tab.", P("VendNum"), P("VendorName"))) SAVEMESSAGE(FALSE) QUESTION("Do you approve this new PO?") CHOICES("1,sYes,0,sNo")

40. Click **Save**.

Part 4: Create the second event action to evaluate the recipient response and act based on that response

1. Click **Create a new object in the current collection**.
2. Select **Fail** from the **Action Type** drop-down list.
3. Click **Save**.
4. Click **Edit Parameters**. The **Event Action Fail (Modal)** form displays.
5. Click **Condition**. The **Event Action Parameter Condition (Modal)** form displays.
6. Click **Expression 1**.
7. Select **VOTINGRESULT** from the **Select a function** drop-down list.
8. Select **10 Prompt** from the **Action** drop-down list.
9. Click **OK**.
10. Select **=** from the **Operator** drop-down list.
11. Type 0 in the **Expression 2** text field.
12. Click **OK**. The **Event Action Parameter Condition (Modal)** form closes.
13. Type *The PO request was rejected by the Purchasing Manager.* in the **Result** text field.

14. Click **OK**. The **Event Action Fail (Modal)** form closes.

15. Click **Save**.

16. Close the **Event Actions** form.

Note: The **Event Handlers** form should be the only remaining form open.

17. Select the **Suspend** check box.

Note: This will suspend the new purchase order record until the recipient responds.

18. Click **Save**.

19. Click **Ctrl+U** to discard the metadata cache.

Part 5: Test that the event fires when a PO is added and does not get committed to the system until after being approved

1. Open the **Purchase Orders** form.

2. Click **Filter in Place**.

3. Click **Create a new object in the current collection**.

4. Select **1 Bicycle Parts Company** from the **Vendor** drop-down list.

5. Click **Save**.

6. Click **Refresh**. The record should not be showing.

7. Click **Filter in Place**.

8. Click the **Calendar** button in the **Date** field. The calendar displays.

9. Select **<today's date>** in the calendar.

10. Click **Filter in Place**.

Note: There should be no PO records for today, since the new record you created is suspended until a response is received.

11. Open the **Inbox** form.

12. View the message with the subject **"New purchase order needs your approval."**

13. Click the **Response** tab. The **Response** tab displays.

14. Click the **Yes** button to approve the PO.

Note: After you vote, the Yes and No buttons are disabled.

15. Close the **Inbox** form. The **Purchase Orders** form displays.

16. Click **Filter in Place**.

Note: Today's date should still be the filter setting.

17. Verify that the new PO record you created has been committed to the system.

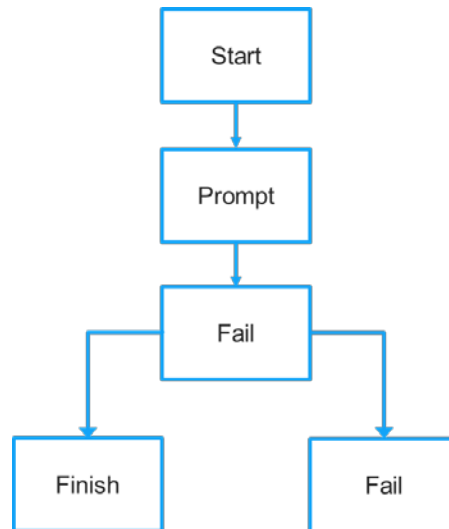
Event handler diagrams

You may find it easier to build or work with an event handler if you can visualize the workflow. You can access a flow chart diagram for any event handler by clicking the **Diagram** button on the **Event Handlers** form.

You can use the **Event Handler Diagram** form to:

- View a graphical (diagrammatic) representation of an event handler and its actions.
- Access the **Event Actions** form to modify event actions.
- Add event actions to the flow.
- Delete event actions from the flow.
- Edit the final diagram before copying, printing, or saving.
- Copy the diagram to the system clipboard.
- Print the diagram.
- Save the diagram as a graphics file.

For example, the diagram for exercise 7.1 looks similar to the diagram shown below:



Event handler with two event actions – prompt and fail

Accessing the Event Actions form to modify event actions

To access the **Event Actions** form for a selected event action in the flow:

- In the diagram display, double-click the node for the event action you want to edit.
- When the **Event Actions** form opens, edit the action as desired.
- Save your changes and close the form.
- When the system asks whether you want to update the event handler diagram with the latest changes, respond Yes.

Adding event actions

To add an event action to an event handler flow:

- Click the **Event Actions** menu bar on the left edge of the diagram display area.

- Drag and drop the new event action into the diagram display area.
- When the **Event Actions** form opens, use it to define the new action.
- When finished, save the new action and close the form.

Deleting actions

To delete an event action from an event handler flow:

- In the diagram display, select the node for the event action you want to delete.
- Right-click the node.
- From the context menu, select **Delete Event Action**.

Editing the diagram display

To edit the diagram display:

- Select the node that you want to move.
- Click and drag the node to the desired location.

Note: Any changes you make to the diagram are not persistent; that is, they only last if you make no more changes to the event handler flow. Therefore, if you want to make manual adjustments to the layout of the diagram before printing it, for example, these manual adjustments should be the very last thing you do.

Copying the diagram to the system clipboard

To copy a diagram to the system clipboard:

- Right-click anywhere inside the diagram display area.
- From the context menu, select **Copy to Clipboard**.
- Once the diagram is copied to the system clipboard, you can paste it into another program, such as MS Word®, PowerPoint®, or a graphics program.

Printing a diagram

To print a diagram:

- Right-click anywhere inside the diagram display area.
- From the context menu, select **Print**.
- The system prints the diagram on whatever printer is designated as the user's default printer.

Saving a diagram

To save a diagram as a graphics file:

- Right-click anywhere inside the diagram display area.
- From the context menu, select **Save To Image**.
- In the **Save As** dialog box, provide a name for the image file and then click **Save**.

Check your understanding



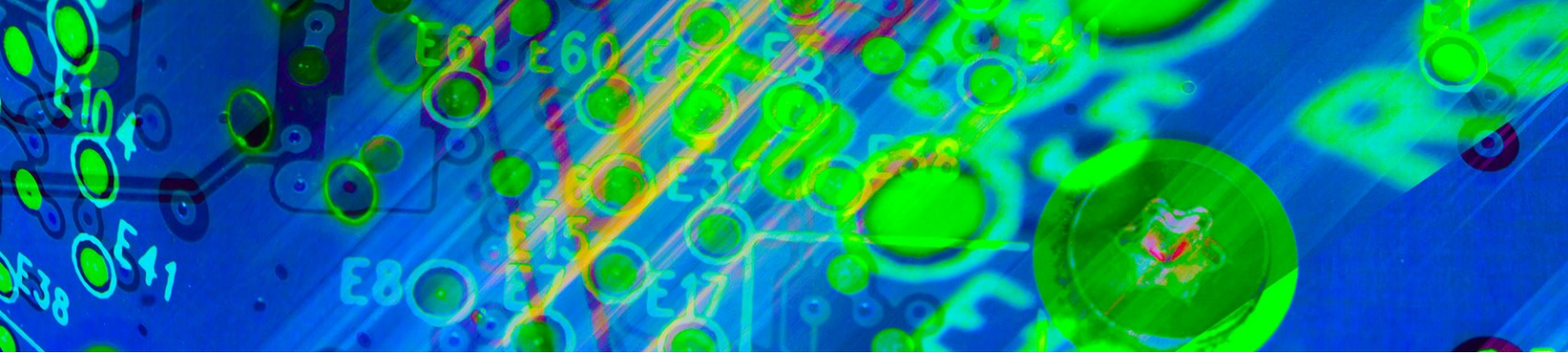
When setting up an event to request manager approval for all new purchase orders, what two action types are needed?

- a) Notify – to notify the manager of each new purchase order
- b) Prompt – to send the manager a message requesting a response
- c) Finish – to tell the system what to do when the approval is received
- d) Fail – to tell the system what to do when approval is denied



What event action parameter function tells the system how to evaluate all possible responses received from a request for approval?

- a) SUBSITUTE
- b) FILTERPROPERTIES
- c) VOTINGRESULT
- d) SETVALUES



Lesson 8: Adding information

Estimated time

.5 hours

Learning objectives

After completing this lesson, you will be able to:

- Explain how to set up an event to add information to a record.

Topics

- About adding information to a record

About adding information to a record

You can use the Application Event System to send a message (a prompt) asking a recipient to provide new information, such as a value. This new information can then be automatically added to an existing record.

When adding new information, the recipient can be given the ability to enter information by selecting from a set of choices (e.g., voting buttons) or as a free-form response, (e.g., by typing certain values). **Note:** Free-form functionality is only available when sending the prompt via the CloudSuite messaging service to a recipient's **Inbox** form.

If you want to...	Then...
Respond to a message requesting new information	• Open the Inbox form. • Open the message with the request. • Type a value on the Variables tab of the message. • Save the modified Inbox record. • Click the Post button on the Response tab to register the response.

You can define an event handler to send prompts for new information to external email addresses, but must provide a set of choices from which the recipient can choose, i.e., by voting for a selection. The system cannot accept free-form values from external email prompts.



Exercise 8.1: Providing a credit limit for a new customer

In this exercise, you will set up an event handler to fire when a new customer is added that sends prompt to the credit manager asking to respond with a credit limit for that customer. The system will use the data from the response to add data to the pending new customer record and then commit the changes to the database.

Exercise steps

Part 1: Create an event handler

1. Open the **Event Handlers** form.
2. Click **Filter in Place**.
3. Click **Create new object in the current collection**.
4. Type *IdoOnItemInsert* in the **Event Name** field.
5. Type *Prompting for a customer credit limit* in the **Handler Description** field.
6. Type *SLCus** in the **Applies to Objects** field.
7. Select **SLCustomers** from the **Applies to Objects** drop-down list.
8. Click **Save**.

Part 2: Create an event action

1. Select **<the Event Handler>** you created in Part 1.
2. Click **Event Actions**.
3. Select **Prompt** in the **Action Type** drop-down list.
4. Click **Edit Parameters**.
5. Click **To**.
6. Click **Recipients**.
7. Select **GC** from the **Select a function** drop-down list.
8. Select **CreditMgr** from the **Argument 1** drop-down list.
9. Click **OK**.
10. Click **OK**.
11. Click **Subject**.
12. Select **Substitute** from the **Select a function** drop-down list.
13. Type *Need credit limit for customer ID {0}, {1}* in the **Argument 1** field.
14. Click in **row 1** in the **Arguments** grid. The row activates.
15. Click **Build Expression**.
16. Select **FP** from the **Select a function** drop-down list.
17. Select **CustNum** from the **Argument 1** drop-down list.
18. Click **OK**.
19. Click in **row 2** in the **Arguments** grid. The row activates.
20. Click **Build Expression**.
21. Select **P** from the **Select a function** drop-down list.
22. Select **Name** from the **Argument 1** drop-down list.
23. Click **OK**.
24. Click **OK** again.
25. Type *Financial* in the **Category** field.
26. Click **Body**.
27. Select **Substitute** from the **Select a function** drop-down list.
28. Type *Please use the Variables tab to provide a credit limit for customer ID {0}, {1}. Then use the Post button on the Response tab to register the new credit limit.* in the **Argument 1** text box.
29. Click in **row 1** in the **Arguments** grid. The row activates.
30. Click **Build Expression**.
31. Select **FP** from the **Select a function** drop-down list.
32. Select **CustNum** from the **Argument 1** drop-down list.
33. Click **OK**.

34. Click in **row 2** in the **Arguments** grid.
35. Click **Build Expression**.
36. Select **P** from the **Select a function** drop-down list.
37. Select **Name** from the **Argument 1** drop-down list.
38. Click **OK**.
39. Click **OK** again.
40. Type *To post the credit limit, save the modified Inbox record and then click the button below.* in the **Question** field.
41. Click **Choices**.
42. Type **1** in the **Return Value** field in row 1.
43. Type *sPost* in the **Button Caption** field in row 1.
44. Click **OK**.
45. Select **Customers** from the **Filter Form** drop-down list.
46. Click **Filter Spec**.
47. Select **Substitute** from the **Select a function** drop-down list.
48. Type *CustNum={0}* in the **Argument 1** text box.
49. Click in **row 1** in the **Arguments** grid.
50. Click **Build Expression**.
51. Select **FP** from the **Select a function** drop-down list.
52. Select **CustNum** from the **Argument 1** drop-down list.
53. Click **OK** three times.
54. Click **Save**.
55. Click **Show Details**.
56. Click the **Variable Access** tab.
57. Type *CreditLimit* in the **Name** field.
58. Select **Mandatory** from the **Access** drop-down list.
59. Click **Save**.
60. Close the **Event Actions (Linked)** tab.
61. Select the **Suspend** check box.
62. Click **Save**.

Part 3: Test the handler

1. Click **Ctrl+U**.
2. Open the **Customers** form.
3. Click **Filter in Place**.
4. Click **Add a new object in the current collection**.

5. Type *<your user ID> Customer* in the **Customer Name** field which is located to the right of the **Customer** field and is not labeled.
6. Select **BK1 Universal Savings** from the **Bank Code** drop-down list.
7. Click the **Credit** tab.
8. Verify that the value in the **Credit Limit** field is **0**.
9. Click **Save**.
10. Click **Filter in Place**.
11. Type **<your user ID>** in the **Customer Name** field which is located to the right of the **Customer** field and is not labeled.
12. Click **Filter in Place**.
13. Verify there are no records resulting from this filter.
Note: The record for the new customer is set to suspend, meaning that and you should not be able to find the record in the list of customers.
14. Open the **Inbox** form.
15. Verify that the new message appears in the **Inbox**.
16. Click the **Variables** tab.
17. Type *<any number>* in the **Row.CreditLimit** variable.
18. Save the record.
19. Click the **Response** tab.
20. Click **Post**.
21. Open the customers form if the **Customers** form is not opened and filter for *sa Customer*.
22. Click **Refresh** if the **Customers** form is open.
23. Verify that the newly created record appears in the list, and contains the credit limit value that you entered on the **Inbox** form.

Check your understanding



Which of the following statements is true about defining an event action that prompts a recipient to send information to add to a record?

- a) The message text cannot exceed 100 characters
- b) Prompts sent to the Inbox form can support free-form responses
- c) The information received must be manually added to the record
- d) Prompts cannot include voting buttons



Sequence the steps that a recipient takes to respond to an Inbox prompt for new information in order from 1-4.



Step description

Step number

Type a value on the Variables tab

Open the message

Open the Inbox form

Click the Post button on the Response tab



Lesson 9: Requesting approvals - advanced

Estimated time

1 hour

Learning objectives

After completing this lesson, you will be able to:

- Explain how to set up an event to request multiple levels of approval.

Topics

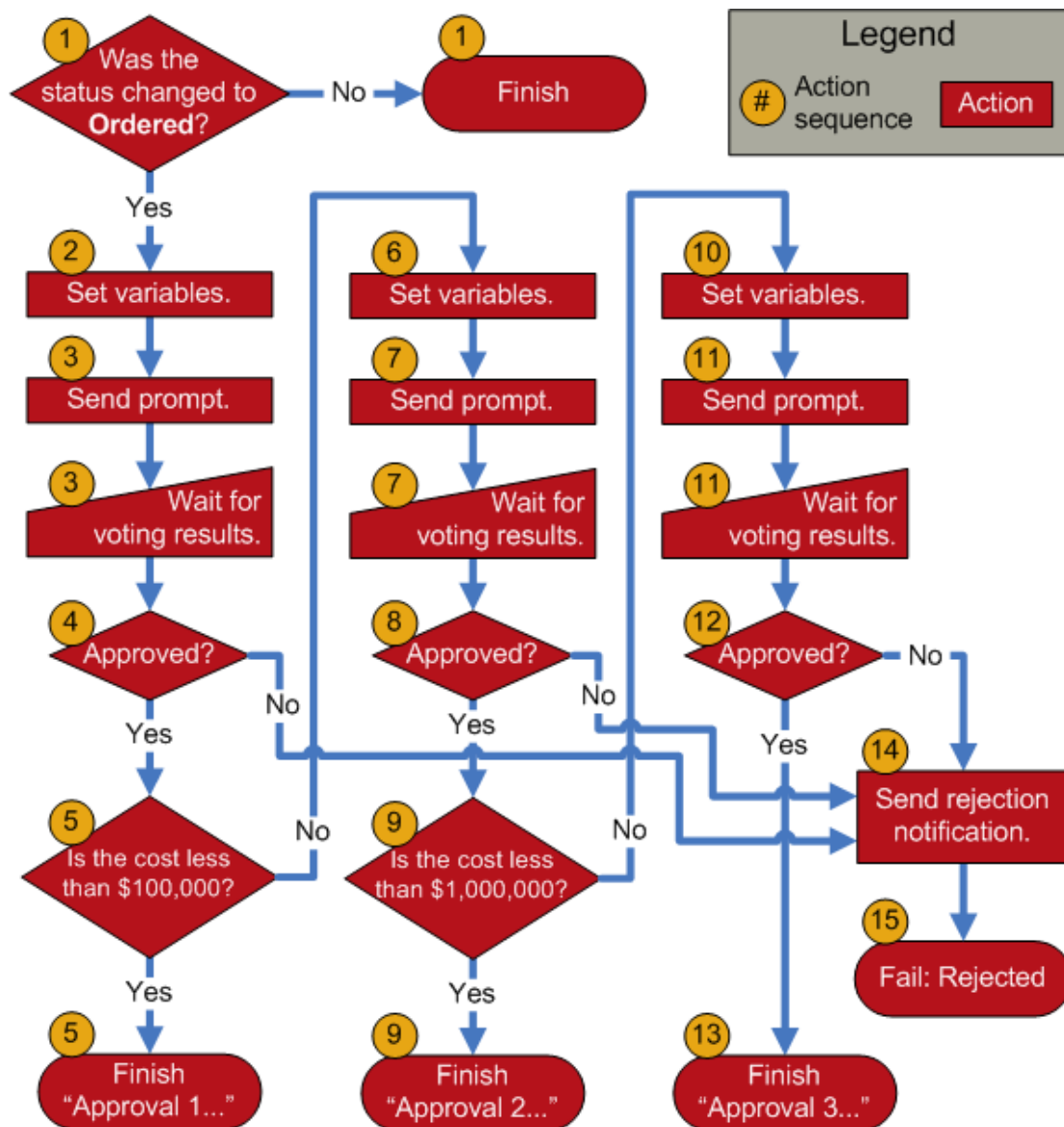
- About multiple and complex approvals
- Approval by multiple levels of management

About multiple and complex approvals

In addition to basic approvals, the Application Event System can also support multiple and complex approvals. These approvals involve multiple people and/or multiple criteria. Here are two examples:

- Additional individuals are prompted to approve only if certain predefined criteria are met
- Multiple people are prompted to give approval, but the ultimate approval decision is based on a tally of their “votes”

The event handler diagram below shows a complex approval process requiring fourteen event actions.



Event handler diagram for three levels of approval

Note that one set of four actions (2-4) is repeated three times: Set variables, Send prompt, Wait for voting results, Evaluate whether the results are approved.

Approval by multiple levels of management

This exercise will demonstrate approval prompts being sent out to multiple people based on predefined criteria, in this case a purchase order status change to “Ordered” and the amount of the purchase order.

We will set up an event handler and event actions so that when a purchase order (PO) status is changed to Ordered, it requires the approval of the purchasing manager. At the same time, if the PO is for more than \$100,000, the PO also requires the approval of the purchasing manager’s supervisor. Finally, if the PO is for more than \$1,000,000, the PO requires the further approval of a senior executive.

If the PO is disapproved at any level, the PO is rolled back to the previous values, and any changes made to it are lost. While the PO is in the process of being approved, it remains suspended until approval or disapproval is determined. This means, among other things, that if one or more approvers fail to respond to the request, the PO is locked and cannot be changed until all required approvers respond.

For this exercise, we will:

- Use the same event global constant for the purchasing manager (PurchasingMgr) that we used for a previous exercise.
- Create new event global constants for the purchasing supervisor and purchasing senior executive, PO values for each management level, and the approval message content.
- Use the IdoOnItemUpdate framework event for the SLPoS IDO. This will cause the event to be generated whenever a PO record is updated either in the **Purchase Orders** form or by another process that attempts to update that record.
- Use an action type called a branch to evaluate each response received (approved or not) and choose the next action based on that response.
- Pass property values and an identifying property (ItemId) to the event as input parameters so the system can store them with the event as event parameters.

The variable named CountMethod is set to count the votes using the MinimumPercentage rule, which says that the first choice to reach a minimum percentage determines the next action. In this case, the minimum percentage is 100%, so all recipients must approve for the PO to reach final approval. If any recipient rejects the request, the entire request is rejected, no matter who has approved it to that point.

The MinimumPercentage voting rule requires that you specify a minimum percentage for passage. This means that, in addition to the VOTINGRULE() keyword, you must also specify the MINIMUM() keyword as part of the variable definition. The resulting declaration for the Value column of the CountMethod variable is: VOTINGRULE(MinimumPercentage) MINIMUM(100).



Exercise 9.1: Approving a PO by up to three management levels

In this exercise, you will create new event global constants and set up fourteen event actions to prompt first the purchasing manager for approval of a new purchase order, and if certain criteria are met, to prompt the purchasing supervisor and the purchasing senior executive.

Exercise steps

Part 1: Create event global constants

1. Open the **Event Global Constants** form.

2. Click **Filter in Place**.
3. Click **Create a new object in the current collection**.
4. Type *PurchasingSuper* in the **Name** field.
5. Type *<your user ID>* in the **Value** text box.
6. Click **Save**.
7. Repeat **steps 3-6** by changing the **Name** and **Value** fields as per the table below:

Name	Value
PurchasingSenior	<your user ID>
SuperCost	100000
SeniorCost	1000000
POApprovalPrompt	SUBJECT(" Purchase Order Update Approval Needed") CATEGORY("Order Approval") TO(GC(TV(Approver))) BODY(SUBSTITUTE("A purchase order, {0}, has been updated to Ordered status for vendor, {1}, number: {2}. Please review the details on the Variables tab and then indicate your approval on the Response tab.", FP("PoNum"), P("VendorName"), FP("VendNum"))) QUESTION("Do you approve this PO change?") CHOICES("1, sYes, 0, sNo") VOTINGRULE(TV(CountMethod)) FILTERFORM("PurchaseOrders") FILTER(SUBSTITUTE("PoNum={0}", FP("PoNum")))

8. Close the **Event Global Constants** form.

Part 2: Create the handler

1. Open the **Event Handlers** form.
2. Click **Filter in Place**.
3. Click **Create a new object in the current collection**.
4. Select **IdoOnItemUpdate** from the **Event Name** drop-down list.
5. Type *Requesting multiple and complex approvals* in the **Handler Description** field.
6. Type *SLPo** in the **Applies to Objects** field.
7. Select **SLPos** from the **Applies to Objects** drop-down list.

8. Click **Save**.

Part 3: Create the event action #10-FINISH

1. Select the Event Handler you created in Part 2.
2. Click **Event Actions**.
3. Type *10* in the **Action Sequence** field.
4. Select **Finish** from the **Action Type** drop-down list.
5. Click **Edit Parameters**.
6. Click **Condition**.
7. Click **Expression 1**.
8. Select **PROPERTYMODIFIED** from the **Select a function** drop-down list.
9. Select **Stat** from the **Argument 1** drop-down list.
10. Click **OK**.
11. Select the **NOT** check box.
12. Click **OK**.
13. Type *OR P("Stat") <> "O"* after NOT PROPERTYMODIFIED("Stat") in the **Condition** field.
Note: You are appending the condition statement. In plain English, it means "If the property status is not modified or the property status is not ordered, then finish." The statement should look like this:

NOT PROPERTYMODIFIED("Stat") OR P("Stat") <> "O"
14. Click **OK**.
15. Click **Show Details**.
16. Click **Save**.

Part 4: Create the event action #20-SET VALUES

1. Click **Create a new object in the current collection**.
2. Type *20* in the **Action Sequence** field.
3. Select **Set Values** from the **Action Type** drop-down list.
4. Click **Edit Parameters**.
5. Click **Variables**.
6. Type *Approver* in the **Variable Name** field in row 1.
7. Click in the **Value** field. The field activates.
8. Type *PurchasingMgr* in the **Value** field.
9. Type *CountMethod* in the **Variable Name** field in row 2.
10. Type *Plurality* in the **Value** field.
11. Click **OK** twice.
12. Click **Save**.

Part 5: Create the event action #30-PROMPT

1. Click **Create a new object in the current collection**.
2. Type *30* in the **Action Sequence** field.
3. Select **Prompt** from the **Action Type** drop-down list.
4. Type *TGC(POApprovalPrompt)* in the **Parameters** tab text box.

Because *TGC()* is a pre-parser function, you cannot use the **Event Action Parameter** forms to create or set it. You must specify this statement directly in the **Parameters** field of the **Event Actions** form. This also means that you cannot use the **Check Syntax** button to check the syntax.



This statement performs a text evaluation of the *POApprovalPrompt* global constant to obtain the parameters for a prompt action. Part of this text evaluation includes an evaluation and insertion of the values for the two variables we set in the previous step.

After evaluating the *POApprovalPrompt* global constant, this action also sends out the prompt message to the purchasing manager and suspends the handler pending the manager's response. Because we did not specify any **Variable Access** rules, the message allows the purchasing manager to modify any variable values before approving it.

5. Click **Save**.

Part 6: Create the event action #40-BRANCH

1. Click **Create a new object in the current collection**.
2. Type *40* in the **Action Sequence** field.
3. Select **Branch** from the **Action Type** drop-down list.
4. Click **Edit Parameters**.
5. Click **Condition**.
6. Click **Expression 1**.
7. Select **VOTINGRESULT** from the **Select a function** drop-down list.
8. Select **30 Prompt** from the **Action** drop-down list.
9. Click **OK**.
10. Select **not equal to (<>)** from the **Operator** drop-down list.
11. Type *1* in the **Expression 2** field.
12. Click **OK**.
13. Type *140* in the **Destination** field.

Note: We have not yet created action 140, but we can reference it before it is created.

14. Click **OK**.
15. Click **Save**.

Part 7: Create the event action #50-FINISH

Note: If this event action fires, you know that the purchasing manager has approved the purchase order. This action determines if the cost is greater than \$100,000 meaning the purchasing supervisor needs to approve, or if it can finish.

1. Click **Create a new object in the current collection**.
2. Type *50* in the **Action Sequence** field.
3. Select **Finish** from the **Action Type** drop-down list.
4. Click **Edit Parameters**.
5. Click **Condition**.
6. Click **Expression 1**.
7. Select **P** from the **Select a function** drop-down list.
8. Select **PoCost** from the **Argument 1** drop-down list.
9. Click **OK**.
10. Select **less than (<)** from the **Operator** drop-down list.
11. Click **Expression 2**.
12. Select **GC** from the **Select a function** drop-down list.
13. Select **SuperCost** from the **Argument 1** drop-down list.
14. Click **OK** twice.
15. Type *Approved by Purchasing Manager* in the **Result** field.
16. Click **OK**.
17. Click **Save**.

Part 8: Create the event action #60-SET VALUES

Note: If this event action fires, you know that the purchasing manager has approved the purchase and that the cost of the purchase order is greater than \$100,000. If one or both conditions were not met, the actions would have finished at event action #5.

1. Click **Create a new object in the current collection**.
2. Type *60* in the **Action Sequence** field.
3. Select **Set Values** from the **Action Type** drop-down list.
4. Click **Edit Parameters**.
5. Click **Variables**.
6. Type *Approver* in the **Variable Name** field in row 1.
7. Click in the **Value** field. The field activates.
8. Type *PurchasingSuper* in the **Value** field.
9. Type *CountMethod* in the **Variable Name** field in row 2.
10. Type *Majority* in the **Value** field.
11. Click **OK** twice.

12. Click **Save**.

Part 9: Create the event action #70-PROMPT

1. Click **Create a new object in the current collection**.
2. Type *70* in the **Action Sequence** field.
3. Select **Prompt** from the **Action Type** drop-down list.
4. Click **Show Details**.
5. Type *TGC(POApprovalPrompt)* in the **Parameters** tab text box.
6. Click **Save**.

Part 10: Create the event action #80-BRANCH

1. Click **Create a new object in the current collection**.
2. Type *80* in the **Action Sequence** field.
3. Select **Branch** from the **Action Type** drop-down list.
4. Click **Edit Parameters**.
5. Click **Condition**.
6. Click **Expression 1**.
7. Select **VOTINGRESULT** from the **Select a function** drop-down list.
8. Select **70 Prompt** from the **Action** drop-down list.
9. Click **OK**.
10. Select **not equal to (<>)** from the **Operator** drop-down list.
11. Type *1* in the **Expression 2** field.
12. Click **OK**.
13. Type *140* in the **Destination** field.
14. Click **OK**.
15. Click **Save**.

Part 11: Create the event action #90-FINISH

Note: If this event action fires, you know that the purchasing supervisor has approved the purchase order which is greater than \$100,000. This action determines if the purchasing senior needs to approve, or if it can finish.

1. Click **Create a new object in the current collection**.
2. Type *90* in the **Action Sequence** field.
3. Select **Finish** from the **Action Type** drop-down list.
4. Click **Edit Parameters**.
5. Click **Condition**.
6. Click **Expression 1**.

7. Select **P** from the **Select a function** drop-down list.
8. Select **PoCost** from the **Argument 1** drop-down list.
9. Click **OK**.
10. Select **less than (<)** from the **Operator** drop-down list.
11. Click **Expression 2**.
12. Select **GC** from the **Select a function** drop-down list.
13. Select **SeniorCost** from the **Argument 1** drop-down list.
14. Click **OK** twice.
15. Type *Approved by both the Purchasing Manager and Purchasing Supervisor* in the **Result** field.
16. Click **OK**.
17. Click **Save**.

Part 12: Create the event action #100-SET VALUES

Note: If this event action fires, you know that the purchasing manager and purchasing supervisor have approved the purchase and that the cost of the purchase order is greater than \$1,000,000. If one or both conditions were not met, the actions would have finished at event action #9.

1. Click **Create a new object in the current collection**.
2. Type *100* in the **Action Sequence** field.
3. Select **Set Values** from the **Action Type** drop-down list.
4. Click **Edit Parameters**.
5. Click **Variables**.
6. Type *Approver* in the **Variable Name** field in row 1.
7. Click in the **Value** field. The field activates.
8. Type *PurchasingSenior* in the **Value** field.
9. Type *CountMethod* in the **Variable Name** field in row 2.
10. Type *EarliestResponse* in the **Value** field.
11. Click **OK** twice.
12. Click **Save**.

Part 13: Create the event action #110-PROMPT

1. Click **Create a new object in the current collection**.
2. Type *110* in the **Action Sequence** field.
3. Select **Prompt** from the **Action Type** drop-down list.
4. Verify the **Parameters** tab is visible. If not, click **Show Details**.
5. Type *TGC(POApprovalPrompt)* in the **Parameters** tab text box.
6. Click **Save**.

Part 14: Create the event action #120 - BRANCH

1. Click **Create a new object in the current collection**.
2. Type *120* in the **Action Sequence** field.
3. Select **Branch** from the **Action Type** drop-down list.
4. Click **Edit Parameters**.
5. Click **Condition**.
6. Click **Expression 1**.
7. Select **VOTINGRESULT** from the **Select a function** drop-down list.
8. Select **110 Prompt** from the **Action** drop-down list.
9. Click **OK**.
10. Select **not equal to (<>)** from the **Operator** drop-down list.
11. Type *1* in the **Expression 2** field.
12. Click **OK**.
13. Type *140* in the **Destination** field.
14. Click **OK**.
15. Click **Save**.

Part 15: Create the event action #130 - FINISH

Note: If this event action fires, you know that the senior purchasing executive has approved the purchase order.

1. Click **Create a new object in the current collection**.
2. Type *130* in the **Action Sequence** field.
3. Select **Finish** from the **Action Type** drop-down list.
4. Click **Edit Parameters**.
5. Type *Purchase order change approved by Senior Executive* in the **Results** field.
6. Click **OK**.
7. Click **Save**.

Part 16: Create the event action #140 - NOTIFY

Note: If this event action fires, you know that the purchasing manager, the purchasing supervisor, or the purchasing senior executive has disapproved the purchase order change.

1. Click **Create a new object in the current collection**.
2. Type *140* in the **Action Sequence** field.
3. Select **Notify** from the **Action Type** drop-down list.
4. Click **Edit Parameters**.
5. Click **To**.
6. Click **Recipients**.

7. Select **ORIGINATOR** from the **Section a function** drop-down list.
8. Click **OK** twice.
9. Click **Subject**.
10. Select **SUBSTITUTE** from the **Select a function** drop-down list.
11. Type *Purchase order {0} change request disapproved* in the **Argument 1** field.
12. Click **Build Expression**.
13. Select **FP** from the **Select a function** drop-down list.
14. Select **PoNum** from the **Argument 1** drop-down list.
15. Click **OK** twice.
16. Type *Notification* in the **Category** field.
17. Click **Body**.
18. Select **SUBSTITUTE** from the **Select a function** drop-down list.
19. Type *Your purchase order change request for PO {0} for {1}, vendor number: {2} has been disapproved. If you have questions, please see the required approvers.* in the **Argument 1** field.
20. Click in row 1 of the **Argument** grid.
21. Click **Build Expression**.
22. Select **P** from the **Select a function** drop-down list.
23. Select **PoNum** from the **Argument 1** drop-down list.
24. Click **OK**.
25. Repeat **steps 22-26** to add the values in the table below:

Function	Argument
P	VendorName
P	VendNum

26. Click **OK**.
27. Click **Check Syntax**. A dialog box opens with the message, "No syntax errors were found."
28. Click **OK**.
29. Verify your parameters look as shown below:

<< Hide Details Check Syntax Edit Parameters
 Parameters Substituted Parameters Variable Access
 CATEGORY("Notification")
 TO(ORIGINATOR())
 SUBJECT(SUBSTITUTE("Purchase order {0} change request disapproved", P("PoNum"))))
 BODY(SUBSTITUTE("Your purchase order change request for PO {0} for {1}, vendor number:", P("PoNum"), P("VendorName"), P("VendNum"))))
 SAVEMESSAGE(FALSE)

30. Click **Save**.

Part 17: Create the event action #150 - FAIL

Note: If this event action fires, you know that one of the managers did not approve the purchase order.

1. Click **Create a new object in the current collection**.
2. Type *150* in the **Action Sequence** field.
3. Select **Fail** from the **Action Type** drop-down list.
4. Click **Edit Parameters**.
5. Type *PO change not approved* in the **Result** field.
6. Click **OK**.
7. Click **Save**.

Part 18: Mark the handler to suspend

1. Click the **Event Handlers** tab. The form displays.
2. Select the **Suspend** check box.

Note: You cannot select this check box until you have an adjourning action. The prompt actions created are the adjourning actions.

3. Click **Save**.

Part 19: Test the handler

1. Click **Ctrl+U** to discard the metadata cache.
2. Open the **Purchase Orders** form.
3. Click the **Date** calendar. The **Calendar** displays.
4. Select **<today's date>**.
5. Click **Filter in Place**. The purchase order you created in exercise 7.1 displays.
6. If the **Lines** button is not visible to the right of the **Purchase Orders** form, click the ellipsis.
7. Click **Lines**. The **Purchase Order Lines (Linked)** form displays.
8. Select **FA-10000 Bicycle, Model 30,26"** from the **Item** drop-down list.
9. Type *10000* in the **Ordered** field.
10. Click **Save**.
11. Close the **Purchase Order Lines (Linked)** form.
12. Click **Refresh**. The total cost of the purchase order, **1,731,751.10**, is displayed.
13. Open the **Inbox** form.
14. Verify that you did not receive a message with the subject "Purchase Order Update Approval Needed," since this event handler should not send a prompt message until the status = ordered.
Note: You may see a message with the subject, "A new row was added to Purchase Order Items" which is from a different event handler.
15. Open the **Purchase Orders** form, and filter for the Purchase Order you updated.

16. Select **Ordered** from the **Status** drop-down list. A dialog box opens with the message, "[Change PO Status] will be performed."
17. Click **OK**.
18. Click **Save**.

Notice that the status remains in a planned state because the purchase order has not yet been approved.
19. Click the **Inbox** tab. The **Inbox** form displays.
20. Click **Refresh**.
21. Open the message with the subject, **Purchase Order Update Needed**.

Check your understanding



Which of the following action types would you use for determining whether a response received resulted in approval or disapproval?

- a) Prompt
- b) Branch
- c) Set Values
- d) Finish



Lesson 10: Generating events from within a form

Estimated time

.5 hours

Learning objectives

After completing this lesson, you will be able to:

- Explain how to generate an event form within a form.

Topics

- Framework events vs. developer-defined events
- Using the Application Event System within a form

Framework events vs. developer-defined events

One of the big similarities between the Form Event System and the Application Event System is, obviously, that they both deal with events. In both cases, the event itself is simply a uniquely named occurrence. Yes, that sounds a bit vague, but the central idea is that when that something happens, we can set up the system to do something else in response to that event.

The chief difference between a form-level event and an application event is the way they are defined. Form events are typically defined by naming them and then assigning at least one event handler to deal with them, nearly always in response to some user action.

The Application Event System takes a somewhat broader view of events. Events in the Application Event System can be one of two general types:

- Framework events
- Developer-defined events
-

We have been using framework events in all the exercises so far, so now let's step back and review them once again.

Framework events

Framework events are events that Infor has predefined and built in to the system. These events—which include system actions like adding a record, modifying a record, deleting a record, and so on—cannot be generated programmatically. That is, you cannot define framework events; they simply get generated every time the system does something.

Now, what this means is that events are constantly being generated in CloudSuite applications. Most of the time, you are not aware of them and the system does nothing in response to them. That is because, if no event handler has been defined to respond to an event, no response is generated. The event occurs, and that is that.

We can, however, define event handlers to respond to these framework events, and that is a large part of what the Application Event System is about.

Developer-defined events

Developer-defined events, such as the custom events you create, are events that you can define yourself, by defining a new row on the **Events** form to respond to specific user actions or other system triggers.

Developer-defined events can respond to:

- Event triggers
- User actions

Developer-defined events that are based on user actions are similar to form events. These events require the user to take some action to which the event system then responds.



The big difference here is a form event is limited to actions on a single form while the application event can involve multiple forms.

Developer-defined events that are based on event triggers are a little different. Typically, event triggers are things that happen within the system that might or might not involve user action of any kind. Here are a few examples:

- You can create an event handler that is designed to respond to an event at a certain time, or after a certain amount of time has passed.
- You can design an event handler to trigger when a certain currency amount has been exceeded.
- You can have an event that is generated based on some action involving a Web service or another application.

The possibilities for triggering an event are nearly endless.

Using the Application Event System within a form

The next exercise is different from others because we will use a developer-designed event, not a framework event. All the exercises we have done previously were built on an existing framework event, for example `IdoOnItemInsert`. So far, we have not had to create an event, only event handlers and event actions.

Not every scenario for which you want an event to occur will fall within the realm of existing framework events. To fulfill an organizational need, you may need to get creative to design a new event to be able to achieve your objectives.

Here is the scenario for the next exercise: CloudSuite users have requested that the system administrator provide them with a means to submit information to the IT department from within the system. In this case, there is no existing framework event (such as an IT Issue IDO) and so we must build out the functionality based on actions that occur in the system by the user. You will set up the event and event handlers as you have in previous exercises, and then you will call to this event from within the new form you create using the action type called "Generate application event."



Exercise 10.1: Calling an application event from a form

In this exercise, you will create a simple form to send an IT problem to the help desk. You will create an event that occurs when the Submit button on the form is pressed. You will also set up an event handler and actions to automatically send the text about each IT problem to a recipient, the IT department. Finally, you will test the event by submitting an IT ticket.

Exercise steps

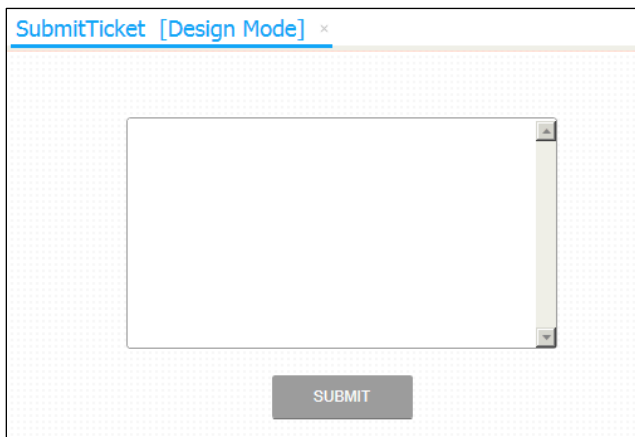
Part 1: Change setting for user permission

1. Open the **Users** form.
2. Click **Filter in Place**.
3. Select the **<your user ID>** record. The record displays.
4. Select **Site Developer** from the **Editing Permissions** drop-down list.
5. Click **Save**.
6. Sign out of **CloudSuite** and then log back in.

Part 2: Create a custom form

1. Click the **Design Mode** icon in the toolbar.
2. Select **Site Default** in the **Scope** drop-down list in the **Identify Your Editing Scope** form. The **User/Group** drop-down list should then be disabled.
3. Click **OK**.
4. Select **System > Form > Definition > New**. The **New Form Wizard** displays.
5. Click **Next**.
6. Type *SubmitTicket* in the **Name** field.
7. Select **Build from Scratch** from the **Form Type** drop-down list.

8. Click **Next**.
9. Clear the **Save Template** check box.
10. Click **Finish**. The **SubmitTicket** form displays.
11. Select **MultiLineEdit** from the **Toolbox** panel. Your cursor appears as a crossbar.
12. Click and drag anywhere on the form to draw a large rectangle text box.
Note: You can resize the text box by clicking and dragging on a corner.
13. Select **Button** in the **Toolbox** panel. Your cursor appears as a crossbar.
14. Click and drag to draw a small rectangle button under the large rectangle text box.
15. Type *Submit* in the **Caption** field in the **Properties** panel.
Note: You must have the button graphic selected in the form to see the Properties panel for the button.
16. Press **Enter**. Your form should look similar to what is shown below:



17. Click **Save**.
18. Click **Enter** or **exit design mode on this form**.
19. Close the **SubmitTicket** form.

Part 3: Create a global event constant for IT

1. Open the **Event Global Constants** form.
2. Click **Filter in Place**.
3. Click **Create a new object in the current collection**.
4. Type *IT* in the **Name** field.
5. Type *<your user ID>* in the **Value** field.
6. Click **Save**.
7. Close the **Event Global Constants** form.

Part 4: Create an event called NotifyIT

1. Open the **Events** form.

2. Click **Filter in Place**.
3. Click **Add a new object in the current collection**.
4. Type *NotifyIT* in the **Event Name** field.
5. Type *Send information to IT department* in the **Description** field.
6. Click **Save**.

Part 5: Create an event handler and event actions

1. Click **Event Handlers**.
2. Type *Notify IT* in the **Handler Description** field.
3. Click **Save**.
4. Click **Event Actions**.
5. Select **Notify** from the **Action Type** drop-down list.
6. Click **Save**.
7. Click **Edit Parameters**.
8. Click **To**.
9. Click **Recipients**.
10. Click **GC** from the **Select a function** drop-down list.
11. Select **IT** from the **Argument 1** drop-down list.
12. Click **OK** twice.
13. Type *New IT Issue Reported* in the **Subject** field.
14. Click **Body**.
15. Select **SUBSTITUTE** from the **Select a function** drop-down list.
16. Type *User {0} has reported the following issue: {1}* in the text field.
17. Click in the **Argument** field for row 1. The field activates.
18. Click **Build Expression**.
19. Select **ORIGINATOR** from the **Select a function** drop-down list.
20. Click **OK**.
21. Click in the **Argument** field for row 2.
22. Click **Build Expression**.
23. Select **E** from the **Select a function** drop-down list.
24. Type *Issue* in the **Expression 1** field.
25. Click **OK** three times.
26. Click **Show Details**.
27. Click **Save**.
28. Close the **Event Actions (Linked)** form.

29. Close the **Event Handlers (Linked)** form.
30. Close the **Events** form.
31. Click **Ctrl+U** to clear the metadata cache.

Part 6: Set up the “call to the event” from the form

1. Open the **SubmitTicket** form.
2. Click **Enter or exit design mode on this form**.
3. Click the **SUBMIT** button. Information associated with the button displays.
4. Click the **Events** tab on the **Component** panel.
5. Click the **ellipsis (...)** next to the **Primary** field. The **Event Handlers** panel displays.
6. Click **New**. The **Event Handlers Properties** panel displays.
7. Type *SubmitIT* in the **Event** field.
8. Select **Generate Application Event** from the **Type** drop-down list.
9. Click the **ellipsis (...)** next to the **Parameters** field. The **Event Handler Parm** panel displays.
10. Click **Type Specific Parameters**. The **Edit Parameters for Generate Application Event Handler** panel displays.
11. Select **NotifyIT** from the **Event** drop-down list.
12. Type *Issue* in the **Name** field in the **Parameters** grid.
13. Type *C(multiLineEdit1_SITE)* in the **Value** field in the **Parameters** grid.
14. Click **OK**. The **Edit Parameters for Generate Application Event Handler** panel closes.
15. Click **OK**. The **Event Handler Parm** panel closes.
16. Click **OK**. The **Event Handlers Properties** panel closes.
17. Click **OK**. The **Event Handlers** panel closes.
18. Click **Save**.
19. Close all open tabs.
20. Click **Ctrl+U** to clear the metadata cache.

Part 7: Test the event

1. Open the **SubmitTicket** form.
2. Type *This is a test* in the **text** box.
3. Click **Submit**.
Note: Click the button only once. You will not receive feedback that it has been submitted.
4. Open the **Inbox** form.
5. Verify the message **“New IT Issue Reported”** has been received.

Check your understanding



You can modify framework events but you cannot create a new event from scratch.

- a) True
- b) False



What action type do you select to call an event from a form?

- a) Set event properties
- b) Call component
- c) Call event
- d) Generate application event



Lesson 11: Setting up external email

Estimated time

.5 hours

Learning objectives

After completing this lesson, you will be able to:

- List the steps for setting up external email.

Topics

- Sending messages to external email
- Format of external email
- Setting up external email

Sending messages to external email

When you define event actions that send messages to recipients, the Application Event System will send prompt and notification messages to the CloudSuite Inbox forms of those recipients. However, it is possible to send messages to recipients' external email addresses as well.

There are three action types that can generate messages sent to external email addresses:

- Notify
- Prompt
- Send Email

If a user is set up to allow the system to "Send External Notifications," then a notify action sends the message both to the CloudSuite Inbox form and to the recipient's inbox in an external email client such as Microsoft® Outlook®.

Similarly, if a user is set up to allow the system to "Send external prompts," then a prompt action sends the message both to the CloudSuite Inbox form and to the recipient's inbox in an external email client. There are some limitations on the format of prompt messages sent to external email addresses.

If a user is configured to receive external email, then a "Send email" action sends the message to the recipient's external email client, but does not send it to the CloudSuite Inbox form.

The chart below summarizes where recipients will receive messages sent by the Application Event System:

Action type	Message sent to Inbox form?	Email sent? (for users set up to receive email)
Notify	Yes	Yes
Prompt	Yes	Yes, but formatting restrictions will limit certain functionality
Send Email	No	Yes

Format of external email

A message that is sent to a recipient's external email system is an HTML-formatted email that consists of these parts:

- Original subject in Subject line
- Category
- List of internal recipients
- List of internal Cc recipients
- Original Body (either in plain text or HTML format)
- Original Question
- Choices, as individual hyperlinks to a .NET active server page (ASP) that records the vote
- Payload, which is the contents of the Variables grid from the Inbox

Text in the Subject, Category, Body, Question, and Choices, as well as payload captions, can be translated and formatted based on the default language specified for the recipient user in the **Users** form.

When the recipients vote by clicking the link in the email, their Windows-default Web browser opens, or opens a new tab if applicable, and sends the information from the link they clicked to the ASP URL that was included (and hidden) in the email. This URL is built by the system based on the web server list.

The ASP then registers the vote programmatically, as if the recipient had logged in to the system, displayed the same message in the Inbox, and selected the corresponding **Choice** button on the **Response** tab.

The Web page then displays a success or failure message.

The message displays both in email and in the **Inbox** form, but only one response is allowed:

- If users respond from email first, when they display the Inbox (properly refreshed), the message is expired and the buttons are inactive.
- If users respond from the Inbox first, then clicking a link in the external email brings up the Web page with a message that the message has expired and they already voted, using the previously selected choice.



All prompts and notifications will go to the Inbox as well as any external email you have set up unless you select an Action Type Send Email.

Troubleshooting email problems

If you have issues with email, go to the **Event Handler Status** form and view the record for the event in question. If the email was not sent, this will be detailed in the **Result** section of the **Handler** tab.



You will not be able to send external email from within the classroom training environment due to firewalls.

Setting up external email

The task of setting up external email requires knowledge of your organization's internal email configuration. Before beginning this process or making any changes to the Intranets form in CloudSuite, check with the system administrator who manages your email to make sure all **SMTP** fields are correct based on the global settings for your organization's email server.

The following shows a high-level overview of the steps for setting up external email:

Step	How to set up external email Message sent to Inbox form?
1	Get SMTP settings for your organization's email server
2	Configure CloudSuite for external email using the Intranets form
3	Add an email address for each user who is to receive email
4	Designate if a user is to receive prompts, notifications, or both



Demo 11.1: Setting up external email accounts

This demonstration shows how to set up external email accounts.

Before you begin:

- To prepare the system for future exercises, this demo must be completed.

Demo steps

Part 1: Set up the SMTP for the system

1. Open the **Intranets** form.
2. Click **Filter in Place**. The record for DEMO opens.
3. Click the **Reporting** tab.
4. Set all fields in the **SMTP** section as directed by your organization's email system administrator.

Part 2: Set up individual users for external email

1. Open the **Users** form.
2. Click **Filter in Place**. A list of users displays.
3. Select **<your user ID>**.
4. Click the **Email Address** tab.
5. Select **Primary** from the **Email Type Description** drop-down list.
6. Type *<your external email address>* in the **Email Type Address** field.
7. Select **Primary** from the **Primary Email** drop-down list.

Note: The email address you entered will populate the form.

8. Clear the **Use Reply To** check box.
9. Select the **Send External Notifications** check box.
10. Select the **Send External Prompts** check box.
11. Click **Save**.

Check your understanding



Put the steps in the proper order from 1-4 to set up external email for the Application Event System.

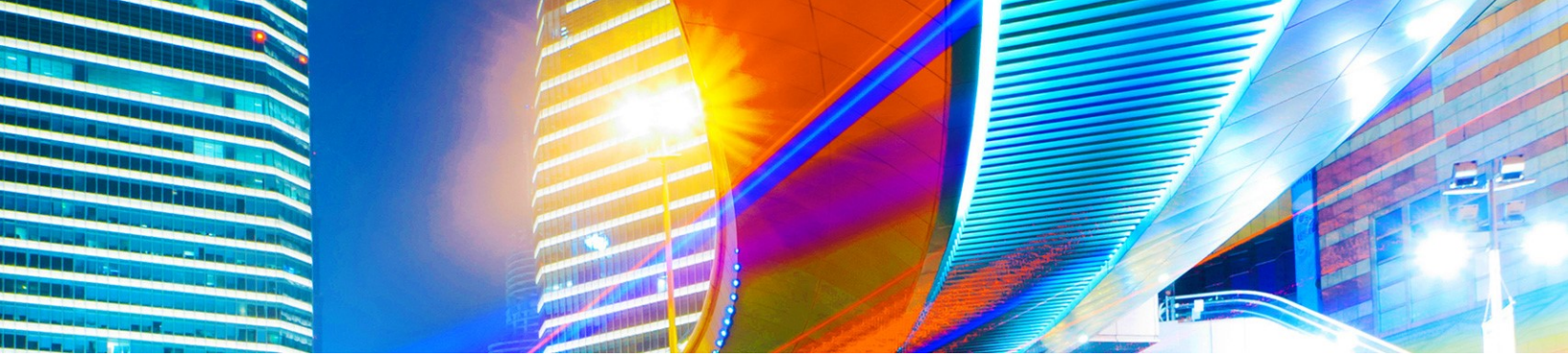


Step description	Step number
Configure CloudSuite for email using the Intranets form	
Add an email address for each user who is to receive email	
Designate if a user is to receive prompts, notifications, or both	
Get SMTP settings for your organization's email server	



If you set up a recipient to receive prompts via email, which of the following is true when a prompt message is generated?

- a) The message will be sent ONLY to the recipient's Inbox form.
- b) The message will be sent ONLY to the recipient's email address.
- c) The message will be sent to BOTH the recipient's Inbox form and email address.
- d) The message waits for the Send Email event action to fire, and then is sent to the recipient's email address.



Lesson 12: Voting

Estimated time

.5 hours

Learning objectives

After completing this lesson, you will be able to:

- Match voting rules to their definitions.

Topics

- Voting rules
- Dealing with indeterminate voting results
- Quorums

Voting rules

When a prompt is sent to a single recipient, the result of the prompt is the return value from that recipient's choice. However, when a prompt is sent to multiple recipients, you must select a vote counting method to determine the result of the prompt and include a Voting Rule parameter in your event action definition.

Rule	Description
Majority	A choice must receive more than 50% of the vote to win. As soon as more than 50% of the recipients respond with a particular choice, that choice wins. Note: If you use the Majority rule, you should use a Voting Tie parameter (VOTINGTIE) to tell the system how to handle a tied vote.
Plurality	The choice with the highest number of votes wins, even if it does not receive more than 50% of the vote. For example, if three choices are offered, and: • The first choice receives 24% of the vote; • The second choice receives 43% of the vote; and • The third choice receives the remaining 33% of the vote; then the second-choice wins, even though it received less than 50% of the total vote. Note: If you use the Plurality rule, you should use a Voting Tie parameter (VOTINGTIE) to tell the system how to handle a tied winning vote.
ConditionalPlurality	The choice with the highest number of votes wins, but only if a specified minimum percentage of votes is reached. For example, if three choices are offered to 19 recipients, and you specify a minimum of 40% to win, then: • In an 8-7-4 split, the choice with eight votes would win because it meets the minimum percentage. • In a 7-6-6 split, there would be no winner, because no choice meets the minimum percentage. In the second case, the system must deal with the vote as an indeterminate result. (Had this been a simple Plurality vote, the choice that reaches seven votes in a 7-6-6 split would win.) Note: If you use the ConditionalPlurality rule, you must also include: • A Minimum Percentage (MINIMUM) parameter. • A Voting Tie parameter (VOTINGTIE) or Voting Disparity parameter (VOTINGDISPARITY) to tell the system how to handle the indeterminate vote.

Rule	Description
MinimumCount	The first choice to reach a specified minimum number of votes wins. For example, if three choices are offered to 13 recipients, and you specify a minimum of five votes to win, the first choice to receive five votes automatically wins. As soon as the minimum count is reached, event execution moves immediately to the next action. In this case, the system expires any responses not yet received, and no further voting can take place. Note: If you use the MinimumCount rule, you must also include a Minimum Count (MINIMUM) parameter.
MinimumPercentage	The first choice to receive a specified percentage of the vote wins. The percentage is based on the number of recipients of the prompt, not the number of respondents. As soon as the minimum percentage is reached for a choice, event execution moves immediately to the next action. In this case, the system expires any responses not yet received, and no further voting can take place. Note: If you use this rule, you must also include a Minimum Percentage (MINIMUM) parameter.
EarliestResponse	The first response to the prompt wins, regardless of the choice. As soon as the first response is received, event execution moves immediately to the next action. In this case, the system expires any responses not yet received, and no further voting can take place.
PreferredChoice	If any one respondent votes for the preferred choice, that choice wins. In a case where none of the respondents select the preferred choice, then this rule behaves like the Plurality rule for the remaining choices. For example, if you have three choices, and you specify the first choice as the preferred choice, then: • If anyone votes for the first choice, that choice wins. • If the end vote is a 0-6-5 split, the second-choice wins. As soon as the preferred choice receives a vote, event execution moves immediately to the next action. In this case, the system expires any responses not yet received, and no further voting can take place. Note: If you use this rule, you must also include a Preferred Choice (PREFCHOICE) parameter to specify which choice is the preferred choice.

Rule	Description
MinimumCount PreferredChoice	If a specified number of votes for a specified choice is cast, that choice wins. For example, if you set the Minimum to 3 for a Preferred Choice of "Approve," and three recipients respond with "Approve," the preferred choice wins. If less than that number of votes are cast for that choice after all recipients have responded, the vote reverts to Plurality. (In that case, the preferred choice may still win.) Note that when you set the Minimum to 1, this rule behaves exactly like Preferred Choice. Note: If you use this rule, you must also include a Minimum (MINIMUM) parameter to specify the minimum count, and a Preferred Choice (PREFCHOICE) parameter to specify which choice is the preferred choice.
MinimumPercentagePreferredChoice	If a specified percentage of votes for a specified choice is cast, that choice wins. For example, if you set the Minimum to 25% for a Preferred Choice of "Approve," and two of eight of recipients respond with "Approve," the preferred choice wins. If less than that percentage of votes are cast for that choice after all recipients have responded, the vote reverts to Plurality. (In that case, the preferred choice may still win.) Note: If you use this rule, you must also include a Minimum (MINIMUM) parameter to specify the minimum percentage, and a Preferred Choice (PREFCHOICE) parameter to specify which choice is the preferred choice.

Dealing with indeterminate voting results

The following situations create "indeterminate" voting results and set action attributes that are exposed as event functions that can be evaluated by subsequent event actions:

- Any disagreement among multiple recipients, registered as soon as a disagreement is detected
 - The associated event function is the `VOTINGDISPARITY()`, which is a Boolean function that indicates only that there was a disagreement.
- A tie in the case of a Plurality or Majority vote, registered at the point when all responses have been received or when the timeout period has expired
 - The associated event function is the `VOTINGTIE()`, which is a Boolean function that indicates only that there was a tie.

You can use the returns from these functions, along with the functions `RECIPIENTS()`, `RESPONDERS()`, `RECIPIENTLIST()`, `RESPONDERLIST()`, and `NONRESPONDERLIST()` to take further actions, such as:

- Reprompting all the recipients and try to get a consensus.
- Reprompting only a select group of the respondents and urging them to adopt a different choice.
- Reprompting only recipients who have not yet responded.
- Taking some other predetermined action.



Quorums

On a Prompt action type, a quorum is automatically calculated based on:

- The number of recipients,
- The voting rule, and
- Any voting parameters set, such as Minimum.

If there are several votes by whose tally a voting result can be determined unambiguously, that number is the quorum. Otherwise, the quorum is the number of recipients, that is, everyone has a chance to vote unless a timeout expires. As soon as the quorum is reached, voting is closed, any remaining unvoted messages are expired, and the event continues to the subsequent event action.

However, if you specify a Quorum value, that overrides the automatic calculation. For example, if a message requiring a response is sent to 10 people, but you want a quorum to be reached when only four have voted, then specify 4 as the Quorum value.

By default, if Quorum is not specified or is specified with a positive value, Wait for Quorum is True; that is, the event waits until the quorum is reached before it continues with the next event action. If Quorum is specified with a non-positive value, the Wait for Quorum default value is False. If these settings conflict, for example Quorum = 3 and Wait for Quorum is False, the system displays an error message.

If Wait for Quorum is False, the event does not wait for a quorum to be reached. As soon as the messages are sent, execution continues with the next event action. If the system is not waiting for a quorum, the event designer needs to determine when a quorum is reached and what further actions to take. This can be done using VOTINGRESULT(), RESPONDERS(), RECIPIENTS(), etc., in combination with the Wait or Sleep actions.

Check your understanding



Match each voting rule to its definition. The possible voting rules are: **1-Majority**, **2-Plurality**, **3-ConditionalPlurality**, **4-EarliestResponse**, and **5-MinimumCount**.



Definition	Voting rules
The choice with the highest number of votes wins.	
The first choice to reach a specified minimum number of votes wins.	
A choice must receive more than 50% of the vote to win.	
The first response to the prompt wins, regardless of the choice.	
The choice with the highest number of votes wins if a specified percentage of votes is reached.	



Course summary

Estimated time

.5 hours

Course objectives

Now that you have completed this course, you should be able to:

- Describe the Application Event System.
- Describe the functionality of events and event handlers.
- Define framework events.
- Identify the forms used for the Application Event System.
- Define the purpose of an event global constant.
- Describe key concepts for creating custom events and handlers.
- Define the New Workflow Wizard.
- Discuss how event messages work with the Inbox form.
- Identify the forms used to track event system status.
- Explain how to set up an event that uses a prompt to request approval.
- Explain how to set up an event to add information to a record.
- Explain how to set up an event to request multiple levels of approval.
- Explain how to generate an event from within a form.
- List the steps for setting up external email.
- Match voting rules to their definitions.

Topics

- Course review

Check your understanding



Describe a scenario where it makes sense to suspend an event handler.





What actions could you take to explore why the Application Event System is not running as you expect? Select all that apply.

- a) Check whether the Infor Framework Event Service has started and is running.
- b) Open the Service Configuration Manager to verify that your configuration has been added.
- c) Change the processing order for queued events to “first in, first out” (FIFO) order.
- d) Disable the Framework Event System.



On what form will you find the Edit Parameters button?

- a) Events form
- b) Event Triggers form
- c) Event Handlers form
- d) Event Actions form



An event handler must always contain at least one event action.

- a) True
- b) False



What is the purpose of an event global constant?





Match each function type to the description and example provided. The possible types are: **Parameter function**, **Word function**, and **Value function**.



Description and example	Function type
These must always appear at the root level and cannot be nested inside another function. Example: SETVARVALUES	
These must never appear at the root level, but must be nested inside another function. Example: SUBSTITUTE	
Verbatim words used inside an event function call. Example: STRING	



Which of the following statements about the Event Actions Parameters form is true? Select all that apply.

- a) It is not a single form, but a set of forms
- b) It helps you build parameters without syntax errors
- c) It lists all event variables including event global constants
- d) To open, click Edit Parameters on any Event Actions form



Which of the following statements is true about the Inbox form?

- a) It is a full working email system
- b) It can be used to communicate with others in your CloudSuite environment
- c) It allows you to create custom folders
- d) It displays system messages but does not allow you to respond



Appendices

The following are included in this section:

- Appendix A: User accounts
- Appendix B: Reference tables
- Appendix C: Expression grammar
- Appendix D: Sample stored procedures
- Appendix E: Synchronization of metadata
- Appendix F: Event flow options
- Appendix G: Sample scenarios

Appendix A: User accounts

Your instructor will assign you a student user ID from the table listed below to use for class exercises.

Note: If you are taking this course as self-directed learning, refer to the Training Desktop Login Instructions on the Lab On Demand page.

Course Name/Code = CloudSuite Industrial: v10 Using the Application Event System (v10.18) (01_0011018_IEN3538_SYT)				
Training Environment entry point (VM)	ID	User	Password	
CSI 10 Desktop	All	EDUDesktop\Infor	Infor123	
GDECSIBE (Landing Server)	Administrator	gdeinfor2\administrator	G!oba!08	
Application	ID	User	Password	Employee Name
Instructor login (for course demos): ADFS	osservice	osservice@gdeinfor2.com	G!oba!08	N/A
Instructor login (for course demos): - CSI - Factory Track	sa	sa	G!oba!08	N/A
Student logins (for course exercises): - CSI - Factory Track - AES - SSRS - WorkBench	train01	train01@gdeinfor2.com	train01	N/A
	train02	train02@gdeinfor2.com	train02	N/A
	train03	train03@gdeinfor2.com	train03	N/A
	train04	train04@gdeinfor2.com	train04	N/A
	train05	train05@gdeinfor2.com	train05	N/A
	train06	train06@gdeinfor2.com	train06	N/A
	train07	train07@gdeinfor2.com	train07	N/A
	train08	train08@gdeinfor2.com	train08	N/A
	train09	train09@gdeinfor2.com	train09	N/A
	train10	train10@gdeinfor2.com	train10	N/A

	train11	train11@gdeinfor2.com	train11	N/A
	train12	train12@gdeinfor2.com	train12	N/A
	train13	train13@gdeinfor2.com	train13	N/A
	train14	train14@gdeinfor2.com	train14	N/A
	train15	train15@gdeinfor2.com	train15	N/A
	train16	train16@gdeinfor2.com	train16	N/A
	train17	train17@gdeinfor2.com	train17	N/A
	train18	train18@gdeinfor2.com	train18	N/A
	train19	train19@gdeinfor2.com	train19	N/A
	train20	train20@gdeinfor2.com	train20	N/A
	train21	train21@gdeinfor2.com	train21	N/A
	train22	train22@gdeinfor2.com	train22	N/A
	train23	train23@gdeinfor2.com	train23	N/A
	train24	train24@gdeinfor2.com	train24	N/A
	train25	train25@gdeinfor2.com	train25	N/A

Appendix B: Reference tables

This appendix provides several reference tables containing detailed information about various aspects of the Application Event System. Topics include:

- Firing events
- Summary of synchronous functionality
- Framework events
- Framework event attributes
- Application events
- Event action types
- Event action parameters
- Expression functions
- Expression operators

Firing events

This table provides details about:

- Where events can be generated from (Tier)
- What can be used to generate them from that location (Triggered by)
- How to set them up (Details for construction)
- Whether the event is generated as a synchronous or asynchronous event (Synchronous?)

Tier	Triggered by	Details for construction	Synchronous?
Client	Event handler in form	Use a response type of Generate Application Event, select the Synchronous option.	Yes
		Use a response type of Generate Application Event, clear the Synchronous option.	No
	Script in form	Generate a custom form event. Create a form event handler for that custom event with a response type of Generate Application Event, without the SYNCHRONOUS() parameter.	Yes
		Generate a custom form event. Create a form event handler for that custom event with a response type of Generate Application Event, with the SYNCHRONOUS() parameter.	No
Middle	Custom IDO method	Invoke the FireApplicationEvent() static .NET method with the Synchronous parameter passed in with a value of True.	Yes
		Invoke the FireApplicationEvent() static .NET method with the Synchronous parameter passed in with a value of False.	No
Database	T-SQL	Use the command: EXEC FireEventSp.	Yes
		Use the command: EXEC PostEventSp.	No
Any	An event action	Use the GenerateEvent action type with a parameter of SYNCHRONOUS(true).	Yes
		Use the GenerateEvent action type with a parameter of SYNCHRONOUS(false).	No
	An event trigger	Select the Synchronous option.	Yes
		Clear the Synchronous option.	No

Summary of synchronous functionality

Source	Syncro-nous?	Consists of	Requester	Initial executor
Framework	Yes	Synchronous (none marked Suspend) and asynchronous event handlers	WinStudio	IDO Runtime
			IDO Runtime	IDO Runtime
			Database	Database
		Synchronous (some marked Suspend) and asynchronous event	WinStudio	IDO runtime (validating mode)
			IDO Runtime	Event Service (Committing mode)
FireEvent OR Generate Event with Synchronous (True)	Yes	Synchronous and asynchronous event handlers	WinStudio	IDO Runtime
			IDO Runtime	IDO Runtime
			Database	Database
			Event Service	Event Service
PostEvent OR Generate Event with Synchronous (False)	No	Synchronous and asynchronous event handlers	WinStudio	Event Service
			IDO Runtime	
			Database	
			Event Service	

Context	Synchronous?	Suspending?	Event's or prior event handler's executor	Initial executor	Can adjourn?
Suspending event (always generated synchronously)	Yes	No	IDO Runtime (validating mode)	IDO Runtime	No
			Event Service (committing mode)	Event Service	

Context	Synchronous?	Suspending?	Event's or prior event handler's executor	Initial executor	Can adjourn?
		Yes	Event service	Event Service	Yes
Non-suspending event generated synchronously	Yes	N/A	WinStudio	IDO Runtime	Yes
			IDO Runtime	IDO Runtime	
			Database	Database	
			Event Service	Event Service	
Non-suspending event generated asynchronously	Yes	N/A	WinStudio	Event Service	Yes
			IDO Runtime		
			Database		
			Event Service		
Any event	No	N/A	WinStudio	Event service	Yes
			IDO Runtime		
			Database		
			Event service		

Framework events

This table lists and describes the framework (Core) events that ship with the system. You can create handlers that execute when these events are generated, but you cannot create your own triggers for these events.

Event name	Trigger	Context attributes passed as event parameters, in addition to the object name
IdoOnLoadCollection	When an IDO collection is loaded into the system	Load flags Property names Post query actions
IdoPostLoadCollection	After an IDO collection is loaded into the system	Result set

Event name	Trigger	Context attributes passed as event parameters, in addition to the object name
IdoOnUpdateCollection	Before an IDO collection update is performed	Custom insert specification Custom update specification Custom delete specification
IdoOnItemInsert	When an IDO item is inserted into a collection	Row (IDO item) being inserted
IdoPostItemInsert	After an IDO item is inserted into a collection	Row (IDO item) that was inserted
IdoOnItemUpdate	When an IDO item is updated	Row (IDO item) being updated Modified flags
IdoPostItemUpdate	After an IDO item is updated	Row (IDO item) that was updated Modified flags
IdoOnItemDelete	When an IDO item is deleted	Row (IDO item) being deleted
IdoPostItemDelete	After an IDO item is deleted	Row (IDO item) that was deleted
IdoPostUpdateCollection	After all IDO collection updates have been processed	N/A
IdoOnInvoke	When an IDO method is invoked	Name of the IDO method that was invoked Number of parameters passed to the method Values of the parameters passed to the method
IdoPostInvoke	After an IDO method is invoked	Name of the IDO method that was invoked Number of the parameters passed to the method Values of the parameters passed to the method
SessionOnLogin	When a new session is requested	User name
SessionOnLogout	When a session is closed	User name
SessionOnVarChanged	When a session variable's value is changed	The name and new value of the session variable

Framework event attributes

This table lists and describes the expression functions associated with each framework (Core) event.

Note: Wherever property names are mentioned in this table, <Property#>, as represented here, specifies the name as a string expression. To specify the property name as a constant value, enclose it in double quotes ("...").

Event name	Trigger	Context attributes passed as event parameters, in addition to the object name
IdoOnLoadCollection	INITIATOR()	Composite initiator (period-separated): Type (form or other client) Name (form name or other client ID)
	IDO()	Name of the IDO collection
	PROPERTYNAMES()	List of property names to be retrieved
	POSTQUERYACTIONS()	List of post-query actions
	FILTERSTRING()	Filter string
	RECORDCAP()	Record cap
IdoPostLoadCollection	INITIATOR()	Composite initiator (period-separated): Type (form or other client) Name (form name or other client ID)
	IDO()	Name of the IDO collection
	PROPERTYNAMES()	List of property names that were retrieved
	PROPERTY(1,<PROPERTY1>) ... PROPERTY(N,<PROPERTYM>)	Value of the first property from the first row retrieved ... Value of the M th property from the N th row retrieved
	P(N,<Property>)	Value of the given property of the N th row
	FILTERPROPERTY(N,<Property>)	Value of the given property of the N th row as a string

Event name	Trigger	Context attributes passed as event parameters, in addition to the object name
	FP(N,<Property>)	Value of the given property of the N th row as a string
	FILTERSTRING()	Filter string
	RECORDCAP()	Record cap
	ROWS()	Number of rows that were loaded
IdoOnUpdateCollection	INITIATOR()	Composite initiator (period-separated): Type (form or other client) Name (form name or other client ID)
	IDO()	Name of the IDO collection
	CUSTOMINSERT()	Specification for a custom insert
	CUSTOMUPDATE()	Specification for a custom update
	CUSTOMDELETE()	Specification for a custom delete
IdoOnItemInsert	INITIATOR()	Composite initiator (period-separated): Type (form or other client) Name (form name or other client ID)
	IDO()	Name of the IDO collection
	PROPERTYNAMES()	List of property names to be saved
	PROPERTY(<Property1>) PROPERTY(<PropertyM>)	Value of the first property of the row to be saved into the database Value of the M th property of the row to be saved into the database
	P(<Property>)	Value of the given property
	FILTERPROPERTY(<Property>)	Value of the given property as a string
	FP(<Property>)	Value of the given property as a string

Event name	Trigger	Context attributes passed as event parameters, in addition to the object name
	PROPERTYMODIFIED (<Property1>)	1 if and only if the first property was updated from its original value; otherwise 0
	PROPERTYMODIFIED (<PropertyM>)	1 if and only if the M th property was updated from the original value; otherwise 0
IdoPostItemInsert	INITIATOR()	Composite initiator (period-separated): Type (form or other client) Name (form name or other client ID)
	IDO()	Name of the IDO collection
	PROPERTY(<Property1>) PROPERTY(<PropertyM>)	Value of the first property of the row that was saved into the database Value of the M th property of the row that was saved into the database
	P(<Property>)	Value of the given property
	FILTERPROPERTY(<Property>)	Value of the given property as a string
	FP(<Property>)	Value of the given property as a string
IdoOnItemUpdate	INITIATOR()	Composite initiator (period-separated): Type (form or other client) Name (form name or other client ID)
	IDO()	Name of the IDO collection
	PROPERTYNAMES()	List of property names to be saved
	PROPERTY(<Property1>) PROPERTY(<PropertyM>)	Value of the first property of the row to be saved into the database Value of the M th property of the row to be saved into the database

Event name	Trigger	Context attributes passed as event parameters, in addition to the object name
		Value of the M th property of the row to be saved into the database
	P(<Property>)	Value of the given property
	FILTERPROPERTY(<Property>)	Value of the given property as a string
	FP(<Property>)	Value of the given property as a string
	PROPERTYMODIFIED (<Property1>) PROPERTYMODIFIED (<PropertyM>)	1 if and only if the first property was updated from its original value; otherwise 0 1 if and only if the M th property was updated from the original value; otherwise 0
IdoPostItemUpdate	INITIATOR()	Composite initiator (period-separated): Type (form or other client) Name (form name or other client ID)
	IDO()	Name of the IDO collection
	PROPERTY(<Property1>) PROPERTY(<PropertyM>)	Value of the first property of the row that was saved into the database Value of the M th property of the row that was saved into the database
	P(<Property>)	Value of the given property
	FILTERPROPERTY(<Property>)	Value of the given property as a string
	FP(<Property>)	Value of the given property as a string
IdoOnItemDelete	INITIATOR()	Composite initiator (period-separated): Type (form or other client) Name (form name or other client ID)
	IDO()	Name of the IDO collection

Event name	Trigger	Context attributes passed as event parameters, in addition to the object name
	PROPERTY(<Property1>)	Value of the first property of the row to be deleted from the database
	PROPERTY(<PropertyM>)	Value of the M th property of the row to be deleted from the database
	P(<Property>)	Value of the given property
	FILTERPROPERTY(<Property>)	Value of the given property as a string
	FP(<Property>)	Value of the given property as a string
	PROPERTYMODIFIED()	Always returns 0
IdoPostItemDelete	INITIATOR()	Composite initiator (period-separated): Type (form or other client) Name (form name or other client ID)
	IDO()	Name of the IDO collection
IdoPostUpdate-Collection	INITIATOR()	Composite initiator (period-separated): Type (form or other client) Name (form name or other client ID)
	IDO()	Name of the IDO collection
IdoOnInvoke	INITIATOR()	Composite initiator (period-separated): Type (form or other client) Name (form name or other client ID)
	IDO()	Name of the IDO collection
	METHOD()	Name of the method to be invoked
	METHODPARMS()	Number of parameters in the lists that follow

Event name	Trigger	Context attributes passed as event parameters, in addition to the object name
	METHODPARAM(<n>)	The value of the n^{th} parameter passed to the method, where n is an integer
	FILTERMETHODPARAM(<n>)	The value of the n^{th} parameter passed to the method as a string
IdoPostInvoke	INITIATOR()	Composite initiator (period-separated): Type (form or other client) Name (form name or other client ID)
	IDO()	Name of the IDO collection
	METHOD()	Name of the method to be invoked
	METHODPARMS()	Number of parameters in the lists that follow
	METHODPARAM(<n>)	The value of the n^{th} parameter passed to the method, where n is an integer
	FILTERMETHODPARAM(<n>)	The value of the n^{th} parameter passed to the method
SessionOnLogin	INITIATOR()	Composite initiator (period-separated): Type (form or other client) Name (form name or other client ID)
	USERNAME()	User requesting the session login
SessionOnLogout	INITIATOR()	Composite initiator (period-separated): Type (form or other client) Name (form name or other client ID)
	USERNAME()	User logging off the session
SessionOnVar Changed	INITIATOR()	Composite initiator (period-separated): Type (form or other client)

Event name	Trigger	Context attributes passed as event parameters, in addition to the object name
		Name (form name or other client ID)
	VARIABLENAME()	Name of the session variable
	VARIABLEVALUE()	Value of the session value

Application events

This table lists and describes the application events that ship with the system. Event handlers already exist for these events.

Event name	Trigger	Context attributes passed as event parameters, in addition to the object name
GenericNotify	When an IDO collection is loaded into the system	ToVar, SubjectVar, CategoryParm, BodyVar (To, Subject, Category, and Body of the notify action)
TaskListCheck	When the TaskListCheck event trigger fires from the Event Service	None

Event action types

This table lists and describes the action types available for use when defining event actions.

Note: Adjourning event action types (that is, Prompt, Wait, and Sleep) cannot be assigned to event actions on a transactional event handler.

Mid-tier event action types (that is, Call IDO Method, Dispatch IDO Request, Execute IDO Request, Load Collection, Load IDO Row, and Update Collection) cannot be executed on a synchronous event handler whose event is fired from the database layer (that is, via FireEventSp, or attached to any Session framework event). If this is attempted, the handler fails with an error.

Action type	What it does	Associated event action parameters	Adjourning?
Achieve Milestone	When an IDO collection is loaded into the system	- Condition - State (text) - Image (file)	No
Attach	Allows you to: Attach or link additional files to a record as part of an event action sequence.	Document action (Attach,	No

Action type	What it does	Associated event action parameters	Adjourning?
	- Updated files already attached to a record. - Detach obsolete attachments from a record as part of an event action sequence.	Link, Update, or Detach) - Document name - Document sequence - Document type - Document description - Document (file) extension - Document internal? - Document media type - Document file name - BLOB property - Attachment embedded - Attachment name - Attachment sequence	
Audit	This action type adds an entry to the Audit Log. This action type can be useful when you want to track various changes of values or milestones achieved, for example.	- Description - Key value - Old value - New value	No
Branch	Goes to a specified other step when a specified condition is met	- Condition - Destination	No
Call Database Method	Calls a stored procedure, passing any parameters required by or returned from the stored procedure	- Stored procedure name - Parameters	No
Call IDO Method	Invokes an IDO method, passing any parameters required by or returned from the IDO method	- IDO - Method - Parameters	No
Call Web Service	Invokes a Web service, passing any parameters required by or returned from the Web service	URL	No

Action type	What it does	Associated event action parameters	Adjourning?
		• Operation (Method) • Parameters • Output parameters	
Dispatch IDO Request	This action type dispatches an IDO request to execute some action on a system to which the user is not currently connected or signed in. This IDO request can be, for example, to load a collection, update a collection, or invoke an IDO method on an IDO runtime service at another "enterprise" (for example, one of your customers or vendors who also uses the same Mongoose application) or with another site in your own enterprise. This action type opens a session on the remote database server, performs the requested action, and closes the session. If appropriate, this action type also receives any response data from the remote database server and places it in a return variable or return parameter. You can then use the Transform XML action type to parse the return data.	• IDO Request XML • URL • Configuration Name • Username • Password • Use Local Password • Response	No
Execute IDO Request	This action type executes an IDO request on the system to which the user is currently connected or signed in. This IDO request can be, for example, to load a collection, update a collection, or invoke an IDO method. This action type: Optionally, opens a new session (closing it when the requested action is finished). Performs the requested action. If appropriate, receives any response data generated by the IDO action and places it in a return variable or return parameter. You can then use the Transform XML action type to parse the return data.	• IDO Request XML • Response	No
Fail	Ends the handler execution with an error status when a specified condition is met and, optionally, specifies a failure or error	• Condition	No

Action type	What it does	Associated event action parameters	Adjourning?
	message to be displayed on the Event Status form	Result (error message)	
Finish	This action type marks the event handler as being finished successfully, optionally when a specified condition is met. Optionally, it specifies a failure or error message to be displayed on the Event Status form.	- Condition - State - Image - Result	No
Generate Event	Generates another event, optionally when a specified condition is met, passing any parameters required by or returned from the other event	- Condition - Event name - Parameters - Output parameters - Synchronous - Transactional	No
Goto	This action type directs the handler flow to an action step other than the next one in the sequence. This can be useful when you want to redirect the flow based on some condition that has been met in another action step.	Destination	No
Load Collection	This action type loads records from a specified collection associated with an IDO, up to the record cap. You can use various options on this form to limit the amount of data returned by this event action. The data retrieved by this action is held in a "result set," which you must then refer to for further action	- IDO - Properties - Filter - Order By - Custom Load - Custom Load Params - Record cap - Result Set Assignment	No
Load IDO Row	Loads a specified record (row) from a collection associated with an IDO	- IDO - Properties - Filter - Order By - Custom Load - Custom Load Params	No

Action type	What it does	Associated event action parameters	Adjourning?
		• Output	
Notify	This action type sends a system-generated message to recipients (system users), optionally when a specified condition is met. This notification is transmitted internally within the WinStudio system, without having to use external email or other messaging systems.	• Condition • To • Cc • Subject • Category • Body • HTML Body • Attach • Save in Sent Items • Filter form • Filter spec • Payload Access • Modified Payload Access • Unmodified Payload Access	No
Prompt	This action type sends a system-generated message to recipients (system users) and awaits a response from each. It also sends an e-mail to recipients for whom Send E-mail Prompts is specified in the Users form. To send emails for prompts, you must have an SMTP server designated on the Intranets form and automatic relaying of email messages configured on the SMTP server. This action type counts votes according to a specified rule. If no response or an insufficient response is received within a certain period, it optionally goes to an alternate action, such as these: Perform an alternate prompt to an alternate set of recipients Set a default and branch to the subsequent event action Go directly to the subsequent event action (the same as if a sufficient number of responses were received)	• Condition • To • Cc • Subject • Category • Body • HTML Body • Attach • Save in Sent Items • Question • Choices • Voting rule • Preferred choice • Minimum • Quorum • Wait for Quorum • Timeout	Yes

Action type	What it does	Associated event action parameters	Adjourning?
		• Timeout destination • Filter form • Filter specification • Entry form • Payload access • Modified payload access • Unmodified payload access	
Run Background Task	Places a task on the background task queue	• Task name • Task parameters • Task status • Task number	No
Send Email	This action type sends an external e-mail to recipients (system users). To use this action, you must have an SMTP server designated on the Intranets form and automatic relaying of email messages configured on the SMTP server. This action allows you to also include attachments with the email.	• Condition • To • Reply To • Cc • Subject • Body • Body is in HTML format • Attach	No
Set Attributes	Sets text that displays in the Event Title field of the Event Status form	Title	No
Set Values	Sets event variables and/or event parameters to given values or expressions, optionally when a specified condition is met	• Condition • Variables • Parameters • Session variables • Method parameters • Value • Row • Properties	No

Action type	What it does	Associated event action parameters	Adjourning?
Sleep	Waits until a specified period of time has elapsed before proceeding to the next action	Interval	Yes
Transform XML	This action type takes XML as input and transforms it for use elsewhere. This action type transforms specified XML, based on transformation rules contained in the XSLT, with the resulting XML stored in a result variable or parameter. A transform XML operation can range from simple (picking out a single specific element from a large document, to be used as part of a SUBSTITUTE expression in a Notify action) to complex (translating an entire document from one format to another, to be used as input to some other process).	• XML • XSLT • Result XML	No
Update Collection	This action type updates an IDO collection by inserting, updating, or deleting records within that collection. Using this action type, you can either update a collection directly from a specified IDO, or you can update a collection that is already loaded into a named result set.	• Update using IDO or loaded collection • IDO • Result Set ID • Action • Properties • Row • Optimistic Lock • Commit	No
Wait	This action type instructs the system to wait for a specified condition to be met before proceeding to the next action. If the condition does not become true within a certain time, optionally goes to an alternate action. A retest interval of zero (0) means that the condition is never retested. A timeout setting of zero (0) means that the event action never times out, that is, that it waits indefinitely.	• Condition • Retest interval • Timeout • Timeout Destination	Yes

Event action parameters

The **Event Action Parameters** table lists (in alphabetic order) and describes all the event action parameters and their functions.

When using the **Event Action Parameters** table, keep these points in mind:

This...	Refers to this...
ALLCAPS Parentheses () Equal signs (=) Commas	Functions and punctuation to be specified and stored verbatim
Content in italics	Content that varies according to the context, for example, for a SUBJECT(expr) parameter, expr represents the actual content that is sent in the message
Ellipses (...)	Indicates more like the preceding element, separated by commas Parentheses indicate that the repetition of elements is optional.
scalarExpr	Scalar (numeric, string, date, or typeless) event expressions
<i>Boolean</i>	A Boolean event expression
<i>name</i> <i>var</i> (variable) <i>parm</i> (parameter) <i>collection</i>	Identifiers used for named or contextual purposes These identifiers can consist of: • An initial uppercase or lowercase letter or underscore (_), followed by • Any combination of letters, numerals, underscores, square brackets ([]), and/or periods.
<i>Prop</i> (property)	This specifies the property name as a string expression. To specify the property name as a literal value, enclose it in double quotation marks (" ").
action	Event action sequence number (values to be adjusted automatically if actions are resequenced)

The following is the Event Actions Parameters table:

Parameter	Used with action types	Description/Purpose	Syntax
Action	Update Collection	Indicates what the system is to do with the data	One of these: ACTION(INSERT) ACTION(UPDATE) ACTION(DELETE)
Attach	• Notify	Specifies which attached documents (files) are to be forwarded along with	ATTACH(option)

Parameter	Used with action types	Description/Purpose	Syntax
	- Prompt - Send Email	the messages generated by the current action Has these possible options: - ALL - NONE - INTERNAL - EXTERNAL	
Attachment Embedded	Attach	Specifies whether the attachment is linked to e-mail messages in such a way that it can be embedded into the HTML version of the message body	ATTACHMENTEMBEDDED (<i>Boolean</i>)
Attachment Name	Attach	This parameter specifies the name by which this document is identified within the application event system. If this value is not specified for these actions, a default value is constructed by concatenating the Document Name with the Document Sequence, separated by a period; for example: MyDocument.000000001.	ATTACHMENTNAME (<i>scalarExpr</i>)
Attachment Sequence	Attach	This parameter specifies the name of the event variable that is to receive the attachment sequence number of the attached, linked, or updated document. This variable can be used in a subsequent action to look up the resulting Attachment Name or other attributes of the document	ATTACHMENTSEQ(<i>ID</i>)
BLOB Property	Attach	This parameter specifies a previously retrieved BLOB (binary large object) property containing the document to be attached or to replace the currently attached document. It must be used with either a P or PROPERTY parameter.	DOCBLOB(P PROPERTY (<i>scalarExpr</i>))
Body	- Notify - Prompt - Send Email	This parameter indicates the content that is to be generated for the body of the message, whether for an internal system message or an email. The CLIENTSUBSTITUTE()	BODY(<i>scalarExpr</i>)

Parameter	Used with action types	Description/Purpose	Syntax
		function can be used to build an expression that is automatically translated locally for each Inbox user and email prompt recipient (applicable only for Notify and Prompt).	
Body is in HTML format	Send Email	When this parameter is selected (set to TRUE), it specifies that the body of the email is to be rendered using HTML.	HTMLFORMAT(Boolean)
Category	• Notify • Prompt	This parameter specifies a category to group the message with others of a similar purpose. The CLIENTSUBSTITUTE() function can be used to build an expression that is automatically translated locally for each Inbox user and email prompt recipient.	CATEGORY(scalarExpr)
Cc	• Notify • Prompt • Send Email	This parameter indicates a list of secondary recipients of the message, delimited by semi-colons (;). Do not leave spaces between multiple recipients in the list. For Notify and Prompt actions, recipients are identified by their user IDs on the system. For Send Email actions, recipients are identified by their email addresses.	CC(scalarExpr) Examples: CC("jdoe;tsmith") CC("jdoe@provider.net") Note that literal values for user IDs or e-mail addresses must be enclosed in double quotes ("").
Choices	Prompt	This parameter indicates a comma-separated list of return values and button labels (or link labels for e mail). Note that the parser strips out whitespace, so it is OK to include spaces between items in the list. In each pair, the first item is the value to be returned if the recipient selects that option. The second item is the label that appears on the selection button/link. For the button/link labels, you can use strings defined in the Strings table or literal values. For emails, strings are translated and formatted based on the Default Language specified for the recipient	CHOICES(scalarExpr) Examples: CHOICES("" + GC(YesValue) + ",sYes," + GC(NoValue) + ",sNo") Note that the expression must be enclosed in double- quotes ("").

Parameter	Used with action types	Description/Purpose	Syntax
		in the Users form. On the Inbox form, strings are translated and formatted based on the Language setting specified on the Runtime Behavior Settings. The scalarExpr must consist of an even number of list items. The default button/link labels are Approve and Disapprove, with return values of 1 and 0 respectively.	
Commit	Update Collection	Determines whether the changes made up to this point in the action flow are to be committed to the database	COMMIT(<i>Boolean</i>)
Condition	• Achieve Milestone • Branch • Fail • Finish • Generate Event • Notify • Prompt • Send Email • Set Values • Wait	Sets up the condition that determines when the action is executed	CONDITION(<i>Boolean</i>) Examples: CONDITION(NOT PROPERTYMODIFIED("CreditLimit")) CONDITION(P("Quantity") < 500)
Configuration Name	• Dispatch IDO Request	Determines the configuration on a remote server from which you want to make an IDO request	CONFIGNAME(<i>scalarExpr</i>)
Custom Load	• Load Collection • Load IDO Row	Indicates the name of the custom load method to use in loading the collection data	CUSTOMLOAD(<i>scalarExpr</i>)

Parameter	Used with action types	Description/Purpose	Syntax
Custom Load Params (Custom load method parameters)	- Load Collection - Load IDO Row	Indicates the parameters to be passed to the custom load method	CUSTOMLOADPARMS(<i>scalarExpr</i>)
Description	Audit	Defines the description to be passed to the audit log. The CLIENTSUBSTITUTE() function can be used to build an expression that is automatically translated locally for each user viewing the Audit Log form	DESC(<i>scalarExpr</i>)
Destination	- Branch - Goto	Indicates the action sequence number of the event action the system is to go to when executing this action	DEST(action) Example: DEST(40)
Document Action	Attach	This parameter determines what type of action is to be taken with the attached document (file). Options include ATTACH, LINK, UPDATE, DETACH.	DOCACTION(<i>action</i>)
Document Description	Attach	Specifies a brief description of the document type or attached document (file)	DOCDESC(<i>scalarExpr</i>)
Doc Extension	Attach	This parameter indicates the extension of the document (file) to be attached for view or export. If this value is not specified, the document extension from the associated Document Type field is used.	DOCEXT(<i>scalarExpr</i>)
Document File Name	Attach	This parameter specifies a file on the system that is to be attached or updated with a new version. The specified file must reside on the database server.	FILENAME(<i>scalarExpr</i>)
Document Internal	Attach	This parameter restricts access to an attached document (file) so that it cannot be accessed by means of the Customer Portal or the Vendor Portal.	DOCINTERNAL (<i>Boolean</i>)

Parameter	Used with action types	Description/Purpose	Syntax
		The Send Email action ignores files for which this option is set. Internal files can only be sent to email recipients using a Notify or Prompt action.	
Document Media Type	Attach	This parameter indicates the Internet media type (formerly MIME, or Multipurpose Internet Mail Extensions) to be associated with files of that document type when attached to an email message. This value suggests to the email client how to present the document, either in line with the message body, or when previewed or opened (for example, image/jpeg or application/pdf).	DOCMEDIATYPE (<i>scalarExpr</i>)
Document Name	Attach	This parameter specifies the name of the document (file) that is attached or is to be attached. This name is an internal name, used only to identify the document in the list of documents.	DOCNAME(<i>scalarExpr</i>)
Document Sequence	Attach	Specifies a number identifier to distinguish multiple documents (files) having the same Document Name	DOCSEQ(<i>int</i>)
Document Type	Attach	Indicates the type of the document (file) to be attached	DOCTYPE(<i>scalarExpr</i>)
Entry Form	Prompt	Indicates the name of the form to be launched to edit the event variables for the event handler	ENTRYFORM(<i>scalarExpr</i>)
Event Name	Generate Event	Indicates the name of the event to be generated	EVENTNAME(<i>scalarExpr</i>)
Filter or Filter Spec	• Load IDO Row • Load Collection • Notify • Prompt	This parameter identifies the filter to use in restricting data retrieval. This parameter is not valid with CUSTOMLOAD. Functionally, this is the same as the Permanent Filter Expression property in WinStudio development.	FILTER(<i>scalarExpr</i>)

Parameter	Used with action types	Description/Purpose	Syntax
Filter Form	• Notify • Prompt	Indicates the name of the form to be launched to display information about the row represented by the event variables for the event handler	<code>FILTERFORM(<i>scalarExpr</i>)</code>
Html Body	• Notify • Prompt	This parameter specifies the content that is to be generated for the body of the message and formatted in HTML for an email. The content in this field is used only for message content being sent by email; internal system messages ignore this field.	<code>HTMLBODY</code> (<i>"HTMLcontent"</i>)
IDO	• Call IDO Method • Load Collection • Load IDO Row • Update Collection	Indicates the name of the IDO on which to perform the operation	<code>IDO(<i>scalarExpr</i>)</code>
IDO Request XML	• Dispatch IDO Request • Execute IDO Request	Contains the correctly formatted XML content of the IDO request	<code>IDOREQUEST(<i>scalarExpr</i>)</code>
Image	• Achieve Milestone • Finish	This parameter specifies the name of the graphics file to display with the event in the status forms. This path and file name must be in the format: <i>Drive:\folder\subfolder\... myGraphicFile.ext</i> The specified graphics file must be one of the supported types.	<code>IMAGE(<i>scalarExpr</i>)</code> Example: <code>IMAGE("C:\WINDOWS\Web\Wallpaper\Tulips.jpg")</code>
Interval	Sleep	Specifies the amount of time (in seconds) that the system is to pause before moving to the next action	<code>INTERVAL(<i>numericExpr</i>)</code>

Parameter	Used with action types	Description/Purpose	Syntax
Key value	Audit	Specifies the text that is to appear in the Primary Key column of the Audit Log form when this action runs	KEY(<i>scalarExpr</i>)
Method	Call IDO Method	Specifies the name of the IDO method to call	METHOD(<i>scalarExpr</i>)
Method Parameters	Set Values	This parameter changes method parameter values during an IdoOnInvoke event. paramNumber is the 1- based position of the parameter to change. scalarExpr is the value to which the parameter is to be set. For multiple parameters, include them as a comma-separated list.	SETMETHODPARAM-VALUES(<i>paramNumber</i> = <i>scalarExpr</i> [, ...])
Minimum (Count or Percentage)	Prompt	Count indicates the minimum number of votes required for a choice to win. It must be used in conjunction with a ConditionalPlurality, or MinimumCount voting rule. Percentage indicates the minimum percentage of the total vote (based on recipients of the message) required for a choice to win. It must be used in conjunction with a MinimumPercentage voting rule.	MINIMUM(<i>numericExpr</i>)
Modified Payload Access	- Notify - Prompt	This parameter specifies how message payload elements that represent modified properties are handled: Default – The modified parts of the payload are visible; the recipient can further modify any of their contents. Hidden – The modified parts of the payload are hidden, for example, for a secret ballot. Read-Only – The modified parts of the payload are display only, so the recipient cannot further modify the contents. This specification applies to event message variables with names in the form Row.Property that have a corresponding	One of these: MODIFIEDPAYLOAD-ACCESS(DEFAULT) MODIFIEDPAYLOAD-ACCESS(HIDDEN) MODIFIEDPAYLOAD-ACCESS(READONLY)

Parameter	Used with action types	Description/Purpose	Syntax
		Row.Property.Modified element with a value of 1. This setting is overridden by settings for the Default Access (Event Variable Groups form) or Access (Event Actions form, Variables grid) for the same property name.	
New Value	Audit	This parameter specifies the text that is to appear in the New Value column of the Audit Log form when this action runs. It represents the new value of something that was changed.	NEW(<i>scalarExpr</i>)
Old Value	Audit	This parameter specifies the text that is to appear in the Old Value column of the Audit Log form when this action runs. It represents the original value of something that was changed.	OLD(<i>scalarExpr</i>)
Operation	Call Web Service	Specifies the name of the Web service to call	METHOD(<i>scalarExpr</i>)
Optimistic Lock	Update Collection	Determines whether optimistic locking will be employed for safety	OPTIMISTICLOCKING(<i>Boolean</i>)
Order By	- Load Collection - Load IDO Row	This parameter specifies the name of a valid property to be used for the sort order on the collection when it is loaded and displayed. For example, if you select CreateDate as the property for this field, the system sorts the collection according to the dates on which transactions were recorded. You can use multiple properties, and the system prioritizes multiple properties in the order in which they are listed. In other words, the first property would be the primary property for sorting; the next property would be the secondary property for sorting; and so on. Multiple properties must be specified as a comma-separated list.	ORDERBY(<i>scalarExpr</i>) ORDERBY("PoLine") ORDERBY("CoRelease ASC, Coltem") Note that, in the second example, there is an attribute "ASC" specified. This tells the system to sort the list in ascending order, which is also the default. You could instead use DESC to tell the system to sort the list in descending order.

Parameter	Used with action types	Description/Purpose	Syntax
		This parameter is not valid with CUSTOMLOAD.	
Output	Load IDO Row	This parameter specifies where the system is to place the IDO row data. You can use return variables (RV) or return parameters (RE). Multiple outputs can be designated, using a comma-separated list of return value-name pairs. A separate value-name pair is required for each property being returned.	SET(RV(<i>var</i>) RE(<i>param</i>) = <i>scalarExpr</i> [, ...]) Each RV or RE represents a name-name pair. <i>var</i> is the name of an event variable; <i>param</i> is the name of an event parameter; <i>scalarExpr</i> evaluates to a property name of the IDO that was loaded. Example: SET(RV(myVar) = "")
Output Parameters	- Generate Event - Call Web Service	This parameter specifies where the system is to place the output (return) data from the event or Web service. You can use return variables (RV) or return parameters (RE). Multiple output parameters can be returned, using a comma-separated list of return value-name pairs. Each name is the name of a node in the XML response document.	SET(RV(<i>var</i>) RE(<i>param</i>) = <i>scalarExpr</i> [, ...]) Each RV or RE represents a name-name pair. In the case of a Generate Event action, the <i>scalarExpr</i> evaluates to the name of an event output parameter of the generated event. In the case of a Call Web Service action, the <i>scalarExpr</i> evaluates to the name of a node in the response XML. Example: SET(RV(RecordsDone) = "rows_succeeded", RE(PostResult) = "result_message")
Parameters[1]	Call Database Method	This parameter specifies a comma-separated list of parameters required by the IDO method or stored procedure. If an event variable is named using this function and that variable does	PARMS(<i>scalarExpr</i> RV(<i>var</i>) RE(<i>param</i>)[, ...])

Parameter	Used with action types	Description/Purpose	Syntax
	Call IDO Method	not already exist for the current handler, the system creates it automatically as a non-persistent variable. If a parameter is set using this function, and that parameter was not passed into the current event, the system creates it automatically as an output parameter.	
Parameters[2]	- Call Web Service - Generate Event	This parameter specifies a comma-separated list of parameters to be passed as input to the event or Web service. These parameters must be in the form of name-value pairs.	PARMS(name= <i>scalarExpr</i> [, ...])
Parameters[3]	Set Values	This parameter specifies parameters as name-value pairs. For multiple parameters, include them as a comma-separated list of name-value pairs. If a parameter is set using this function, and that parameter was not passed into the current event, the system creates it automatically as an output parameter.	SETPARMVALUES(<i>paramName</i> = <i>scalarExpr</i> [, ...])
Password	Dispatch IDO Request	Indicates the password associated with the user ID specified in the User Name field	PASSWORD(<i>scalarExpr</i>)
Payload Access	- Notify - Prompt	This parameter sets the recipient's access to payloads for variables whose name is in the form Row.Property: Default – Payload variables are visible and the recipient can modify any of their contents. Hidden – The payload is hidden, for example, because it contains no information relevant to the voting. Read-Only – The entire payload is display only, so the recipient cannot modify any of the contents.	One of these: PAYLOADACCESS (DEFAULT) PAYLOADACCESS (HIDDEN) PAYLOADACCESS (READONLY)

Parameter	Used with action types	Description/Purpose	Syntax
		This specification applies to event message variables with names in the form Row.Property. This setting is overridden by settings for the Default Access (Event Variable Groups form) or Access (Event Actions form, Variables grid) for the same property name.	
Preferred Choice	Prompt	This parameter specifies the preferred choice among multiple voting choices. This parameter must be used in conjunction with one of the PreferredChoice voting rules.	PREFCHOICE(<i>scalarExpr</i>)
Properties [1]	- Load Collection - Load IDO Row	Identifies a comma-separated list of properties to load from the IDO collection that is specified in the IDO field	PROPERTIES(<i>scalarExpr</i>)
Properties [2]	- Update - Collection	Specifies the primary keys identifying the rows to be updated, followed by the properties for which you want to set values, both as name-value pairs, in a comma-separated list.	SETPROPVALUES(<i>propName = scalarExpr</i> [, ...])
Properties [3]	Set Values	This parameter specifies the properties for which you want to set values as name-value pairs. For multiple properties, include them as a comma-separated list. Used only with IdoOnItemInsert, IdoOnItemUpdate, and IdoOnItemDelete framework (Core) events.	SETPROPVALUES(<i>propName = scalarExpr</i> [, ...])
Question	Prompt	This parameter specifies the content of the direct question that recipients are to respond to when they receive the message. This question appears in the Question field on the Response tab of the Inbox form. The CLIENTSUBSTITUTE() function can be used to build an expression	QUESTION(<i>scalarExpr</i>)

Parameter	Used with action types	Description/Purpose	Syntax
		that is automatically translated locally for each Inbox user and email prompt recipient.	
Quorum	Prompt	This parameter specifies the number of votes to await before closing voting, expiring the remaining unvoted messages, and continuing to the subsequent event action. If this field contains a value, the number overrides the automatic quorum algorithm used to calculate whether further votes are required to make the VOTINGRESULT() non-blank. If the Quorum value is positive, it implies that Wait for Quorum is TRUE. If the Quorum value is non-positive, it implies that Wait for Quorum is FALSE.	QUORUM(<i>numericExpr</i>)
Record Cap	Load Collection	This parameter designates the maximum number of records to be returned when the collection is loaded. The value you set for this field overrides any system or personal settings when executing this handler.	RECORDCAP(<i>numericExpr</i>) Example: RECORDCAP(500)
Reply To	Send Email	Specifies the email addresses of any users or email recipients who are to receive replies to the email generated by this action	REPLYTO(<i>scalarExpr</i>)
Response	- Dispatch IDO Request - Execute IDO Request	Specifies the type—return variable (RV) or return parameter (RE)—and name of the container that is to hold the XML response (RESULT) data from the IDO request	SET(RV(<i>var</i>) = RESULT) SET(RE(<i>param</i>) = RESULT)
Result	- Finish - Fail	This parameter specifies the content of the message that is to be displayed if the event handler fails or finishes at this point. This message displays in the Results field of the Event Status form.	RESULT(<i>scalarExpr</i>) Example: RESULT("The transaction was disapproved by the purchasing manager.")

Parameter	Used with action types	Description/Purpose	Syntax
		The CLIENTSUBSTITUTE() function can be used to build an expression that is automatically translated locally for each user viewing the Event Status form.	
Result Set Assignment	Load Collection	Designates the name (ID) of the result set where the system is to put the loaded collection	SET(R(<i>ID</i>) = RESULT)
Result Set ID	Update Collection	Identifies the name (ID) of a result set that was previously loaded using a Load Collection action type	COLLECTION(<i>ID</i>)
Result XML	Transform XML	Specifies the return type— return variable (RV) or return parameter (RE)—and name of the variable or parameter that is to receive the resulting XML transformation output	SET(RV(<i>var</i>) = RESULT) SET(RE(<i>param</i>) = RESULT)
Retest Interval	Wait	Designates the amount of time, in seconds, that the system is to wait before retesting for the condition	RETEST(<i>numericExpr</i>)
Row	- Update Collection - Set Values	Indicates the row (record) to be updated in the collection	ROW(<i>numericExpr</i>)
Save in Sent Items	- Notify - Prompt	Determines whether a copy of the message is to be saved in the originator's Sent Items folder	SAVEMESSAGE(<i>Boolean</i>)
Session Variables	Set Values	This parameter specifies session variables as name-value pairs. For multiple variables, include them as a comma-separated list.	SETGLOBVALUES (<i>varName</i> = <i>scalarExpr</i> [, ...])
State	- Achieve Milestone - Finish	This parameter indicates the text to display on the Event Status form, Event State field, when the action runs. The CLIENTSUBSTITUTE() function can be used to build an expression that is automatically translated locally for each user viewing the Event Status form.	STATE(<i>scalarExpr</i>)

Parameter	Used with action types	Description/Purpose	Syntax
Stored Procedure	Call Database Method	Specifies the name of the stored procedure to call	METHOD(<i>scalarExpr</i>)
Subject	- Notify - Prompt - Send Email	This parameter specifies the subject line for the message, whether for an internal system message or an email. The CLIENTSUBSTITUTE() function can be used to build an expression that is automatically translated locally for each Inbox user and email prompt recipient (applicable only for Notify and Prompt).	SUBJECT(<i>scalarExpr</i>)
Synchronous	Generate Event	Instructs the system whether to treat the generated event as a synchronous event TRUE = Synchronous FALSE = Asynchronous	SYNCHRONOUS(<i>Boolean</i>)
Task Name	Run Background Task	Specifies the name of the task Background to be placed on the Task background task queue, as defined on the Background Task Definitions form	TASKNAME(<i>scalarExpr</i>)
Task Number	Run Background Task	Indicates the name of the event variable that is to receive the task number as output from the Task Manager when the task is placed on the background task queue	TASKNUMBER(<i>ID</i>)
Task Parameters	Run Background Task	Specifies any parameters that the background task requires to run correctly	TASKPARMS(<i>scalarExpr</i>)
Task Status	Run Background Task	This parameter specifies the task status that the status is to have initially when it is placed on the Background Task Queue. Ready- Places the task on the Background Task Queue to be run as soon as possible. Waiting- Places the task on the Background Task Queue to be run only when the scheduling	Either of these: TASKSTATUS(READY) TASKSTATUS(WAITING)

Parameter	Used with action types	Description/Purpose	Syntax
		requirements for that task, as set on the Background Task Queue form.	
Timeout	- Prompt - Wait	The amount of time (in seconds) that the system is to wait for a response (in the case of a Prompt action) or that it must wait before retesting (in the case of a Wait action)	TIMEOUT(<i>numericExpr</i>)
Timeout Destination	- Prompt - Wait	This parameter specifies which action sequence step the system should move to when the time- out limit has been reached. The action sequence step is identified by the Action Sequence number on the Event Actions form.	DEST(<i>action</i>)
Title	Set Attributes	Specifies the Event Title as you want it to appear on the Event Status form, when the event is running or finished	SET(EVENTTITLE = <i>scalarExpr</i>)
To	- Notify - Prompt - Send Email	This parameter designates the list of the primary recipients of the message, delimited by semi- colons (;). Do not leave spaces between multiple recipients. For Notify and Prompt actions, recipients are identified by their user IDs. For Send Email actions, recipients are identified by their email addresses.	TO(<i>scalarExpr</i>) Examples: TO("jdoe;tsmith") TO("jdoe@provider.net") Note that literal values for user IDs or email addresses must be enclosed in double quotes ("").
Transactional	Generate Event	Instructs the system whether to treat the generated event as transactional	TRANSACTIONAL(<i>Boolean</i>)
Unmodified Payload Access	- Notify - Prompt	This parameter specifies how message payload elements that represent unmodified properties are handled: - Default – The unmodified parts of the payload are visible; the recipient can modify any of their contents. - Hidden – The unmodified parts of the payload are hidden. This allows the recipient to focus on the	One of these: - UNMODIFIEDPAYLOAD ACCESS(DEFAULT) - UNMODIFIEDPAYLOAD ACCESS(HIDDEN) - UNMODIFIEDPAYLOAD ACCESS(READONLY)

Parameter	Used with action types	Description/Purpose	Syntax
		modified parts of the payload when evaluating the question. Read-Only – The unmodified parts of the payload are display only, so the voter cannot modify the contents. This specification applies to event message variables with names in the form Row.Property that do not have a corresponding Row.Property.Modified element with a value of 1. This setting is overridden by settings for the Default Access (Event Variable Groups form) or Access (Event Actions form, Variables grid) for the same property name.	
Update Using IDO/Update Loaded Collection	Update Collection	This parameter specifies what IDO collection is to be updated (mutually exclusive choices): - Update Using IDO allows you to specify an IDO that is not currently loaded into the form. - Update Loaded Collection (default) updates a collection that has already been loaded into a named result set. Each option enables a different set of parameter options	N/A
URL	- Call Web Service - Dispatch IDO Request	This parameter identifies the URL that is to receive the request. The URL must be in standard HTTP format.	URL(<i>scalarExpr</i>)
Use Local Password	Dispatch IDO Request	This parameter instructs the system whether to automatically retrieve the password associated with the user ID (User Name) and include it in the IDO request. TIP: If you want to use this option, you must make sure that the remote system has an identical login (user	USELOCALPASSWORD(<i>Boolean</i>)

Parameter	Used with action types	Description/Purpose	Syntax
		name and password) to the one you want to use from the current local system.	
User Name	Dispatch IDO Request	This parameter designates a valid user ID for the remote system configuration. This allows a session to open. TIP: Make sure you specify a user ID that has permissions and licensing sufficient to do whatever you are requesting from the IDO on the remote system.	USERNAME(<i>scalarExpr</i>)
Value	Set Values	This parameter is used only for the SessionOnVarChanged framework event. It specifies a value to be assigned to the session variable that, when changed, generated the event. This value replaces whatever value was requested by the process that initiated the change. In effect, this overrides the original change request behind the scenes. For example, suppose you have an event that is set to be executed whenever a user tries to reset a session variable. This event can be used to override the user-initiated change and reset the variable to whatever value you designate here.	SETVALUE(<i>scalarExpr</i>)
Variables	Set Values	This parameter specifies variable definitions as name-value pairs. For multiple variables, include them as a comma-separated list. If a variable does not exist when the event is executed, the system creates the variable when needed. TIP: Including a null value for a variable (such as myVar =) generates a syntax error. To designate a null value for the value of a variable, use a double set of quotation marks (myVar = "").	SETVARVALUES(<i>varName</i> = <i>scalarExpr</i> [, ...])

Parameter	Used with action types	Description/Purpose	Syntax
Voting Rule	Prompt	This parameter designates the method to be used for counting and evaluating votes. Possible values include: • Majority • Plurality • ConditionalPlurality • MinimumCount • MinimumPercentage • EarliestResponse • PreferredChoice • MinimumCountPreferredChoice • MinimumPercentagePreferredChoice	VOTINGRULE(<i>rule</i>) Example: VOTINGRULE(Plurality)
Wait for Quorum	Prompt	This parameter specifies whether to await a quorum of responses before continuing to the subsequent event action on a prompt action: FALSE - Voting remains open, no messages are expired, and Timeout and Timeout Destination are ignored. TRUE - Execution is adjourned until a quorum is attained. Default - When Quorum is not specified, the default is TRUE. When Quorum is specified with a positive value, the default is TRUE. When Quorum is specified with a non-positive value, the default is FALSE.	WAITFORQUORUM(<i>Boolean</i>)
XML	Transform XML	This parameter specifies the XML code (content) to be transformed. The result can be retrieved using the RESULT function.	XML(<i>scalarExpr</i>)
XSLT	Transform XML	Designates the stylesheet (XSLT) or a reference to the XSLT file containing the transformation rules (using the FILECONTENTS function)	XSLT(<i>scalarExpr</i>)

Expression functions

Standard functions

To construct the expressions used in Event Action Parameters, use the standard expression functions.

When using the standard expression functions table, keep these points in mind:

This...	Refers to this...
<i>FUNCTION(value)</i>	Syntax for functions and values
Content in italics	Content that varies according to the context, for example, for a V(varName) function, varName represents the actual value (the variable name) to be used by the V (variable) function
Use of double quotes	The identifiers listed as arguments below are not enclosed in double-quotes (" ") and should not be unless they are cast as literal string values in the resulting expression

The table that follows lists standard expression functions.

Function/Syntax	What it does	Arguments	Return data type
ACTIONSEQ()	Returns the design-time sequence of the currently executing event action	N/A	Numeric
ACTIONTYPENAME()	This function returns the English-like name of the currently executing event action's type. This function returns the same value as the T- SQL function <code>dbo.EventActionTypeNa me()</code> when it is passed the <code>EventAction.ActionType</code> for this event action.	N/A	String
ANYHANDLERS-FAILED()	Indicates whether any event handlers for the running event have exited with a status of Failure	N/A	Boolean
APPNAME()	This returns the name of the application to which the user is currently signed in. TIP: The name	N/A	String

Function/Syntax	What it does	Arguments	Return data type
	of the application is set in the Configuration Manager, Edit Application dialog box.		
BEGINDATE()	Returns the date and time the event began executing		Date/Time
CAST(<i>expr</i> AS <i>DATATYPE</i>)	This causes the system to treat the specified expression as the designated data type. For example, CAST(P("CreditLimit") AS STRING) would cause the system to treat the value of the property "CreditLimit" as a string, rather than as a number.	• <i>expr</i> – Expression of any data type • AS – Keyword, literal • DATATYPE – • Keyword (must be STRING, DATE, or NUMBER)	Varies
CEILING()	Returns the least integer greater than or equal to a specified numeric expression Note: This function is equivalent to CEILING in T-SQL.	N/A	Numeric
CLIENTSUBSTITUTE	This creates a string combined of literal values and replacements for which values are substituted at run time. CLIENTSUBSTITUTE() allows inclusion of literal commas and parentheses in the string. You can concatenate the string that results from evaluating the CLIENTSUBSTITUTE() expression with other strings. You can pass the string to SUBSTITUTE to build a larger expression, for	• <i>pattern</i> – Evaluated into a string name • <i>arg</i> – Each argument is evaluated into a string, passed through WinStudio literal massaging, which converts commas and parentheses into special characters, and, unless the argument consists of an embedded STRINGS() or • FORMAT() call, is wrapped in ~LIT~(). The resulting strings are separated by commas, in the same order given, and wrapped in a FORMAT() statement. This allows the presence of commas	String

Function/Syntax	What it does	Arguments	Return data type
	example, BODY(), that contains the unchanged ~LIT~ () expressions.	and/or parentheses within the arguments. WinStudio can still parse the arguments as originally separated.	
COMPANYNAME()	Returns the name of the company associated with the application to which the user is currently signed in Tip: The name of the company is set using the Configuration Manager, Edit Application dialog box.	N/A	String
CONFIGNAME()	Returns the configuration name associated with the running event	N/A	String
CURDATETIME()	Returns the current date and time, corrected for the server's time zone (except for the database server) Note: This function is equivalent to dbo.GetSiteDate(getdate()) in T-SQL.	N/A	Date/Time
CUSTOMDELETE()	Returns the specification for a Custom Delete attribute	N/A	String
CUSTOMINSERT()	Returns the specification for a Custom Insert attribute	N/A	String
CUSTOMUPDATE()	Returns the specification for a Custom Update attribute	N/A	String
DATE(<i>year, month, day</i>) or DATE(<i>year, month, day, hour, minute, second, millisecond</i>)	This returns a date and/or time with the given attributes. You must use one of the two syntax models, or the system returns an error.	<i>year</i> – A four-digit integer <i>month</i> – A one-digit or two-digit integer (1–12) <i>day</i> – A one-digit or two-digit integer (1–31)	Date/Time

Function/Syntax	What it does	Arguments	Return data type
		• <i>hour</i> – A one-digit or two-digit integer (1–23) • <i>minute</i> – A one-digit or two-digit integer (0–59) • <i>second</i> – A one-digit or two-digit integer (0–59) • <i>millisecond</i> – A one- digit to three-digit integer (0–999)	
DATEADD(<i>datePart</i> , <i>number</i> , <i>date</i>)	This returns a new date/time value based on adding or subtracting several intervals to the specified date. Note: This function is equivalent to DATEADD in T-SQL. For more information about possible values for arguments, see the online help topic for this function.	• <i>datePart</i> – The type of interval to be added to or subtracted from the date. • <i>number</i> – The number of intervals to be added to or subtracted from the date. For addition (resulting in a value later than date), use a positive value; for subtraction (resulting in a value earlier than date), use a negative value. • <i>date</i> – An expression that returns the Date/ Time value to or from which you want to add or subtract.	Date/Time
DATEDIFF(<i>datePart</i> , <i>startDate</i> , <i>endDate</i>)	This function returns the number of date or time boundaries crossed between two specified dates. For example, you could use this function to find the number of days in a certain interval of dates. The startDate value is subtracted from the endDate. If the startDate is later than the endDate, then the return value is a negative number. If the result is out of range for integer values, the system returns an error.	• <i>datePart</i> – Specifies the type of interval on which to calculate the difference • <i>startDate</i> – The beginning date for the calculation • <i>endDate</i> – The ending date for the calculation	Numeric

Function/Syntax	What it does	Arguments	Return data type
	Note: This function is equivalent to DATEDIFF in T-SQL. For more information about possible values for arguments, see the online help topic for this function.		
DATEPART(<i>datePart, date</i>)	This function returns a number indicating the value of a part of a date value for a specified date. For example, if you wanted the value of the month for a date, you might get a return value of 10 (for October). Note: This function is equivalent to DATEPART in T-SQL. For more information about possible values for arguments, see the online help topic for this function.	• <i>datePart</i> – The part of the date for which you want to return the value • <i>date</i> – The date for which you want to return the part value	Numeric
DBFUNCTION(<i>functionName, arg1, arg2, ...</i>)	Calls a database function Note: This function is available only for database operations.	• <i>functionName</i> – The name of the database function you want to call • <i>arg1, arg2, ...</i> – Any arguments required by the database function	Typeless
E(<i>param</i>)	Returns the value of an event parameter	<i>param</i> – The name of the event parameter	Typeless
EVENTNAME()	Returns the name of the event being processed	N/A	String
EVENTREVISION()	Returns the design-time revision number of the active group of handlers of the event being processed	N/A	Numeric

Function/Syntax	What it does	Arguments	Return data type
EVENTSTATE()	Returns the designer-defined state of the running event	N/A	String
EVENTTITLE()	Returns the title associated with a specified event	N/A	String
FE(<i>param</i>)	This retrieves the value of an event parameter as a string enclosed in quotes. If the parameter is empty, it returns NULL as an unquoted string.	<i>param</i> – Name of the parameter for which the value is being returned	String
FGC(<i>constant</i>)	This retrieves the value of an event global constant as a string enclosed in quotes. If the global constant has no value assigned, returns NULL as an unquoted string.	<i>constant</i> – Name of the global constant for which the value is being returned	String
FILECONTENTS(<i>location</i>)	Returns the contents of a named text file at a specified location	<i>location</i> – Location of the text file using the standard format: C:\myFiles\myFolder\myTextFile.txt Tip: This file is not required to be a *.txt file, necessarily, but it must be a file that is text-based to be useful. Other useful file types include (but are not limited to) *.htm, *.rtf, *.xml, and so on. For more information, see the online help topic for this function.	String
FILTER(<i>expr</i>)	This converts and presents an expression as a string enclosed in quotes. If the expression is empty, it presents it as NULL with no quotes.	<i>expr</i> – The expression to be converted and presented	String

Function/Syntax	What it does	Arguments	Return data type
FILTERMETHODPARAM (<i>numExpr</i>)	This function returns the value of the parameter for an IDO method call, at a specified index number. The value is returned as a string in quotes. If the parameter value is empty, it returns NULL as an unquoted string. This function is used only with the <code>IdoOnInvoke</code> and <code>IdoPostInvoke</code> framework events.	<i>numExpr</i> – An integer or expression that resolves to an integer	String
FILTERPROPERTY([[<i>collection</i> ,] <i>row#</i> ,] <i>propertyName</i>)	This function returns the value of a specified property, optionally in a designated row and collection result set, as a string enclosed in quotes. If the value is empty, returns NULL as an unquoted string. Note: Functionally, this is equivalent to the FP function. For more information, see the online help for this function.	<i>collection</i> – The name of a result set containing a collection you loaded earlier using a Load Collection action. This is optional. <i>row#</i> – The one- based number of the row containing the property value you want to retrieve. Note: This argument is used only with the <code>IdoOnLoadCollection</code> and <code>IdoPostLoadCollection</code> framework events, or with a Load Collection action. <i>propertyName</i> – The name of the property for which you want to retrieve the value Note: When using the <code>IdoOnItem</code> or <code>IdoPostItem</code> framework events to retrieve a property that is part of that event, this is the only accepted parameter.	String
FILTERSTRING()	This returns the filter used during an event. It applies only to the <code>IdoOnLoadCollection</code> and <code>IdoPostLoadCollection</code> framework (Core) events.	N/A	String
FLOOR()	This function returns the greatest integer that is	N/A	Numeric

Function/Syntax	What it does	Arguments	Return data type
	less than or equal to the value of a number or numeric expression. Note: This function is equivalent to FLOOR in T-SQL.		
FP([<i>collection</i> , <i>row#</i> ,] <i>propertyName</i>)	See FILTERPROPERTY.	N/A	String
FSV(<i>var</i>)	This returns the value of a session variable as a string enclosed in quotes. If the variable is empty, it returns NULL as an unquoted string.	<i>var</i> – The name of the session variable for which you want to return the value	String
FV(<i>var</i>)	This returns the value of an event variable as a string enclosed in quotes. If the variable is empty, it returns NULL as an unquoted string.	<i>var</i> – The name of the variable for which you want to return the value	String
GC(<i>constant</i>)	Retrieves the value of an event global constant	<i>constant</i> – The name of the event global constant for which you want to return the value	Typeless
HANDLERIGNORES FAILURE()	Indicates whether the currently executing event handler ignores failures	N/A	Boolean
HANDLERSEQ()	Returns the design-time sequence of the currently executing event handler	N/A	Numeric
HANDLERSUSPEND S()	Indicates whether the currently executing event handler is part of a chain that suspends a framework IDO event	N/A	Boolean
HANDLERSYNCHRO US()	Indicates whether the currently executing event handler is part of a chain that was fired synchronously and has not adjourned	N/A	Boolean

Function/Syntax	What it does	Arguments	Return data type
HANDLERTRANSACTIONAL()	Indicates whether the unadjoined portion of the currently executing event handler is (or was) enclosed in a database transaction	N/A	Boolean
HASBEGUN(<i>action</i>)	Indicates whether a specified action has begun executing	<i>action</i> – Action sequence number	Boolean
HASFINISHED(<i>action</i>)	Indicates whether the specified action has finished executing	<i>action</i> – Action sequence number	Boolean
IDO()	Returns the name of the specified IDO collection for which an operation was requested	N/A	String
IF(<i>condition</i> , <i>true_expression</i> , <i>false_expression</i>)	Depending on the outcome of a condition test, this returns one of two expressions.	• <i>condition</i> – The condition to be tested for • <i>true_expression</i> – The expression to be returned if the condition tests TRUE • <i>false_expression</i> – The expression to be returned if the condition tests FALSE	Can vary
INITIATOR()	This returns an identifier indicating the object or process that initiated an event. This specification normally consists of a composite initiator type (form or other client) and name (form name or other client ID), separated by a period.	N/A	String
INSIDEDATABASE()	Indicates whether the current event action is executing inside the database	N/A	Boolean
INSTR(<i>string1</i> , <i>string2</i>)	This function returns the starting position of the	• <i>string1</i> – String to be evaluated	Integer

Function/Syntax	What it does	Arguments	Return data type
	first character in one string (<i>string2</i>) within another string (<i>string1</i>). If <i>string2</i> is not found within <i>string1</i> , then this function returns zero (0).	<i>string2</i> – The (sub)string you are looking for within <i>string1</i>, for which the starting position is to be returned	
LEN(<i>string</i>)	Returns the length of a specified string, as several characters	<i>string</i> – The literal string or an expression that resolves to a string	Integer
LOADFLAGS()	This returns a comma-separated list of the options applicable to an event: - A Position/Direction indicator: First, Next, Prev, or Last - An optional Distinct indicator: Distinct (If the load is not flagged Distinct, this indicator is not present.) - An optional Read Mode indicator: ReadCommitted, or ReadUncommitted (If the load is flagged Default Read Mode, this indicator is not present.) Indicators appear in the order shown above, if present. This applies only to the IdoOnLoadCollection and IdoPostLoadCollection framework (Core) events	N/A	String
LOWER(<i>string</i>)	Returns a string with uppercase characters converted to lowercase	<i>string</i> – String to be converted to lowercase	String
MESSAGE(<i>msgID</i> [,	This function constructs and returns one or more messages using	<i>msgID</i> – Message identifier containing the object name	String

Function/Syntax	What it does	Arguments	Return data type
<i>msgParam1, msgParam2, ...]</i>)	MsgAppSp. For more information, see the online help for this function.	<i>msgParam#</i> – The parameters required to construct the message	
METHOD ()	This function returns the name of the method being invoked. This function is used only with the IdoOnInvoke and IdoPostInvoke framework events.	N/A	String
METHODPARAM(<i>numExpr</i>)	This function returns the value of a parameter for an IDO method call, at the specified index number. This function is used only with the IdoOnInvoke and IdoPostInvoke framework events.	<i>numExpr</i> – A number or numeric expression that tells the system what index number to go to for the parameter value	Typeless
METHODPARMS()	This function returns a count of the number of parameters required by the method. This function is used only with the IdoOnInvoke and IdoPostInvoke framework events.	N/A	Integer
NEWGUID()	Generates a new globally unique identifier (GUID)	N/A	String
NONRESPONDERLIST(<i>action</i>)	Returns a semicolon-delimited (;) list of recipients who have not responded to a specified Prompt message	<i>action</i> – Action sequence number	String
ORIGINATOR()	This function returns the user ID for the user who initiated the running of the event handler. This function can be useful when you need to identify who initiated the action that triggered the	N/A	String

Function/Syntax	What it does	Arguments	Return data type
	event or when you want the user who initiated the action to receive a copy of the message, for example.		
P([<i>collection</i> , <i>row#</i> ,] <i>propertyName</i>)	See PROPERTY.	N/A	Typeless
POSTQUERY-ACTIONS()	This returns the list of post-query actions to be executed (server side) for every row returned by an event. It applies only to the IdoOnLoadCollection and IdoPostLoadCollection framework (Core) events.	N/A	String
POWER(<i>numExpr</i> , <i>power</i>)	This function returns the value of a specified numeric expression to a designated power. Note: This function is equivalent to POWER in T-SQL.	• <i>numExpr</i> – A number or an expression that resolves to a numeric value • <i>power</i> – the exponent, or power to which the <i>numExpr</i> is to be raised	Numeric
PROPERTY([<i>collection</i> , <i>row#</i> ,] <i>propertyName</i>)	Returns the value of a specified property, optionally in a designated row and collection result set	• <i>collection</i> – The name of a result set containing a collection you loaded earlier using a Load Collection action. This is optional. • <i>row#</i> – The one- based number of the row containing the property value you want to retrieve. Note: This argument is used only with the IdoOnLoadCollection and IdoPostLoadCollection framework events, or with a Load Collection action. • <i>propertyName</i> – The name of the property for which you want to retrieve the value. Note: When using the IdoOnItem or IdoPostItem framework events to retrieve	Typeless

Function/Syntax	What it does	Arguments	Return data type
		a property that is part of that event, this is the only accepted parameter.	
PROPERTY-MODIFIED(<i>propertyName</i>)	This function indicates whether a specified property has been modified or not. This function is used only with the IdoOnItem framework events.	<i>propertyName</i> – The name of the property for which you want to know whether it has been modified	Boolean
PROPERTY-NAMES()	Returns a list of properties to be retrieved in the current operation	N/A	String
RECIPIENTLIST(<i>action</i>)	This returns a semicolon-delimited (;) list of recipients who received a message sent as the result of a specified action. The target action step must be a Notify or Prompt action type.	<i>action</i> – The action sequence number for the action that sent the message (Notify or Prompt)	String
RECIPIENTS(<i>action</i>)	This returns the number of recipients who received the message sent by a specified action. The target action step must be a Notify or Prompt action type.	<i>action</i> – The action sequence number for the action that sent the message (Notify or Prompt)	Integer
RECORDCAP()	This returns the record cap in effect during an event. It applies only to the IdoOnLoadCollection and IdoPostLoadCollection framework (Core) events.	N/A	Integer
REPLACE(<i>string1</i> , <i>string2</i> , <i>string3</i>)	Within a string, replaces all instances of a specified substring with a different substring	<i>string1</i> – String in which the replacements are to be made <i>string2</i> – Substring to be replaced	String

Function/Syntax	What it does	Arguments	Return data type
		<i>string3</i> – Replacement substring Any of these strings can be referenced using an expression.	
RESPONDERLIST(<i>action</i> [, <i>choice</i>])	Returns a semicolon-delimited (;) list of recipients who have responded to a specified Prompt message, optionally with a selected choice	<i>action</i> – The action sequence number for the prompt action that produced the messages whose responses are to be checked. <i>choice</i> – The choice for which you want to see who selected it. This is optional. If omitted, the list includes all responders who selected any choice. The value for this argument must be a valid return value for the Prompt choices as defined on the Event Action Prompt and Event Action Prompt Choices forms.	String
RESPONDERS(<i>action</i> [, <i>choice</i>])	Indicates the number of recipients who have responded to a specified Prompt message, optionally for a selected choice	<i>action</i> – The action sequence number for the Prompt action <i>choice</i> – The choice for which you want to see how many selected it. This is optional. If omitted, the return value is a total of all who responded, regardless of their choices. The value for this argument must be a valid return value for the Prompt choices as defined on the Event Action Prompt and Event Action Prompt Choices forms.	Integer
ROUND(<i>numExpr</i> , <i>length</i>)	Returns a numeric expression rounded to the specified length or precision	<i>numExpr</i> – The numeric expression containing the value you want to have rounded <i>length</i> – The number of places or the precision to	Numeric

Function/Syntax	What it does	Arguments	Return data type
	Note: This function is equivalent to ROUND in T-SQL.	which the number should be rounded. When <i>length</i> is a positive number, <i>numExpr</i> is rounded to the number of decimal positions specified by <i>length</i>. When <i>length</i> is a negative number, <i>numExpr</i> is rounded on the left side of the decimal point, as specified by <i>length</i>.	
ROWS(<i>collection</i>)	Returns a number indicating how many records (rows) are in a specified result set (as the result of a Load Collection action)	<i>collection</i> – The name of the result set for which you want to know how many records have been loaded	Integer
ROWS()	This returns a number indicating how many rows were loaded by the collection that generated an event. It applies only to the IdoPostLoad-Collection event.	N/A	Integer
SUBSTITUTE(<i>string</i> , <i>arg1</i> [, <i>arg2</i> , ...])	This function creates a string combined of literal values and "replacement zones" for which values are substituted at run time. Replacement zones are indicated using curly braces { } enclosing numbers. Numbers must start with zero and be consecutive, although they need not necessarily appear in the string in the correct order. For more information, see the online help topic for this function.	• <i>string</i> – The basic literal string containing the replacement zones • <i>arg#</i> – The sources for the replacement text to go into the replacement zones. These are typically expressions. These must appear in the order they are numbered within the <i>string</i>. That is, the first argument (<i>arg1</i>) is used for {0}; the second argument (<i>arg2</i>) is used for {1}, and so forth.	String

Function/Syntax	What it does	Arguments	Return data type
SUBSTRING(<i>string</i> , <i>start</i> , <i>length</i>)	Returns part of a designated string	<i>string</i> – An expression containing the string to be processed <i>start</i> – A number indicating the starting position from which the substring is to be returned. The first character of a string is position 1. If start indicates a position beyond the length of string, the system returns an empty string. <i>length</i> – (Optional) A number indicating how many characters are to be returned. If length indicates more characters than are remaining in the string from the start location to the end of the string, or if this argument is not used, the system returns the entire remainder of the string from the start location.	String
SV(<i>var</i>)	Retrieves the value of a designated session variable	<i>var</i> – The name of the session variable	Typeless
TRUNC(<i>numExpr</i> , <i>length</i>)	Returns a numeric value, rounded to the specified length or precision and then truncated to the appropriate number of places Note: This function is equivalent to ROUND in T-SQL, with a third argument (function) of 1.	<i>numExpr</i> – The numeric expression containing the value you want to have rounded and then truncated <i>length</i> – The number of places or the precision to which the number should be rounded and then truncated When length is a positive number, numExpr is rounded to the number of decimal positions specified by length. When length is a negative number, numExpr is rounded on the left side of the decimal point, as specified by length.	Numeric

Function/Syntax	What it does	Arguments	Return data type
UPPER(string)	Returns a string with lowercase characters converted to uppercase	<i>string</i> – the string expression to be converted to uppercase	String
USERDESC()	Returns the user description, as specified in the User Description field of the Users form, for the user whose action initiated the event handler	N/A	String
USERNAME()	Returns the user login ID for the system user either logging on or logging off the system	N/A	String
V(<i>var</i>)	Retrieves the value of a specified event variable	<i>var</i> – The name of the variable for which you want to retrieve the value	Typeless
VARIABLENAME()	Returns the name of the session variable for which a requested change triggered the SessionOnVarChanged event	N/A	String
VARIABLEVALUE()	Returns the new value of the session variable for which a requested change triggered the SessionOnVarChanged event	N/A	String
VOTINGDISPARITY(<i>action</i>)	Indicates whether any two selected choices disagree for a specified Prompt action A return of: • FALSE indicates that there is no disagreement. • TRUE indicates that at least two responders' selections disagree.	<i>action</i> – The action sequence (step) number for the Prompt action	Boolean

Function/Syntax	What it does	Arguments	Return data type
VOTINGRESULT(<i>action</i>)	Returns the winning choice resulting from the voting on a specified Prompt action	<i>action</i> – The action sequence (step) number for the Prompt action	String
VOTINGTIE(<i>action</i>)	Indicates whether a specified Prompt action has resulted in a tie vote A return of: • FALSE indicates that it was not a tie vote. • TRUE indicates that it was a tie vote.	<i>action</i> – The action sequence (step) number for the Prompt action	Boolean
WORKINGDIR()	This function returns the working directory for the event handler that is running. For more detailed information, see the online help topic for this function. Note: This function is equivalent to the WORKINGDIR() method in the WinStudio API.	N/A	String

Pre-parser functions

These additional functions can be used to expand an event variable or event global constant textually, that is, with no assumptions about the structure or data type of the contained value. These functions are useful in cases where the expanded value contains other grammar elements that must be further evaluated, for example, to share common expressions, expression elements, or groups of functions.

Function/Syntax	What it does	Arguments	Return data type
TGC (conName)	Retrieves the value of an event global constant for which the value can contain other grammar elements	conName – The name of the global constant	Varies

Function/Syntax	What it does	Arguments	Return data type
TV (varName)	Retrieves the value of an event variable in which that value can contain other grammar elements	varName – The name of the variable containing other elements	Varies

When an event action begins, the system first evaluates all TV() references recursively, until no more references to known variables remain. Next, the system evaluates all TGC() references recursively, until no more references to known global constants remain. Finally, the resulting parameters string is passed to the parser to evaluate all other functions and operators contextually using the grammar found in Appendix B, Expression grammar.

For example, consider this parameters string:

CONDITION(TGC(Over1WeekOld)) TGC(OldHandlerPromptParms)

Assume that the system is using this event global constant metadata:

Function/Syntax	Value
Over1WeekOld	DATEDIFF(day, BEGINDATE(), CURDATETIME()) > 7
OldHandlerPromptParms	TO(USERNAME()) SUBJECT("Old Handler Alert") QUESTION("Do you really want to continue this old Handler?") CHOICES("1,sYes,0,sNo")

The effective parameters passed to the event system parser would be:

CONDITION(DATEDIFF(day, BEGINDATE(), CURDATETIME()) > 7) TO(USERNAME())
SUBJECT("Old Handler Alert") QUESTION("Do you really want to continue this old Handler?")
HOICES("1,sYes,0,sNo")

Complex expansions

More complex expansions are possible. For example:

GC(MyVarTV(VarSuffix))

In this example, first, the value of the current event handler's "VarSuffix" variable is retrieved. Then that value is appended to the name "MyVar" to construct an event global constant name, for which the value is then retrieved.

The value of the "VarSuffix" variable might be set dynamically by an event action on the handler, or it might be included in different event initial states that are linked from referring handlers, in which case the GC() reference itself could be moved to its own global constant and referred to from all actions of the handlers using TGC().

Note that no operator or other syntax is used around the TV() reference, because it is evaluated and substituted in place textually, without regard to data type or context.

Nested expansions

Nested expansions are also possible. For example, consider this parameters string:

TGC(MarkItUp)

Assume that the system is using this event global constant metadata:

Name	Value
StdMarkup	* 0.1 + 5
MarkItUp	CONDITION(V(Price)TGC(StdMarkup) < 100) SETVARVALUES(Price=Price TGC(StdMarkup))

This has the effect of increasing the value of the "Price" variable by 10% plus 5, but only if the resulting value is less than 100.

Expression operators

Expression operators can be either unary or binary, meaning they can operate on either one or two expressions. Most operators are limited as to what kind of expressions they can operate with. The kinds of expressions possible include:

- Scalar expressions (scalarExpr) – These expressions can be either of a known type (numeric, string, or date) or an unknown type (typeless). **Note:** In the table below, if the expression type is given as scalarExpr, it can be any of these four types.
- Numeric expressions (numericExpr) – These expressions evaluate using numeric values. These are the expressions used to perform mathematic operations. If a string or typeless expression is supplied where a numeric expression is expected, it is automatically converted into a numeric value. If that is not possible due to the presence of non-numeric characters, the current handler fails with an error.
- String expressions (stringExpr) – These expressions are text-based sets of characters. They may include numbers, but if so, the numbers are treated as text characters, not numerals. If another type of expression is supplied where a string expression is expected, it is automatically converted into a string representation.
- Date expressions (dateExpr) – These expressions involve dates or parts of dates, including times such as 10:00AM.
- Typeless expressions (typelessExpr) – These are expressions that could be one of at least two different types. The way the system treats these expressions depends on the context in which the expression is used.
- Boolean expressions (BooleanExpr) – These expressions consist of an OR conjunction of one or more AND conjunctions.

To construct the expressions used in "Event Action Parameters," you can use the expression operators in this table in conjunction with the "Expression Functions."

Operator	Purpose	Example
scalarExpr = scalarExpr	Is equal to	V(var) = 1

Operator	Purpose	Example
scalarExpr != scalarExpr scalarExpr <> scalarExpr	Is not equal to	V(var) != 1
scalarExpr > scalarExpr	Is greater than	V(var) > 1
scalarExpr < scalarExpr	Is less than	V(var) < 1
scalarExpr >= scalarExpr	Is greater than or equal to	V(var) >= 1
scalarExpr <= scalarExpr	Is less than or equal to	V(var) <= 1
scalarExpr : list scalarExpr IN list	Is in the list The list is enclosed in parentheses and the elements separated by semi-colons.	V(var) IN (1;2;3;4) "b" : ("a";"b";"c")
scalarExpr !: list	Is not in the list The list is enclosed in parentheses and the elements separated by semi-colons.	V(var) !: (1;2;3;4)
BooleanExpr AND BooleanExpr	Boolean AND	V(var) = 1 AND V(var2) > 5
BooleanExpr OR BooleanExpr	Boolean OR	V(var) = 1 OR V(var2) > 5
NOT BooleanExpr	Boolean negation	NOT(V(var) = 1 OR V(var2) > 5)
numericExpr + numericExpr	Addition	V(var) + 1
stringExpr + stringExpr	String concatenation	V(var) + "Item"
numericExpr – numericExpr	Subtraction	V(var) – 1
–numericExpr	Unary negative	– V(var)
numericExpr * numericExpr	Multiplication	V(var) * 5
numericExpr / numericExpr	Division	V(var) / 2

Operator	Purpose	Example
stringExpr LIKE stringExpr	String similarity using WinStudio filtering syntax: • The underscore (_) character represents a single wildcard character. • The asterisk (*) represents any number of wildcard characters.	V(var) LIKE 'A*'

Appendix C: Expression grammar

This appendix contains the complete grammar available for constructing expressions for event action parameters. Major sections include:

- Restrictions
- Start symbol
- Character sets
- Terminals
- Rules: Variable, constant, and event parameter references
- Rules: Expressions
- Rules: Boolean rules
- Rules: Typeless rules
- Rules: String rules
- Rules: Numeric rules
- Rules: Date rules
- Rules: Restricted arguments
- Keyword paren lists
- Enumerations

Restrictions

Expressions in this grammar are subject to the restrictions shown in this table:

Restriction	Additional information
Superfluous parentheses are usually not allowed	For example, this produces a parsing error: V(this)=1 and 2=V(that)) and 1=(2) Instead, you can write it as follows to work around the grammar limitation: V(this)=1 and 2=V(that) and 1=2
Operation chains containing a mix of typed and typeless arguments must begin with a typed value	For example, this example produces an error because the first argument is not a typed value: '12'=V(a)+'B'+V(c) Instead, and because string concatenation is not commutative, you can write it as: '12'="+V(a)+'B'+V(c) Alternatively, you can declare the type for the first argument as: '12'=CAST(V(a) AS STRING)+'B'+V(c)

Restriction	Additional information
	Again, this example produces an error because the first argument is not a typed value: <pre>12>V(b)+1+5</pre> Instead, you can write the expression in one of these ways: <pre>12>0+V(b)+1+5 12>CAST(V(b) AS NUMBER)+1+5 12>1+V(b)+5</pre> The last solution works because numeric addition is commutative.
Functions cannot be used for variable, parameter, or constant names	For example, because the letter y is an abbreviation for year in the DATEPART() and DATEDIFF() event functions, this expression produces an error: <pre>y = 5</pre> Instead, you can declare y as yVar. This expression does not produce an error. <pre>yVar = 5</pre>

Start symbol

"Start Symbol" = <functionParenList>

Character sets

This table lists and describes the acceptable characters for elements of the code described. You can include any amount of white space between elements.

This...	Designates this...
<i>Number</i>	The set of numerals: 0123456789
<i>Letter</i>	The set of all uppercase and lowercase letters: abcdefghijklmnopqrstuvwxyz ABCDEFGHIJKLMNOPQRSTUVWXYZ
<i>Alphanumeric</i>	The set of all characters listed as part of the Number and Letter sets
<i>Printable</i>	Printable designates the set of all standard characters that can be printed onscreen. This includes the characters from #32 to #127 and #160 (nonbreaking space). The nonbreaking space character is included because it is often used in source code.
<i>Whitespace</i>	Whitespace designates the set of all characters that are normally considered "whitespace" and ignored by the parser. The set consists of: • A space (regular)

This...	Designates this...
	• Horizontal tab • Line feed • Vertical tab • Form feed • Carriage return • Nonbreaking space

$\{ID\ Head\} = \{Letter\} + [\]$

$\{ID\ Tail\} = \{Alphanumeric\} + [\] + [''] + ['] + [.]$

$\{String\ Ch\ 1\} = \{Printable\} - [''] + \{LF\} + \{CR\}$

$\{String\ Ch\ 2\} = \{Printable\} - [''] + \{LF\} + \{CR\}$

Terminals

String constants are constructed using one of these rules. The string can be:

- Enclosed in double-quotes, containing no double-quote characters as illustrated by this example:

StringLiteral = "{String Ch 1}*"

- Enclosed in double-quotes, containing paired double quotes that are each interpreted as a single embedded double-quote character as illustrated by this example:

StringLiteral = "({String Ch 1})"

- Enclosed in single quotes, containing no single-quote characters as illustrated by this example:

StringLiteral = '{String Ch 2}*

- Enclosed in single quotes, containing paired single quotes that are each interpreted as a single single-quote character as illustrated by this example:

StringLiteral = '({String Ch 2})'

Each string constant in an expression or parameter list can be constructed using any of these rules, independently of other string constants.

Integer constants contain no decimal point:

IntegerLiteral = {Digit}+

Real constants contain a decimal point:

RealLiteral = {Digit}+.{Digit}+

Identifiers (variable, constant, and event parameter names) must begin with a letter or underscore, and continue with zero or more alphanumeric characters and/or underscores:

Id = {ID Head}{ID Tail}*

Rules: Variable, constant, and event parameter references

<IdValue> consists of one of these:

V(Id)
GC(Id)
SV(Id)
E(Id)

<FilterIdValue> consists of one of these:

FV (Id)
C (Id)
FSV (Id)
FE (Id)

Rules: Expressions

A scalar expression is either of a known type (Number, Date, or String), or its type is unknown.

<Scalar Exp> consists of one of these:

<Numeric-castable Exp>
<Date Exp>
<String Exp>
<Typeless Exp>

A numeric-castable expression is either Number, String, or an unknown type.

<Numeric-castable Exp> consists of one of these:

<Numeric Exp>

Rules: Boolean rules

A Boolean expression is an OR conjunction of one or more AND conjunctions.

<Boolean Exp> consists of one of these:

<And Exp>
<And Exp> OR <Boolean Exp>

An AND expression is an AND conjunction of one or more negatable predicates.

<AND Exp> consists of one of these:

<Not Exp>
<Not Exp> AND <And Exp>

A NOT expression is a negatable predicate.

<Not Exp> consists of one of these:

NOT <Predicate>

<Predicate>

A predicate consists of one of these:

- A comparison of like-typed expressions
- A call to a Boolean function
- A Boolean sub-expression enclosed in parentheses

<Predicate> consists of one of these:

<String Exp> LIKE <String Exp>

<String Exp> IN <Scalar Tuple>

<String Exp> : <Scalar Tuple>

<String Exp> !: <Scalar Tuple>

<String Exp> = <String Exp>

<String Exp> <> <String Exp>

<String Exp> != <String Exp>

<String Exp> > <String Exp>

<String Exp> >= <String Exp>

<String Exp> < <String Exp>

<String Exp> <= <String Exp>

<String Exp> LIKE <Typeless Exp>

<String Exp> = <Typeless Exp>

<String Exp> <> <Typeless Exp>

<String Exp> != <Typeless Exp>

<String Exp> > <Typeless Exp>

<String Exp> >= <Typeless Exp>

<String Exp> < <Typeless Exp>

<String Exp> <= <Typeless Exp>

<Typeless Exp> LIKE <String Exp>

<Typeless Exp> = <String Exp>

<Typeless Exp> <> <String Exp>

<Typeless Exp> != <String Exp>

<Typeless Exp> > <String Exp>

<Typeless Exp> >= <String Exp>

<Typeless Exp> < <String Exp>

<Typeless Exp> <= <String Exp>

<Date Exp> IN <Scalar Tuple>

<Date Exp> : <Scalar Tuple>
 <Date Exp> !: <Scalar Tuple>
 <Date Exp> = <Date Exp>
 <Date Exp> <> <Date Exp>
 <Date Exp> != <Date Exp>
 <Date Exp> > <Date Exp>
 <Date Exp> >= <Date Exp>
 <Date Exp> < <Date Exp>
 <Date Exp> <= <Date Exp>
 <Date Exp> = <Typeless Exp>
 <Date Exp> <> <Typeless Exp>
 <Date Exp> != <Typeless Exp>
 <Date Exp> > <Typeless Exp>
 <Date Exp> >= <Typeless Exp>
 <Date Exp> < <Typeless Exp>
 <Date Exp> <= <Typeless Exp>
 <Typeless Exp> = <Date Exp>
 <Typeless Exp> <> <Date Exp>
 <Typeless Exp> != <Date Exp>
 <Typeless Exp> > <Date Exp>
 <Typeless Exp> >= <Date Exp>
 <Typeless Exp> < <Date Exp>
 <Typeless Exp> <= <Date Exp>
 <Numeric Exp> IN <Scalar Tuple>
 <Numeric Exp> : <Scalar Tuple>
 <Numeric Exp> !: <Scalar Tuple>
 <Numeric Exp> = <Numeric Exp>
 <Numeric Exp> <> <Numeric Exp>
 <Numeric Exp> != <Numeric Exp>
 <Numeric Exp> > <Numeric Exp>
 <Numeric Exp> >= <Numeric Exp>
 <Numeric Exp> < <Numeric Exp>
 <Numeric Exp> <= <Numeric Exp>
 <Numeric Exp> = <Typeless Exp>
 <Numeric Exp> <> <Typeless Exp>

<Numeric Exp> != <Typeless Exp>
 <Numeric Exp> > <Typeless Exp>
 <Numeric Exp> >= <Typeless Exp>
 <Numeric Exp> < <Typeless Exp>
 <Numeric Exp> <= <Typeless Exp>
 <Typeless Exp> = <Numeric Exp>
 <Typeless Exp> <> <Numeric Exp>
 <Typeless Exp> != <Numeric Exp>
 <Typeless Exp> > <Numeric Exp>
 <Typeless Exp> >= <Numeric Exp>
 <Typeless Exp> < <Numeric Exp>
 <Typeless Exp> <= <Numeric Exp>
 <Typeless Exp> IN <Scalar Tuple>
 <Typeless Exp> : <Scalar Tuple>
 <Typeless Exp> !: <Scalar Tuple>
 <Typeless Exp> = <Typeless Exp>
 <Typeless Exp> <> <Typeless Exp>
 <Typeless Exp> != <Typeless Exp>
 <Typeless Exp> > <Typeless Exp>
 <Typeless Exp> >= <Typeless Exp>
 <Typeless Exp> < <Typeless Exp>
 <Typeless Exp> <= <Typeless Exp>

TRUE

FALSE

ANYHANDLERSFAILED ()

HANDLERSYNCHRONOUS ()

HANDLERSUSPENDS ()

HANDLERTRANSACTIONAL ()

HANDLERIGNORESFailure ()

VOTINGDISPARITY (<EventActionRef>)

VOTINGTIE (<EventActionRef>)

HASBEGUN (<EventActionRef>)

HASFINISHED (<EventActionRef>) INSIDEDATABASE ()

PROPERTYMODIFIED (<String Exp>) PROPERTYMODIFIED (<Typeless Exp>)

Sub-expression:

(*<Boolean Exp>*)

Rules: Typeless rules

A typeless expression is a concatenation or sum of elements whose type (between String, Numeric, and Date) we cannot distinguish without context.

<Typeless Exp> consists of one of these:

<Typeless Exp> + <Typeless Value>

<Typeless Value>

A typeless value can be any of these:

- A variable reference
- A function calls whose type cannot be determined without context
- A typeless sub-expression enclosed in parentheses

<Typeless Value> consists of one of these:

<IdValue>

IF (*<Boolean Exp>*, *<Typeless Exp>*, *<Typeless Exp>*)

DBFUNCTION (*<Parameter List>*)

Framework event parameters:

PROPERTY (Id, *<Numeric Exp>*, *<String Exp>*)

PROPERTY (Id, *<Numeric Exp>*, *<Typeless Exp>*)

PROPERTY (Id, *<Typeless Exp>*, *<String Exp>*)

PROPERTY (Id, *<Typeless Exp>*, *<Typeless Exp>*)

P (Id, *<Numeric Exp>*, *<String Exp>*)

P (Id, *<Numeric Exp>*, *<Typeless Exp>*)

P (Id, *<Typeless Exp>*, *<String Exp>*)

P (Id, *<Typeless Exp>*, *<Typeless Exp>*)

PROPERTY (*<Numeric Exp>*, *<String Exp>*)

PROPERTY (*<Numeric Exp>*, *<Typeless Exp>*)

PROPERTY (*<Typeless Exp>*, *<String Exp>*)

PROPERTY (*<Typeless Exp>*, *<Typeless Exp>*)

P (*<Numeric Exp>*, *<String Exp>*)

P (*<Numeric Exp>*, *<Typeless Exp>*)

P (*<Typeless Exp>*, *<String Exp>*)

P (*<Typeless Exp>*, *<Typeless Exp>*)

PROPERTY (*<String Exp>*)

PROPERTY (<Typeless Exp>)

P (<String Exp>)

P (<Typeless Exp>)

METHODPARAM (<Numeric Exp>)

METHODPARAM (<Typeless Exp>)

Sub-expression:

(<Typeless Exp>)

Parameter list:

A parameter list is a comma-separated list of scalar expressions of any type.

<Parameter List> consists of one of these:

<Scalar Exp>

<Scalar Exp>, <Parameter List>

Scalar tuples:

A scalar tuple is a parenthesized set of one or more scalar expressions.

<Scalar Tuple> consists of this:

(<Scalar Expr Set>)

Scalar expression sets:

A scalar expression set is a semi-colon-separated list of one or more scalar expressions, as in this.

<Scalar Expr Set> consists of one of these:

<Scalar Exp>; <Scalar Expr Set>

<Scalar Exp>

Rules: String rules

A string expression is a concatenation of expressions of type String or of unknown type (see the second restriction under "Restrictions").

<String Exp> consists of one of these:

<String Value>

<String Exp> + <String Value>

<String Exp> + <Typeless Value>

A string value can be any of these:

- A string literal
- A quoted variable reference
- A function call returning a string value
- A string sub-expression enclosed in parentheses

<String Value> consists of one of these:

StringLiteral

<FilterIdValue>

IF (<Boolean Exp>, <String Exp>, <String Exp>)

IF (<Boolean Exp>, <String Exp>, <Typeless Exp>)

IF (<Boolean Exp>, <Typeless Exp>, <String Exp>)

String built-in functions:

CLIENTSUBSTITUTE (<String Exp>, <String Expr List>)

CLIENTSUBSTITUTE (<Typeless Exp>, <String Expr List>)

SUBSTITUTE (<String Exp>, <String Expr List>)

SUBSTITUTE (<Typeless Exp>, <String Expr List>)

SUBSTRING (<String Exp>, <Numeric Exp>)

SUBSTRING (<String Exp>, <Typeless Exp>)

SUBSTRING (<String Exp>, <Numeric Exp>, <Numeric Exp>)

SUBSTRING (<String Exp>, <Numeric Exp>, <Typeless Exp>)

SUBSTRING (<String Exp>, <Typeless Exp>, <Numeric Exp>)

SUBSTRING (<String Exp>, <Typeless Exp>, <Typeless Exp>)

SUBSTRING (<Typeless Exp>, <Numeric Exp>)

SUBSTRING (<Typeless Exp>, <Typeless Exp>)

SUBSTRING (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>)

SUBSTRING (<Typeless Exp>, <Numeric Exp>, <Typeless Exp>)

SUBSTRING (<Typeless Exp>, <Typeless Exp>, <Numeric Exp>)

SUBSTRING (<Typeless Exp>, <Typeless Exp>, <Typeless Exp>)

UPPER (<String Exp>)

UPPER (<Typeless Exp>)

LOWER (<String Exp>)

LOWER (<Typeless Exp>)

REPLACE (<String Exp>, <String Exp>, <String Exp>)

REPLACE (<String Exp>, <String Exp>, <Typeless Exp>)

REPLACE (<String Exp>, <Typeless Exp>, <String Exp>)

REPLACE (<String Exp>, <Typeless Exp>, <Typeless Exp>)

REPLACE (<Typeless Exp>, <String Exp>, <String Exp>)

REPLACE (<Typeless Exp>, <String Exp>, <Typeless Exp>)

REPLACE (<Typeless Exp>, <Typeless Exp>, <String Exp>)

REPLACE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp>)

Event attributes:

EVENTNAME()
ORIGINATOR()
CONFIGNAME()
EVENTSTATE()
EVENTTITLE()
ACTIONTYPENAME ()
VOTINGRESULT (<EventActionRef>)
RECIPIENTLIST (<EventActionRef>)
RESPONDERLIST (<EventActionRef>)
RESPONDERLIST (<EventActionRef>, <String Exp>)
RESPONDERLIST (<EventActionRef>, <Typeless Exp>)

Environment attributes:

APPNAME()
COMPANYNAME()
USERNAME()
USERDESC()
WORKINGDIR()
FILECONTENTS (<String Exp>)
FILECONTENTS (<Typeless Exp>)
NEWGUID()

Framework event parameters:

IDO()
INITIATOR()
FILTERSTRING()
LOADFLAGS()
PROPERTYNAMES()
POSTQUERYACTIONS()
CUSTOMINSERT()
CUSTOMUPDATE()
CUSTOMDELETE()
VARIABLENAME() VARIABLEVALUE()
FILTERPROPERTY (Id, <Numeric Exp>, <String Exp>)
FILTERPROPERTY (Id, <Numeric Exp>, <Typeless Exp>)

FILTERPROPERTY (Id, <Typeless Exp>, <String Exp>)
 FILTERPROPERTY (Id, <Typeless Exp>, <Typeless Exp>)
 FP (Id, <Numeric Exp>, <String Exp>)
 FP (Id, <Numeric Exp>, <Typeless Exp>)
 FP (Id, <Typeless Exp>, <String Exp>)
 FP (Id, <Typeless Exp>, <Typeless Exp>)
 FILTERPROPERTY (<Numeric Exp>, <String Exp>)
 FILTERPROPERTY (<Numeric Exp>, <Typeless Exp>)
 FILTERPROPERTY (<Typeless Exp>, <String Exp>)
 FILTERPROPERTY (<Typeless Exp>, <Typeless Exp>)
 FP (<Numeric Exp>, <String Exp>)
 FP (<Numeric Exp>, <Typeless Exp>)
 FP (<Typeless Exp>, <String Exp>)
 FP (<Typeless Exp>, <Typeless Exp>)
 FILTERPROPERTY (<String Exp>)
 FILTERPROPERTY (<Typeless Exp>)
 FP (<String Exp>)
 FP (<Typeless Exp>)
 METHOD ()
 FILTERMETHODPARG (<Numeric Exp>)
 FILTERMETHODPARG (<Typeless Exp>)
 FILTER (<String Exp>)
 FILTER (<Typeless Exp>)
 CAST (<Scalar Exp> AS STRING)

Sub-expression:

(<String Exp>)
 <Message Conversation>

String expression lists

A string expression list is an ordered, comma-separated list of string and/or scalar expressions.

<String Expr List> consists of one of these:

<Scalar Exp>
 <String Expr List>, <Scalar Exp>

Message conversations

A message conversation is one or more MESSAGE() calls separated by | newline/concateration operators.

<Message Conversation> consists of one of these:

<Message>

<Message Conversation> | <Message>

Messages

A message is the MESSAGE function surrounding a parenthesized, ordered, comma-separated list of string and/or typeless expressions.

<Message> consists of one of these:

MESSAGE (<String Exp>)

MESSAGE (<Typeless Exp>)

MESSAGE (<String Exp>, <String Expr List>)

MESSAGE (<Typeless Exp>, <String Expr List>)

Rules: Numeric rules

<Numeric Exp> consists of this:

<Sum>

Sums

A sum is a chain of sums and/or differences:

<Sum> consists of one of these:

<Addend>

<Sum> + <Addend>

<Sum> – <Addend>

Addends

An addend is a negatable expression, or a product. These can be added to or subtracted from one another without parentheses surrounding each, due to the accepted arithmetic operator precedence.

<Addend> consists of one of these:

<Negate Exp>

<Addend> * <Negate Exp>

<Addend> / <Negate Exp>

Negatable expressions

A negatable expression is a numeric value with or without a unary negative.

<Negate Exp> consists of one of these:

- <Numeric Value>
- <Numeric Value>
- <IdValue>

Numeric values

A numeric value can be any of these:

- A numeric constant
- A function call returning a numeric value
- A numeric sub-expression enclosed in parentheses

<Numeric Value> consists of one of these:

IntegerLiteral
 RealLiteral
 IF (<Boolean Exp>, <Numeric Exp>, <Numeric Exp>)
 IF (<Boolean Exp>, <Numeric Exp>, <Typeless Exp>)
 IF (<Boolean Exp>, <Typeless Exp>, <Numeric Exp>)

Numeric built-in functions:

DATEDIFF (<TimeInterval>, <Date Exp>, <Date Exp>)
 DATEDIFF (<TimeInterval>, <Date Exp>, <Typeless Exp>)
 DATEDIFF (<TimeInterval>, <Typeless Exp>, <Date Exp>)
 DATEDIFF (<TimeInterval>, <Typeless Exp>, <Typeless Exp>)
 DATEPART (<TimeInterval>, <Date Exp>)
 DATEPART (<TimeInterval>, <Typeless Exp>)
 LEN (<String Exp>)
 LEN (<Typeless Exp>)
 INSTR (<String Exp>, <String Exp>)
 INSTR (<String Exp>, <Typeless Exp>)
 INSTR (<Typeless Exp>, <String Exp>)
 INSTR (<Typeless Exp>, <Typeless Exp>)
 CEILING (<Numeric Exp>)
 CEILING (<Typeless Exp>)
 FLOOR (<Numeric Exp>)
 FLOOR (<Typeless Exp>)
 POWER (<Numeric Exp>, <Numeric Exp>)
 POWER (<Numeric Exp>, <Typeless Exp>)
 POWER (<Typeless Exp>, <Numeric Exp>)
 POWER (<Typeless Exp>, <Typeless Exp>)

ROUND (<Numeric Exp>, <Numeric Exp>)
 ROUND (<Numeric Exp>, <Typeless Exp>)
 ROUND (<Typeless Exp>, <Numeric Exp>)
 ROUND (<Typeless Exp>, <Typeless Exp>)
 TRUNC (<Numeric Exp>, <Numeric Exp>)
 TRUNC (<Numeric Exp>, <Typeless Exp>)
 TRUNC (<Typeless Exp>, <Numeric Exp>)
 TRUNC (<Typeless Exp>, <Typeless Exp>)

Event state attributes:

EVENTREVISION ()
 HANDLERSEQ ()
 ACTIONSEQ ()

Event attributes:

RECIPIENTS (<EventActionRef>)
 RESPONDERS (<EventActionRef>)
 RESPONDERS (<EventActionRef>, <String Exp>)
 RESPONDERS (<EventActionRef>, <Typeless Exp>)

Framework event parameters:

RECORDCAP()
 ROWS (Id) ROWS ()
 METHODPARMS ()
 CAST (<Scalar Exp> AS NUMBER)

Sub-expression:

(<Numeric Exp>)

Rules: Date rules

A date expression is a date value (because there are no natural arithmetic date operators).

<Date Exp> consists of this:

<Date Value>

A date value is a function call returning a date value, or a date sub-expression enclosed in parentheses.

<Date Value> consists of one of these:

DATE (<Numeric Exp>, <Numeric Exp>, <Numeric Exp>)
 DATE (<Numeric Exp>, <Numeric Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Numeric Exp>, <Typeless Exp>, <Numeric Exp>,<Numeric Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Numeric Exp>, <Typeless Exp>, <Numeric Exp>,<Numeric Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Numeric Exp>, <Typeless Exp>, <Numeric Exp>,<Numeric Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Numeric Exp>, <Typeless Exp>, <Numeric Exp>,<Typeless Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Numeric Exp>, <Typeless Exp>, <Numeric Exp>,<Typeless Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Numeric Exp>, <Typeless Exp>, <Numeric Exp>,<Typeless Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Numeric Exp>, <Typeless Exp>, <Typeless Exp>,<Numeric Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Numeric Exp>, <Typeless Exp>, <Typeless Exp>,<Numeric Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Numeric Exp>, <Typeless Exp>, <Typeless Exp>,<Numeric Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Numeric Exp>, <Typeless Exp>, <Typeless Exp>,<Numeric Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Numeric Exp>, <Typeless Exp>, <Typeless Exp>,<Typeless Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Numeric Exp>, <Typeless Exp>, <Typeless Exp>,<Typeless Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Numeric Exp>, <Typeless Exp>, <Typeless Exp>,<Typeless Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>, <Numeric Exp>,<Numeric Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>, <Numeric Exp>,<Numeric Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>, <Numeric Exp>,<Numeric Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>, <Numeric Exp>,<Typeless Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>, <Numeric Exp>,<Typeless Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>, <Numeric Exp>,<Typeless Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>, <Numeric Exp>,<Typeless Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>, <Typeless Exp>,<Numeric Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>, <Typeless Exp>,<Numeric Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>, <Typeless Exp>,<Numeric Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>, <Typeless Exp>,<Numeric Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>, <Typeless Exp>,<Typeless Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>, <Typeless Exp>,<Typeless Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>, <Typeless Exp>,<Typeless Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Numeric Exp>, <Typeless Exp>,<Typeless Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Numeric Exp>,<Numeric Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Numeric Exp>,<Numeric Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Numeric Exp>,<Numeric Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Numeric Exp>,<Numeric Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Numeric Exp>,<Typeless Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Numeric Exp>,<Typeless Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Numeric Exp>,<Typeless Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Numeric Exp>,<Typeless Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Typeless Exp>,<Numeric Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Typeless Exp>,<Numeric Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Typeless Exp>,<Numeric Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Typeless Exp>,<Numeric Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Typeless Exp>,<Typeless Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Typeless Exp>,<Typeless Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Typeless Exp>,<Typeless Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Numeric Exp>, <Typeless Exp>, <Typeless Exp>, <Typeless Exp>,<Typeless Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Numeric Exp>,<Numeric Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Numeric Exp>,<Numeric Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Numeric Exp>,<Numeric Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Numeric Exp>,<Numeric Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Numeric Exp>,<Typeless Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Numeric Exp>,<Typeless Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Numeric Exp>,<Typeless Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Numeric Exp>,<Typeless Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Typeless Exp>,<Numeric Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Typeless Exp>,<Numeric Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Typeless Exp>,<Numeric Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Typeless Exp>,<Numeric Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Typeless Exp>,<Typeless Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Typeless Exp>,<Typeless Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Typeless Exp>,<Typeless Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Numeric Exp>, <Numeric Exp>, <Typeless Exp>,<Typeless Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Numeric Exp>, <Numeric Exp>,<Typeless Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Numeric Exp>, <Numeric Exp>,<Typeless Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Numeric Exp>, <Numeric Exp>,<Typeless Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Numeric Exp>, <Typeless Exp>,<Numeric Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Numeric Exp>, <Typeless Exp>,<Numeric Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Numeric Exp>, <Typeless Exp>,<Numeric Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Numeric Exp>, <Typeless Exp>,<Numeric Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Numeric Exp>, <Typeless Exp>,<Typeless Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Numeric Exp>, <Typeless Exp>,<Typeless Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Numeric Exp>, <Typeless Exp>,<Typeless Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Numeric Exp>,<Numeric Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Numeric Exp>,<Numeric Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Numeric Exp>,<Numeric Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Numeric Exp>,<Numeric Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Numeric Exp>,<Typeless Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Numeric Exp>,<Typeless Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Numeric Exp>,<Typeless Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Typeless Exp>,<Numeric Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Typeless Exp>,<Numeric Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Typeless Exp>,<Numeric Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Typeless Exp>,<Numeric Exp>,
<Typeless Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Typeless Exp>,<Typeless Exp>,
<Numeric Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Typeless Exp>,<Typeless Exp>,
<Numeric Exp>, <Typeless Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Typeless Exp>,<Typeless Exp>,
<Typeless Exp>, <Numeric Exp>)

DATE (<Typeless Exp>, <Typeless Exp>, <Typeless Exp> ',' <Typeless Exp>,<Typeless Exp>,
<Typeless Exp>, <Typeless Exp>)

CURDATETIME()

DATEADD (<TimeInterval>, <Numeric Exp>, <Date Exp>)

DATEADD (<TimeInterval>, <Numeric Exp>, <Typeless Exp>)

DATEADD (<TimeInterval>, <Typeless Exp>, <Date Exp>)

DATEADD (<TimeInterval>, <Typeless Exp>, <Typeless Exp>)

BEGINDATE()

CAST (<Scalar Exp> AS DATE)

Sub-expression:

(<Date Exp>)

Date enumerations

<TimeInterval> consists of one of these:

<DatePartYear>

<DatePartQuarter>

<DatePartMonth>

<DatePartDayOfYear>

<DatePartDay>

<DatePartWeek>

<DatePartWeekDay>

<DatePartHour>

<DatePartMinute>

<DatePartSecond>

<DatePartMillisecond>

<DatePartYear> consists of one of these:

year

yy

yyyy

<DatePartQuarter> consists of one of these:

quarter

qq

q

<DatePartMonth> consists of one of these:

month

mm

m

<DatePartDayOfYear> consists of one of these:

dayofyear

dy

y

<DatePartDay> consists of one of these:

day

dd

d

<DatePartWeek> consists of one of these:

week

wk

ww

<DatePartWeekDay> consists of one of these:

weekday

dw

<DatePartHour> consists of one of these:

hour

hh

<DatePartMinute> consists of one of these:

minute

mi

n

<DatePartSecond> consists of one of these:

second

ss

s

<DatePartMillisecond> consists of one of these:

millisecond

ms

Rules: Restricted arguments

<EventActionRef> consists of this:

IntegerLiteral

Keyword paren lists

A keyword paren list is one or more "KEYWORD(value)" statements separated by whitespace.

<KeywordParenList> consists of one of these:

<KEYWORD(value)>

<KEYWORD(value)> ... <KEYWORD(value)>

For a complete list and description of KEYWORD(value) statements as they relate to action types, see "Event action parameters."

Enumerations

These lists of keywords can be used with ACTION(), VOTINGRULE(), and TASKSTATUS() expressions.

ACTION()

<SaveAction> consists of one of these:

<SaveActionInsert>

<SaveActionUpdate>

<SaveActionDelete>

<SaveActionInsert> consists of this:

INSERT

<SaveActionUpdate> consists of this:

UPDATE

<SaveActionDelete> consists of this:

DELETE

VOTINGRULE()

<VotingRule> consists of one of these:

<VotingRuleMajority>

<VotingRulePlurality>

<VotingRuleConditionalPlurality>

<VotingRuleMinimumCount>

<VotingRuleMinimumPercentage>

<VotingRuleEarliestResponse>

<VotingRulePreferredChoice>

<VotingRuleMinimumCountPreferredChoice>

<VotingRuleMinimumPercentagePreferredChoice>

<VotingRuleMajority> consists of this:

Majority

<VotingRulePlurality> consists of this:

Plurality

<VotingRuleConditionalPlurality> consists of this:

ConditionalPlurality

<VotingRuleMinimumCount> consists of this:

MinimumCount

<VotingRuleMinimumPercentage> consists of this:

MinimumPercentage

<VotingRuleEarliestResponse> consists of this:

EarliestResponse

<VotingRulePreferredChoice> consists of this:

PreferredChoice

<VotingRuleMinimumCountPreferredChoice> consists of this:

MinimumCountPreferredChoice

<VotingRuleMinimumPercentagePreferredChoice> consists of this:

MinimumPercentagePreferredChoice

TASKSTATUS()

<InitialTaskStatus> consists of one of these:

<TaskStatusReady>

<TaskStatusWaiting>

<TaskStatusReady> consists of this:

READY

<TaskStatusWaiting> consists of this:

WAITING

Appendix D: Sample stored procedures

The sample stored procedures in this appendix provide sample code and function usage.

Passing parameters to a synchronous event

```
SET @MyEventParmId = NEWID()
EXEC InsertEventInputParameterSp @MyEventParmId, 'Parameter1', @Variable1
EXEC InsertEventInputParameterSp @MyEventParmId, 'Parameter2', @Variable2
EXEC InsertEventInputParameterSp @MyEventParmId, 'OutParameter3', NULL, 1
EXEC FireEventSp 'EventName', @MyEventParmId
PRINT dbo.EventOutputParameterValue(@MyEventParmId, 'OutParameter3')
```

Calling a synchronous event within a transaction and handling failure

First, determine the current site, after which you must name a configuration, by convention:

```
DECLARE @Site SiteType
SELECT @Site = site FROM parms
```

Then determine the current SessionId:

```
DECLARE @SessionId RowPointerType
SET @SessionId = dbo.SessionIdSp()
```

Finally, add the procedure code:

```
BEGIN TRANSACTION
UPDATE coitem
SET due_date = dbo.CalcDueDate(@Parm1, @Parm2) WHERE coitem.co_num = @CoNum
AND coitem.co_line = @CoLine
AND coitem.co_release = @CoRelease
```

```
SET @MyEventParmId = NEWID()
EXEC InsertEventInputParameterSp @MyEventParmId, 'CoNum', @CoNum
EXEC InsertEventInputParameterSp @MyEventParmId, 'CoLine', @CoLine
EXEC InsertEventInputParameterSp @MyEventParmId, 'CoRelease', @CoRelease
```

```
DECLARE
@anyHandlersFailed [tinyint],
@result [nvarchar](4000),
@Infobar [nvarchar](4000)
```

```

EXEC @Severity = FireEventSp
@eventName = SetCoitemDueDate',
@configName = 'SyteLine',
@sessionID = @SessionID,
@eventTrxId = null,
@eventParmId = @MyEventParmID OUTPUT,
@transactional = 0,
@anyHandlersFailed = @anyHandlersFailed output,
@result = @result output,
@Infobar = @infobar output

IF @Severity > 0
BEGIN
EXEC RaiseError @Infobar, @Severity
ROLLBACK TRANSACTION
END
...
COMMIT TRANSACTION

```

Appendix E: Synchronization of metadata

This appendix discusses the various concepts and strategies for synchronizing metadata belonging to different owners, and provides several detailed examples.

Overview and rationale

Application event system metadata can come from three primary sources:

- The Infor system framework
- Applications produced by Infor and/or Infor's business partners or other authorized vendor developers
- End-customer development

The ownership for the metadata from each of these sources is controlled by the Access As identifier.

Any of these sources can upgrade and reissue their metadata at times independent of the others. The upgrade process needs to:

- Update any changed handlers that they own.
- Insert any new handlers they might have created since the last version.
- Maintain other owners' handlers and the relationships between them.

Therefore, a synchronization mechanism is needed, to make sure that changes by one metadata owner do not adversely affect the functioning of another owner's metadata.

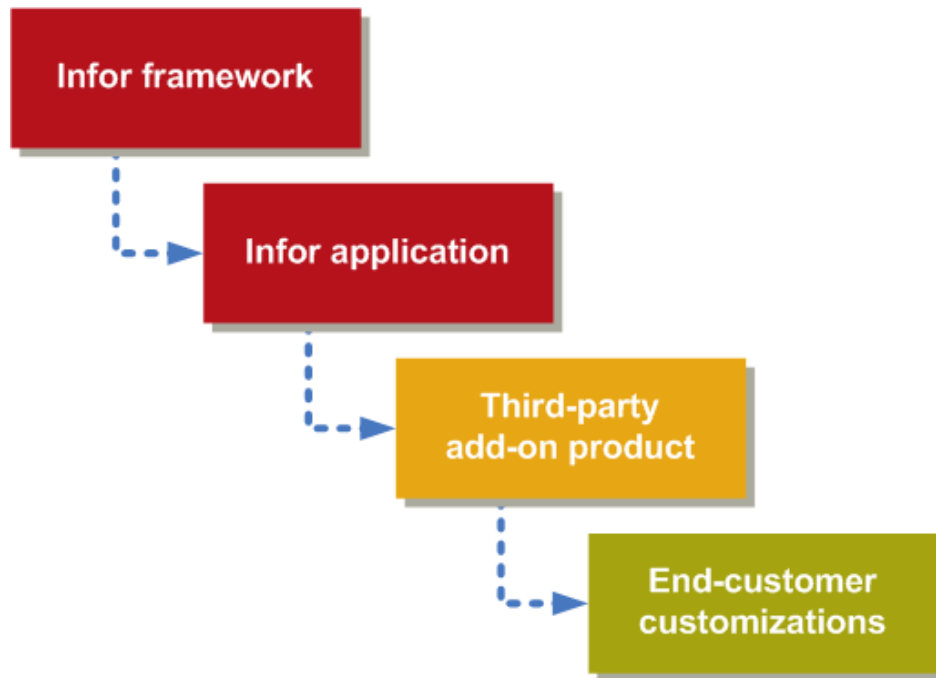
This mechanism is provided in two components:

- The Access As identifier
- The App Metadata Sync and App Metadata Transport utilities, which both provide the capability to synchronize event metadata belonging to different owners

Using these utilities, metadata developers can export their events and event handlers and make them available for import by other metadata owners. They also use this utility to import their own or others' metadata into their system.

The inherent hierarchy of metadata

In the Application Event System, there is no programmatic ranking of Access As identifiers. There is, however, an inherent hierarchy, based on the normal production flow and use of the system software. As illustrated in this diagram, Infor's framework developers are the first to develop event system objects. Other Infor application developers then can add their own event system objects, as can authorized business partners and other vendor developers. Finally, end customers can make custom modifications and develop their own custom event system objects.



Hierarchy of metadata

The system is designed so that no metadata owner can modify or delete the metadata of another owner. So, in that sense, they are all equal. However, this real-time production flow creates an inherent hierarchy that allows us to think of them as "higher-level" (that is, Infor) and "lower-level" (that is, end-customer) owners.

With that in mind, we can state these general rules:

- Lower-level owners can insert their handlers between two higher-level handlers for the same event.
- In many cases, lower-level owners can override higher-level handlers. There is, however, an option for higher-level owners to disallow overrides for individual handlers.
- Higher-level handlers that are overridden remain in the metadata store but are marked as Inactive.

This means, among other things that, if the lower-level handler is later deleted, the higher-level handler is still available and can become active again.

Chronology rules allow downstream owners to integrate and control the sequence of their events and handlers with respect to those upstream.

Maintaining handler IDs through metadata updates

Each event handler is identified with a unique (and hidden) ID, which is referenced by the **Keep With** field on the **Event Handlers** form. This ID, rather than the actual Handler Sequence number, becomes the "fixed" reference point for that handler. This means that an event handler owner does not need to worry about maintaining the handler sequence numbers across releases: The system takes care of it automatically, by preserving the hidden ID number and reassigning handler sequence numbers as required.

After each insertion, update, or deletion of a handler, and during a merge performed by the App Metadata Sync utility or the App Metadata Transport utility, the system calculates new integers, if necessary, for

display in the **Handler Sequence** field. The underlying ID, however, remains unchanged. When a handler is deactivated and another added in the same position, the new handler gets a new ID.

Protecting running events from metadata changes

Once an event handler begins executing, it is essential to prevent changes to its attributes and actions. Otherwise, unpredictable behavior could result, especially if actions are resequenced.

One way to do prevent these changes would be to make a copy of all active, non-obsolete handlers and their actions each time an event is triggered and control execution from this copy. However, that method would result in the persistence of a great number of identical copies, assuming that handlers are modified much less frequently than they are executed.

Since the system stores state data separately from metadata, it is sufficient to make a copy only when the metadata changes, and furthermore only when the corresponding event is triggered and a copy of the last metadata modifications has not yet been made.

In other words, handler metadata for an event can be created and edited as many times as necessary. The first time the event is triggered, a copy of the last saved metadata is made. This copy is called an event revision. The execution of the event's handlers is then controlled by the event revision and not by the original metadata (which happen to be identical at this point).

The event can be triggered as many times as necessary, all the time controlled by this event revision, if no intervening modifications have been made to the original metadata.

After one or more metadata edits have been saved, though, the next time the event is triggered, the system copies a new event revision from the last-saved metadata.

How sequencing and synchronization work

This section provides detailed examples of how sequencing and synchronization work in the Application Event System.

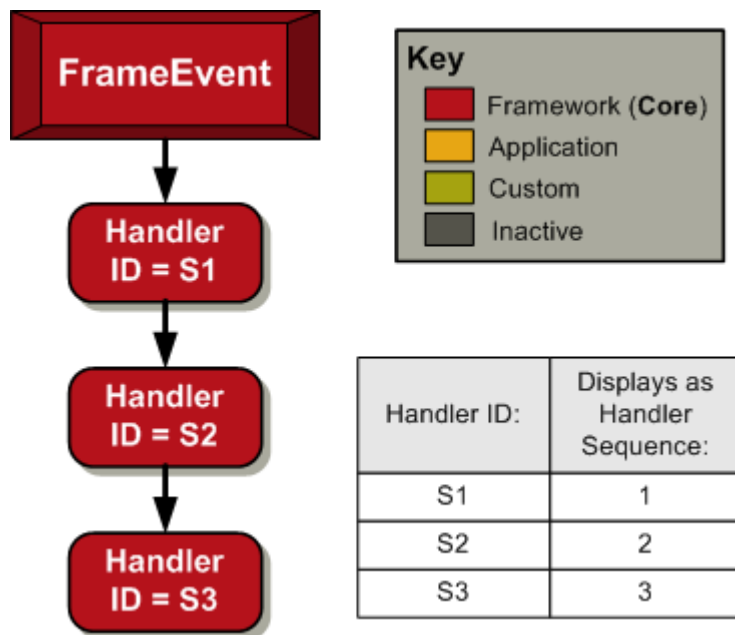
Example 1 ► Using specific chronology

The primary way for a lower-level handler creator to resequence existing handlers (from higher-level owners) is to use what is known as specific chronology. That is, the handler's creator can attach the new handler to an existing handler and specify the order in which the two handlers are to execute with respect to each other.

The mechanism used to do this are the **Keep With** and **Chronology** fields on the **Event Handlers** form. These fields allow you to specify whether your handler should run before, after, or in place of the handler it is associated with, as in the following example.

Part 1: Infor creates a framework event with three handlers

The Infor framework team creates an event, FrameEvent, and creates three event handlers that execute in order when the event is generated.



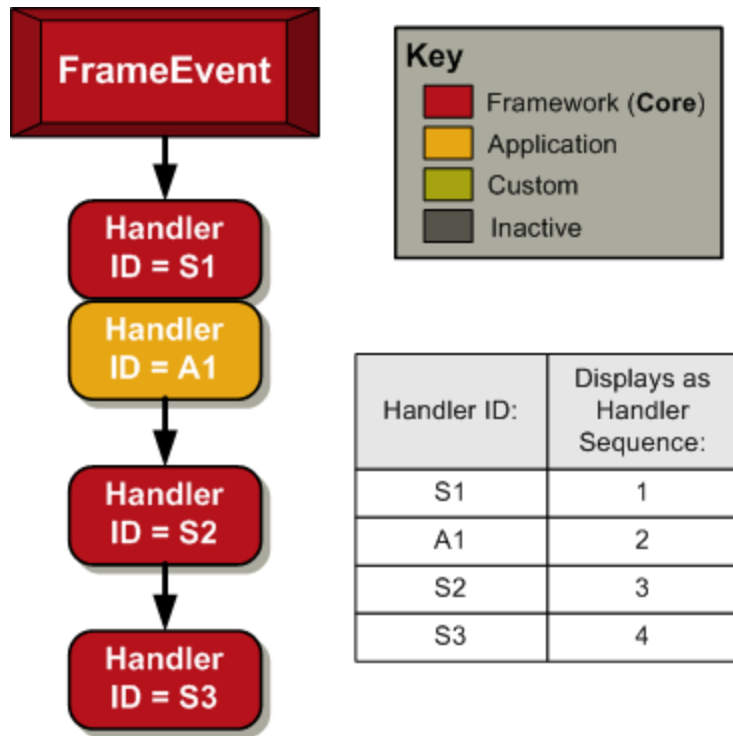
This event and these handlers are all included with the application software when it ships. Note that, if a "lower-level" developer wants to add or modify handlers, these rules apply:

- Lower-level handler creators cannot change the sequence of higher-level handlers.
- Any handlers lower-level creators want to use that will affect the sequence in specific ways must be associated with an already-existing handler, using the **Keep With** and **Chronology** fields.

Part 2: A business partner creates an additional handler

An Infor business partner decides to add a handler that will execute just after the first framework handler. The business partner creates the event handler and uses the **Keep With** and **Chronology** fields to keep their handler with ID 1 (S1) and to execute after that handler.

The sequence is now as follows:



Note that, when the new event handler is saved, the system automatically assigns new handler sequence numbers, so that you can tell which handler executes in what order.

The business partner uses the App Metadata Sync or App Metadata Transport utility to export the new metadata, along with the existing metadata, to a file that can be imported by the end customer.

The business partner then sells the add-on product to the end customer. The add-on product includes this file, along with any product code the business partner has developed.

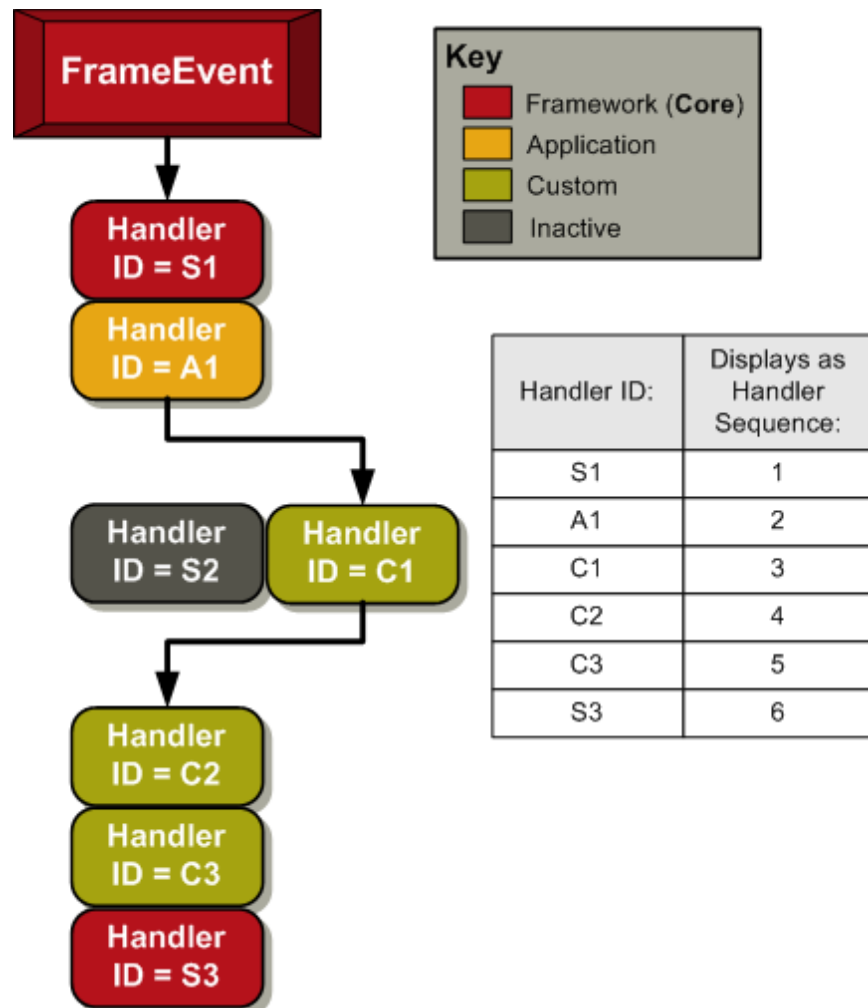
Part 3: The end customer creates three more handlers

When the end customer receives the add-on software from the third-party business partner, in addition to whatever other software installation procedures the customer must perform, the customer must also use the App Metadata Sync or App Metadata Transport utility to import the event metadata from the business partner.

The end customer now decides to use a different handler in place of the second framework handler. The customer also wants two custom handlers to run just before the third framework handler. To accomplish this, the customer:

- Creates a handler and uses the **Keep With** field to assign this handler to ID 2 (S2). In the **Chronology** field, the customer selects **Instead**.
- Creates a second handler and uses the **Keep With** field to assign this handler to ID 3 (S3). In the **Chronology** field, the customer selects **Before**.
- Creates a third handler and uses the **Keep With** field to also assign this handler to ID 3 (S3). In the **Chronology** field, again the customer selects **Before**.

After saving their new handlers with the existing handlers, the sequence is now as follows:



Again, note that, when the customer saves the new handlers, the system automatically assigns and displays the correct handler sequence number on the **Event Handlers** form. The underlying handler ID, however, remains unchanged.

Note that event handler S2 is now inactive and does not execute at all. It is still in the system, but the system ignores it in favor of the new customer handler.

Note also that the custom handlers C2 and C3 execute in that order unless you use the **Up/Down** buttons on the **Event Handler Sequence** form to alter the default order.

End of example

Example 2 ► Using non-specific chronology

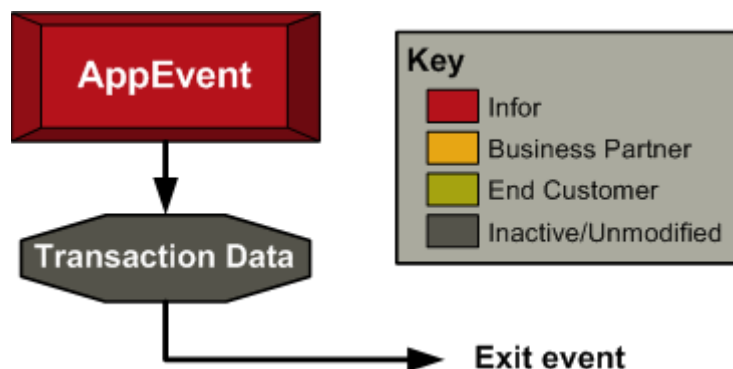
A second way for downstream developers to affect the sequence of handlers is with the use of non-specific chronology. Non-specific chronology allows developers to indicate that a particular handler should always execute either first or last.

This is done by selecting either the **First** or **Last** option from the **Chronology** drop-down field, as in this example.

Part 1: Infor creates an event with no handler

The Infor development team adds code to generate a new application event (AppEvent) before a certain transaction posting is performed.

The transaction data is passed into the event and received from the event, so that any downstream subscribers can test and modify the values before the posting is performed if they want. An event failure is trapped and aborts the posting, displaying the error message returned by the event. Infor adds no event handlers for the event at this time.

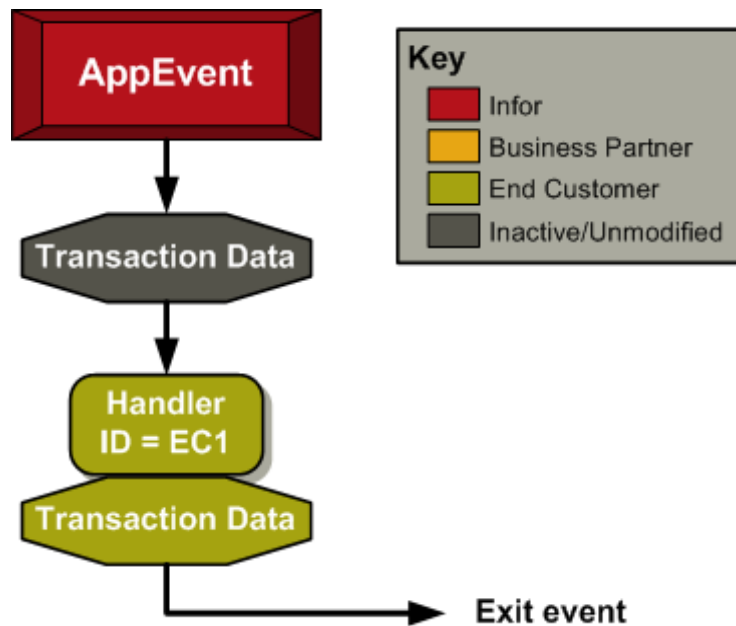


The product is then released. The event is now published and available to downstream developers who install this product.

Part 2: An end-customer creates a handler

An end customer installs the software and decides to create a handler (EC1) to validate that the posting data is within a certain range. If it detects an out-of-range condition, the handler fails the event and aborts the posting.

At this point, the sequence is this:



Because no upstream handlers exist yet (and the end customer is the furthest downstream developer), a specific chronology cannot be specified now.

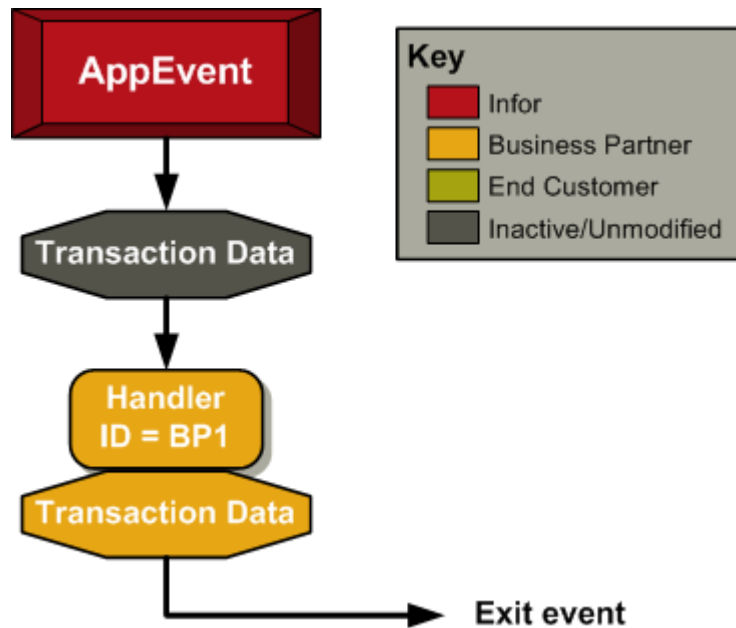
Part 3: A business partner creates an add-on product

In the meantime, an Infor business partner creates an add-on product that stores custom parameters which can be used to adjust the transaction data before posting. They add their own handler (BP1) to input, adjust, and output the data.

In the **Chronology** field, they select **First** to indicate that handler BP1 should run before:

- Any existing, downstream handlers with no specified chronology that might already exist for this published event. **Note:** A specific chronology (Before, After, Instead) takes precedence over a non-specific one (First and Last).
- Any future upstream or peer handlers.

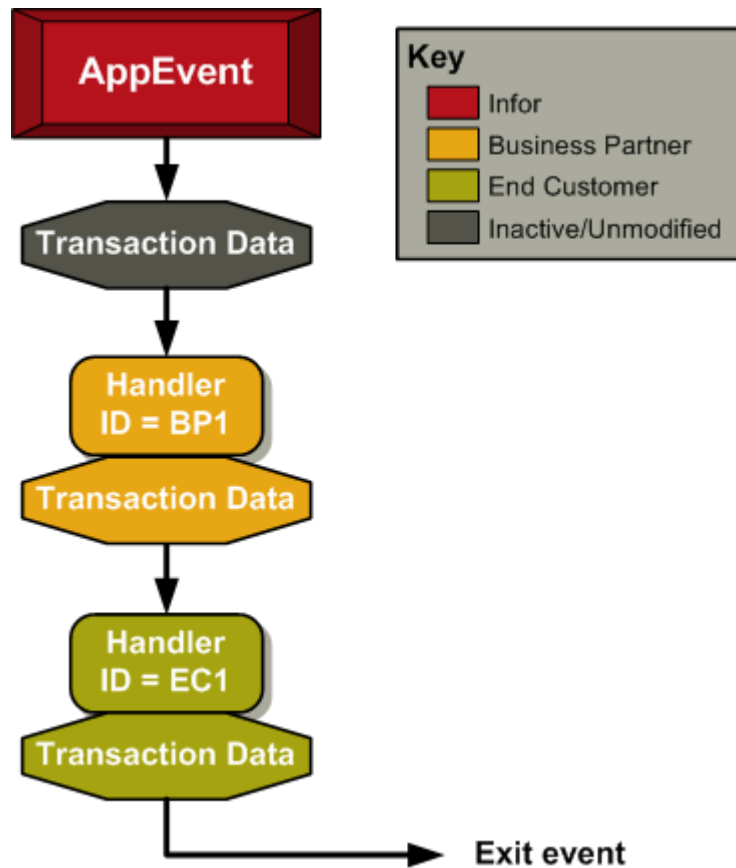
With this add-on product, at the business partner level, the sequence is now this:



Part 4: The end-customer installs the business partner's add-on product

Now the end customer purchases and installs the add-on from the Infor business partner and uses the App Metadata Sync or App Metadata Transport utility to synchronize the metadata from the business partner with their own.

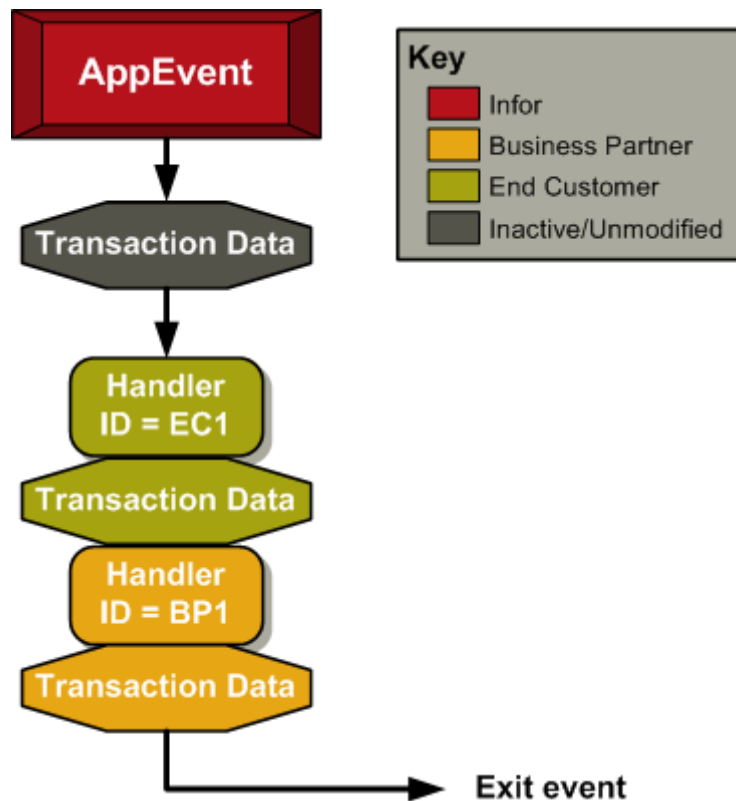
Because the business partner specified that their handler should always run first, the sequence is now this:



Part 5: The end customer rearranges the sequence

After installing the business partner's add-on product, the end customer decides that their handler really should run before the business partner's. So, the end customer uses the **Keep With** and **Chronology** fields to associate handler EC1 with the business partner's handler (BP1) and to execute before it.

After saving the event handler EC1, the sequence is this:

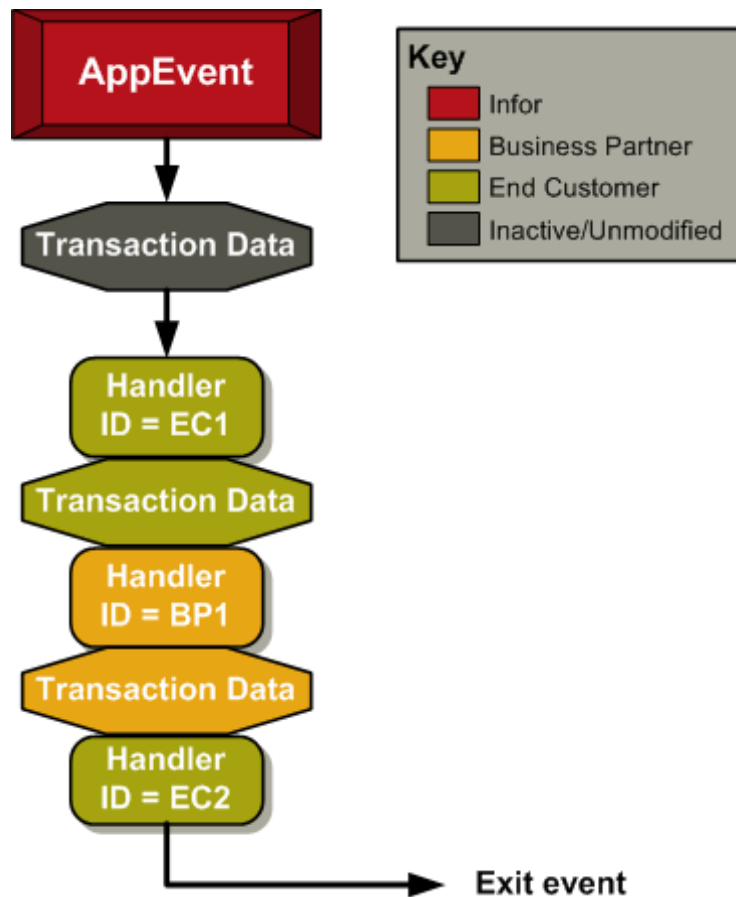


This is possible because a specific chronology designation takes precedence over a non-specific chronology.

Part 6: The end customer decides to add another handler

After this, the end customer adds a new handler (EC2) to save the final, adjusted transaction data into a data warehouse. They use the **Keep With** and **Chronology** fields to associate the new handler with the business partner's handler, specifying that the new one execute after the business partner's (BP1).

After saving the new event handler, the sequence is this:

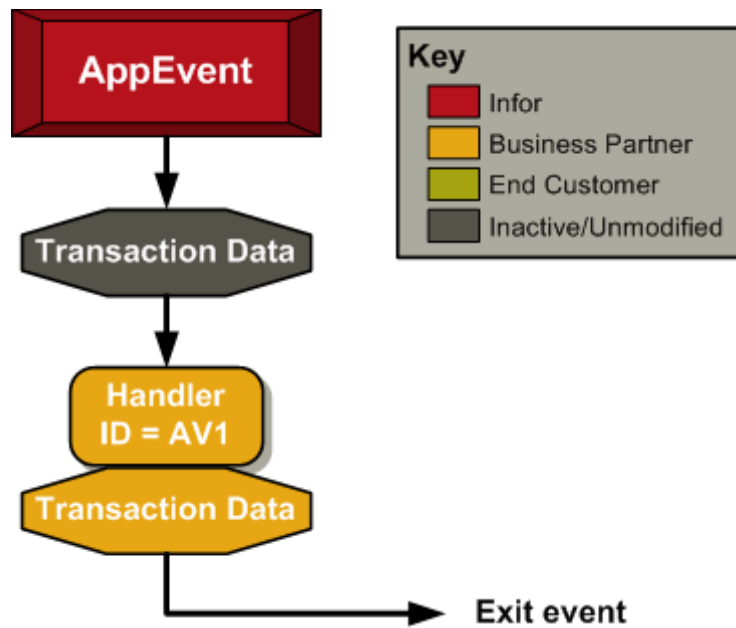


Part 7: Another business partner adds a handler

Independently of all this, a second Infor business partner installs the standard Infor software and decides to add another event handler for their add-on product. This event handler (AV1) is to post additional information to the journal based on the transaction data that was specified.

Because this handler should run after all transaction data has been processed, the business partner specifies in the **Chronology** field that event handler AV1 should execute last.

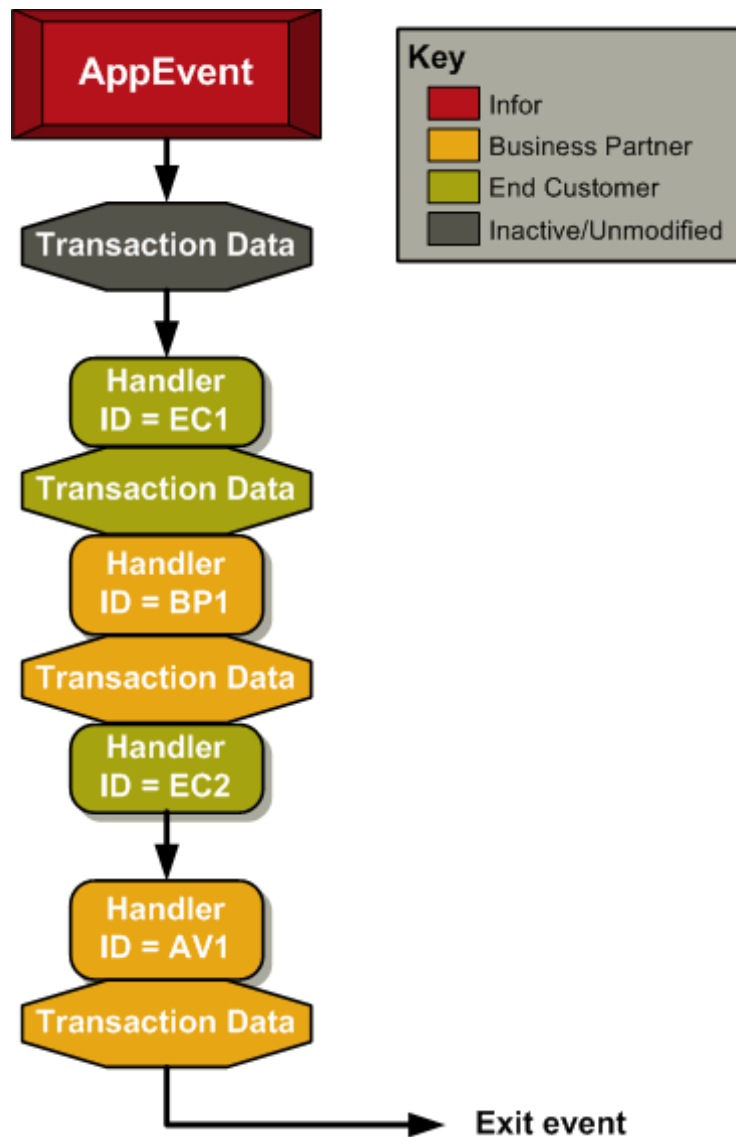
At this point, the flow looks similar to the first business partner's add-on flow:



However, the effect of designating this handler to run last is quite different.

Part 8: The end customer adds the second business partner's handler

Finally, the end customer purchases and installs the second business partner's add-on product. After using the App Metadata Sync or App Metadata Transport utility to synchronize the new metadata with the existing metadata, the flow now proceeds as follows:



Note that, even without the Last designation, in this case, handler AV1 would have executed last simply because the handler BP1 is designated to execute first, and the other two handlers are both "attached" to BP1.

End of example

Performing upgrades

It is likely that at some point in the future, Infor will make changes to the existing events and handlers, and these changes will be included in an upgrade. This poses a few problems and questions. The upgrade process must:

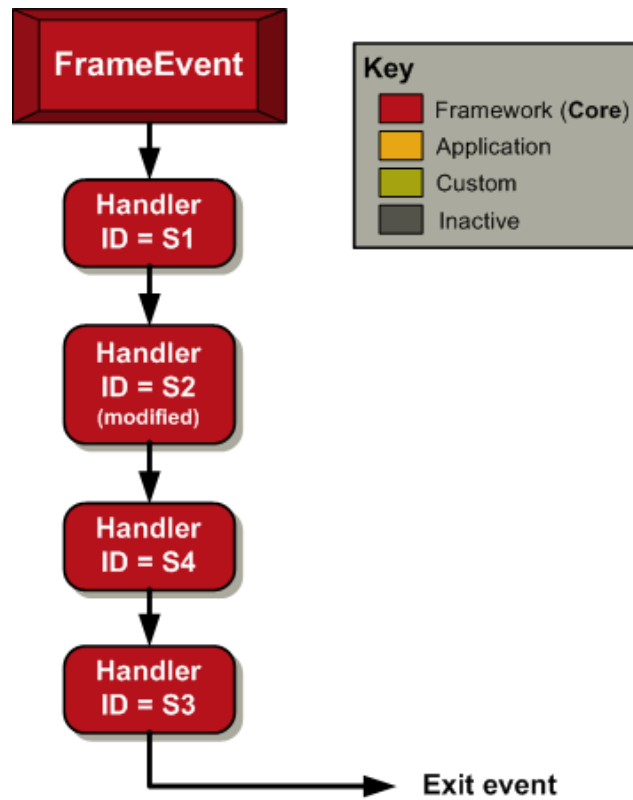
- Update any changed Infor handlers and insert new ones while maintaining custom handlers belonging to business partners and end customers.
- Keep custom handlers in the correct sequence with Infor's handlers (especially for overrides) beyond the point where Infor inserts a new handler.

To illustrate how this works, we return to the previous examples (in Example 1: Using specific chronology and Example 2: Using non-specific chronology).

Example 3 ► Using specific chronology, continued

This example starts at the end of Example 2: Using specific chronology.

After the end customer has installed the Infor software and the business partner add-on product and added its own custom handlers, Infor introduces a new version in which one of the original base handlers (S2) has been modified and a new one, S4, has been added between S2 and S3. The new base flow in the upgrade is illustrated in this diagram:



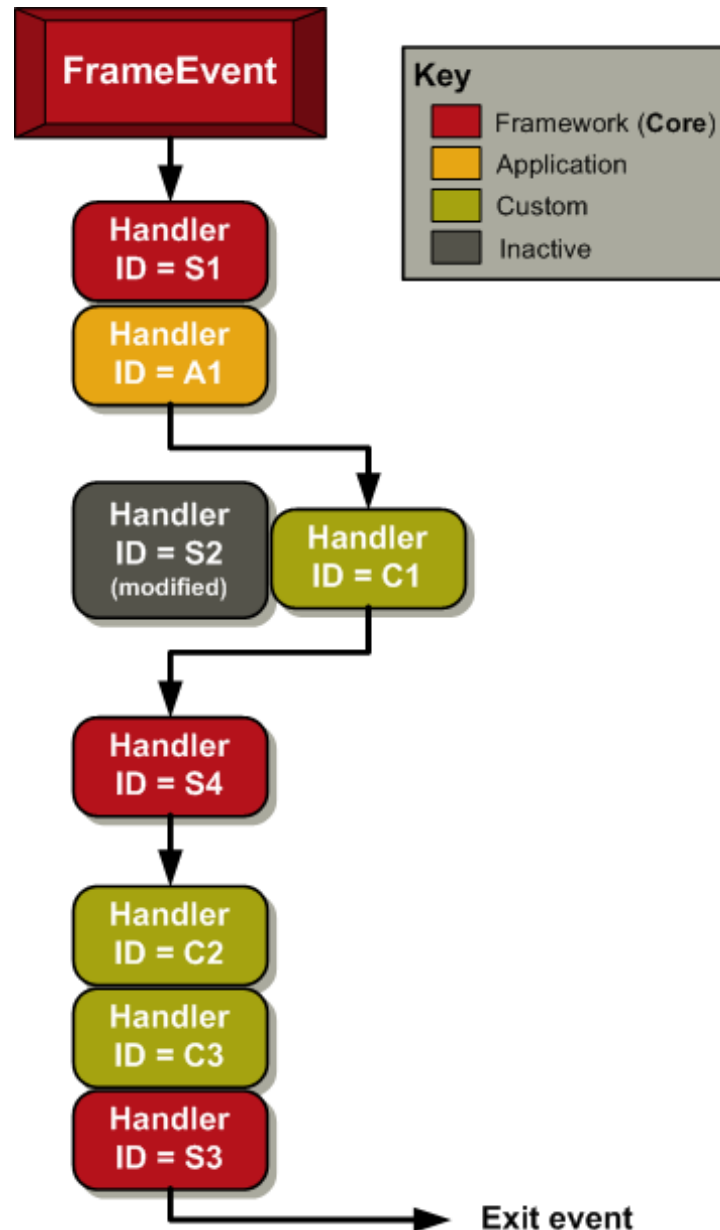
Note that, when this version is released, the numbers actually displayed in the **Handler Sequence** field of the **Event Handlers** form are as follows:

Handler ID	Handler sequence number
S1	1
S2	2
S3	4
S4	3

Note that what was previously the third event handler in the flow (S3), now appears as 4 in the **Handler Sequence** field. This is because, when the Infor developer saves the new handler, the system

automatically assigns the correct handler sequence number, without changing the underlying handler ID number.

The end customer now installs the new Infor upgrade and uses the App Metadata Sync or App Metadata Transport utility to import and synchronize the updated metadata for this event. After the integration is complete, the new end customer sequence is this:



Here is what happened:

- Infor handler S2 has been updated. In this case, the update does not run, because of the end customer handler C1, which replaces it. However, the changes are there (stored as handler metadata) and available if the end customer should ever decide to delete handler C1.

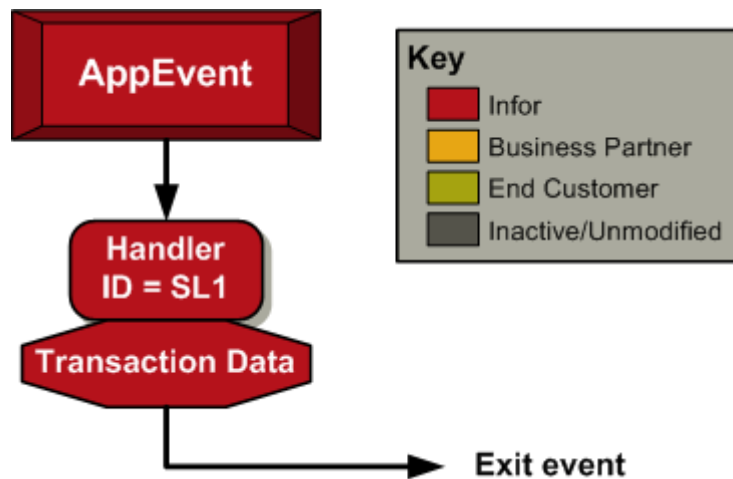
- The synchronization process inserts the new Infor handler between end customer handlers C1 and C2. This is because end customer handler C1 is "attached" to Infor handler S2; end customer handlers C2 and C3 are "attached" to Infor handler S3; and the new Infor handler has been inserted between S2 and S3.

End of example

Example 4 ► Using non-specific chronology, continued

This example starts at the end of Example 1: Using Non-specific chronology.

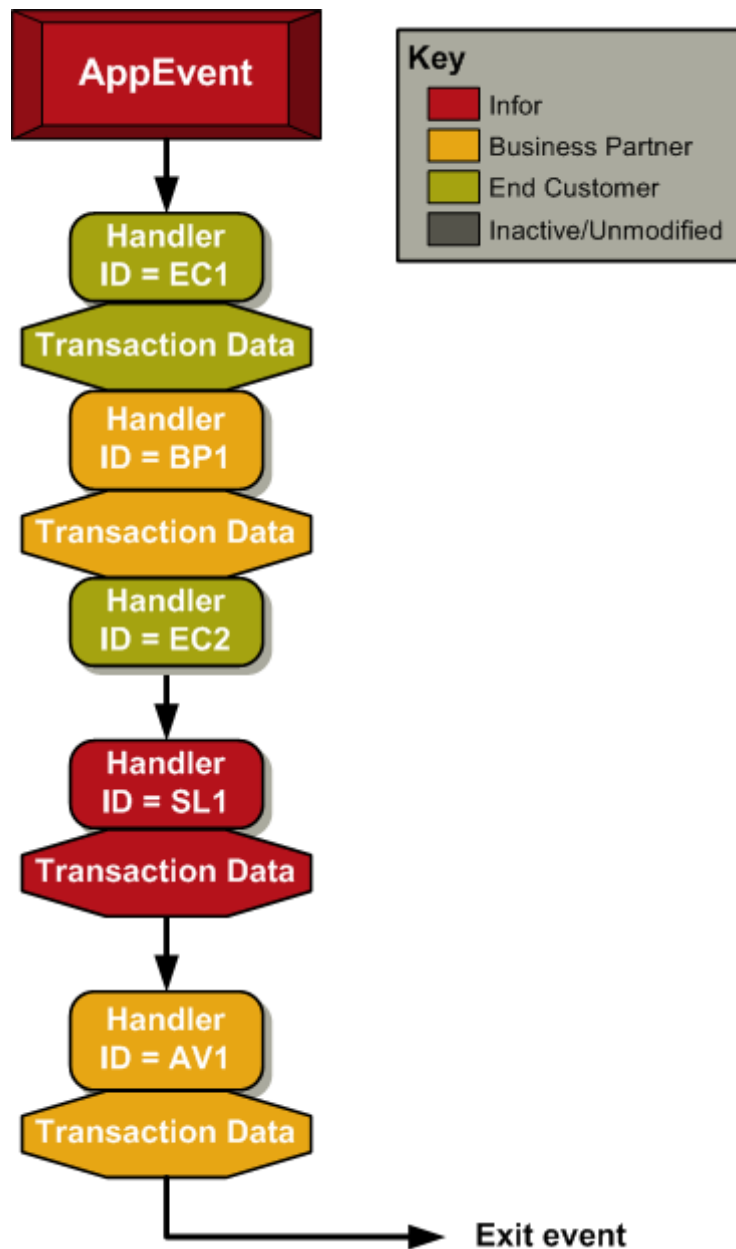
After the end customer has installed the Infor software and both business partner add-on products, Infor introduces a new version in which a new handler to validate data formats has been added and designated to run first. The new base flow in the upgrade is illustrated in this diagram:



Note that this new upgrade will now cause the event to be generated and check data formats whenever the appropriate transaction is posted. This is different, because before the Infor base version of this event did nothing.

Now the end customer installs the Infor upgrade and uses the App Metadata Sync or App Metadata Transport utility to import and synchronize the new handler metadata with the existing metadata for this event. Because the business partner handler BP1 is marked to execute first, that handler and all handlers "attached" to it (EC1 and EC2) are placed at the beginning. Because the business partner handler AV1 is marked to execute last, that handler is placed at the end. Because the Infor handler SL1 is not marked to execute in any particular order, it is inserted between EC2 (which is attached to BP1) and AV1.

After the integration is complete, the new handler sequence is as follows:



End of example

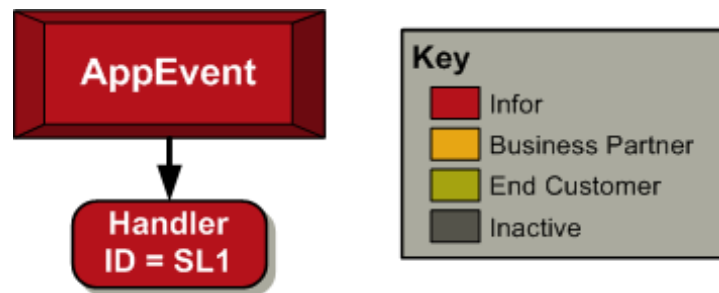
Overriding others' handlers

As demonstrated briefly in Example 1, downstream event handler creators can create handlers that, in effect, replace an upstream handler. This is done primarily with the use of the Instead option in the **Chronology** field on the **Event Handlers** form. This option allows multiple overriding handlers to coexist. In other words, this option is considered a non-exclusive override.

There is also a second option for situations where a handler's creator might want to override all other handlers, regardless of who owns them or when they were added to the sequence. This is done using the Exclusively (instead) option in the second **Keep With** field. This option is considered an exclusive override.

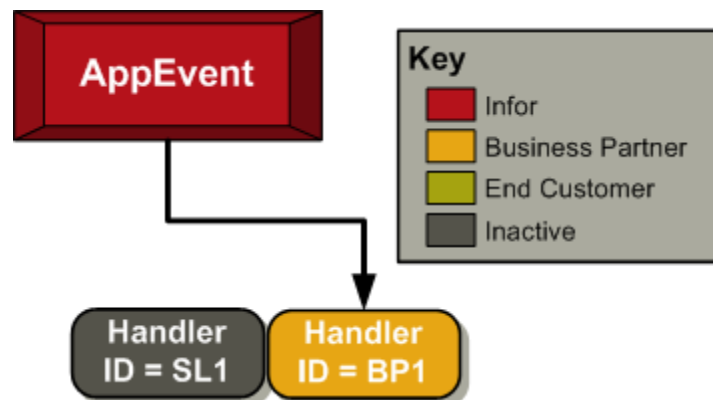
Example 5 ► Non-exclusive overrides

In this example, the Infor application development team creates an application event and adds an event handler, SL1, for it.



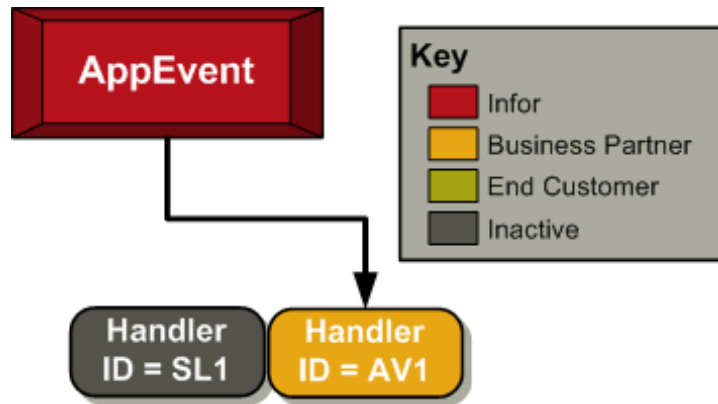
The application is released with this event metadata included as part of the standard product.

An Infor business partner installs the software with this event metadata. They decide, however, that they want to replace the standard Infor event handler with one of their own. So, they create handler BP1 and select **Instead** in the **Chronology** field on the **Event Handlers** form. After saving their handler, the sequence is as follows:

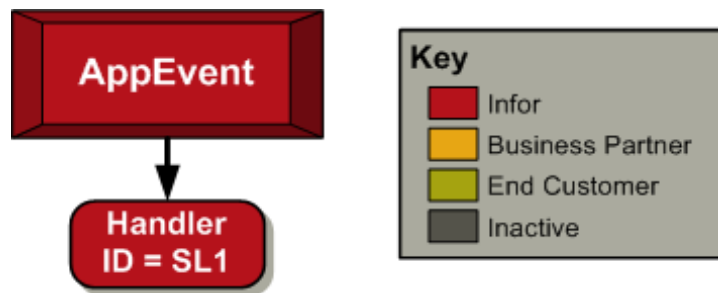


Notice that this effectively bypasses and deactivates the original Infor event handler, SL1. The business partner then releases the add-on product with this new event handler in place

In the meantime, a second Infor business partner does the same thing. For the sake of the example, the second business partner's handler is labeled AV1.



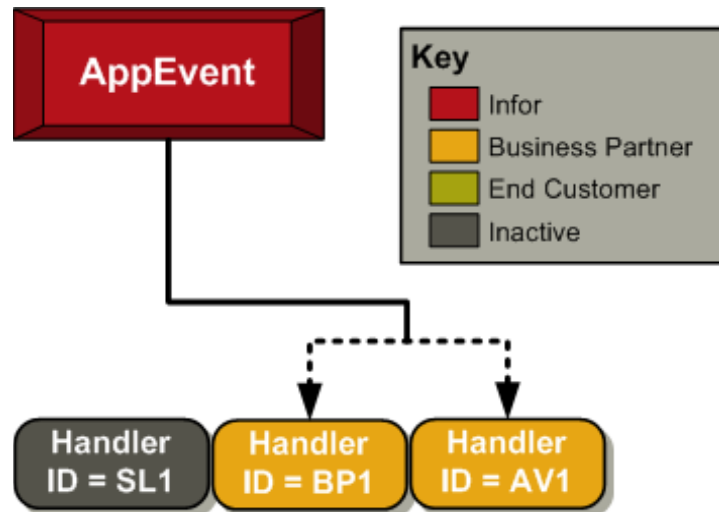
Now suppose that a customer purchases and installs the standard Infor product. At this point, the sequence is the same as what both business partners received originally:



When the event is generated, the Infor event handler executes.

The customer then purchases and installs the add-on product from the first business partner. At this point, the sequence is the same as the first business partner's. The App Metadata Sync or App Metadata Transport utility recognizes the override of BP1 and automatically deactivates the Infor handler SL1.

But then, the customer also purchases the second business partner's add-on product and installs it. The App Metadata Sync or App Metadata Transport utility recognizes the new override of AV1 and allows both overrides to coexist. When the event is generated, both handlers BP1 and AV1 execute. The sequence now looks something like this:



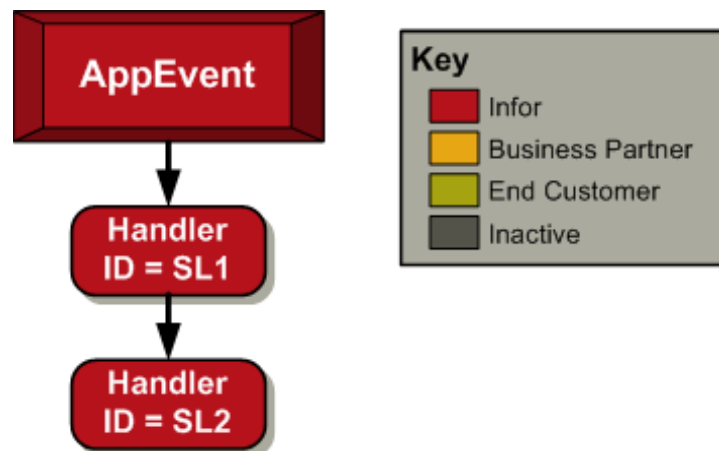
Note that, in this case, there is no reliable way to know the precise order in which event handlers BP1 and AV1 will execute, since they coexist at the same level in the sequence and execute independently of one another.

End of example

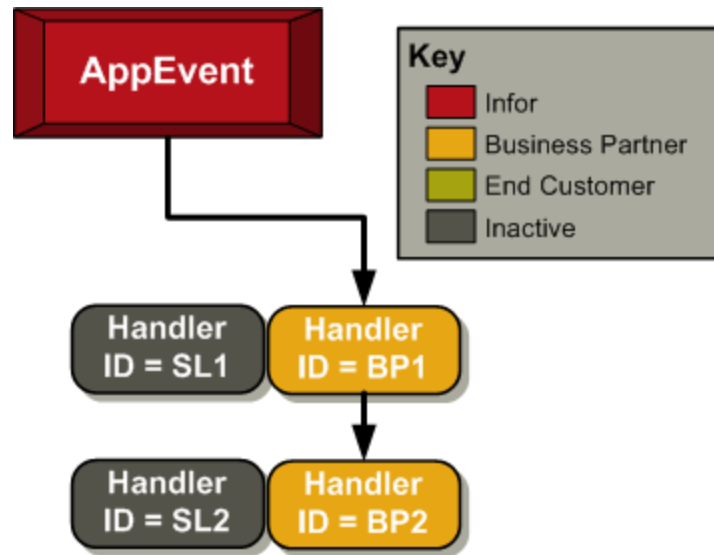
To continue the example, suppose that the Infor application development team now adds another event handler, SL2, for the same event.

Example 6 ► Exclusive overrides

When Infor releases the upgrade, the sequence for this event now looks like this:

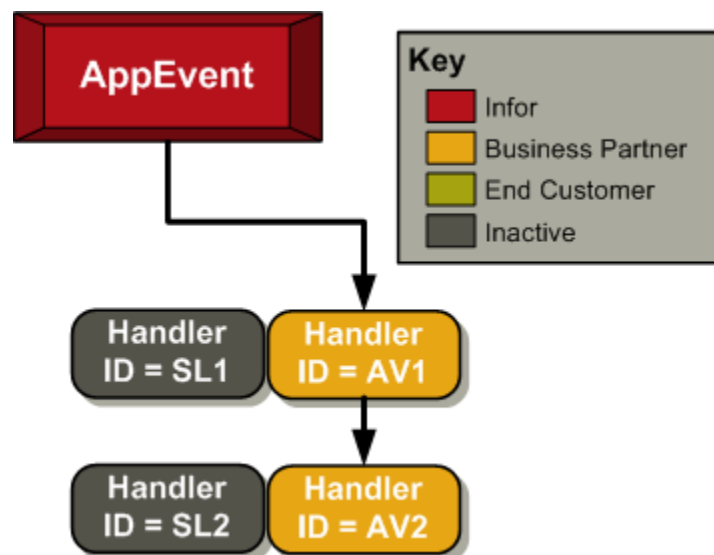


The first business partner installs the upgrade using the App Metadata Sync or App Metadata Transport utility. Once again, they decide they want to override the new Infor handler with one of their own. So, they create a new handler, BP2, and assign it to run Instead of SL2. When they release the new version of their add-on product, the sequence for this event now looks like this:



Note that, so far, this is typical non-exclusive override behavior.

The second business partner does the same thing as the first, with one exception: They specify that the new handler, AV2, is to execute Exclusively Instead, regardless of any other handlers that might exist at that point in the sequence. When they release their new version, the sequence for this event looks very similar to that of the first business partner:



However, the outcome is quite different.

Suppose the customer now installs the Infor software upgrade and tries to install the upgrades for both business partners as well. All goes well until the customer tries to install the new version of the second business partner's add-on product. When the customer tries to synchronize the event metadata from the second business partner, the App Metadata Sync or App Metadata Transport utility generates an error, because the handler AV2 is set to execute exclusively, which puts it in conflict with the first business partner's handler BP2. The two cannot coexist.

To resolve the conflict, the customer would have to uninstall one of the business partner handlers and contact them for guidance.

In practice, this should be a very rare occurrence.

Disabling the ability to override

Owners of "higher-level" handlers can designate that their handlers cannot be overridden. Downstream developers cannot use a Chronology setting of **Instead** or **Exclusive Instead** with such a handler.

To designate an event handler as not subject to being overridden, on the **Event Handlers** form, use the **Overridable** check box. When this check box is cleared, no lower-level handlers can override that handler. In this case, the **Instead** and **Exclusive Instead** options are disabled for any other handlers (downstream) that use the **Keep With** field to associate those handlers with this handler.

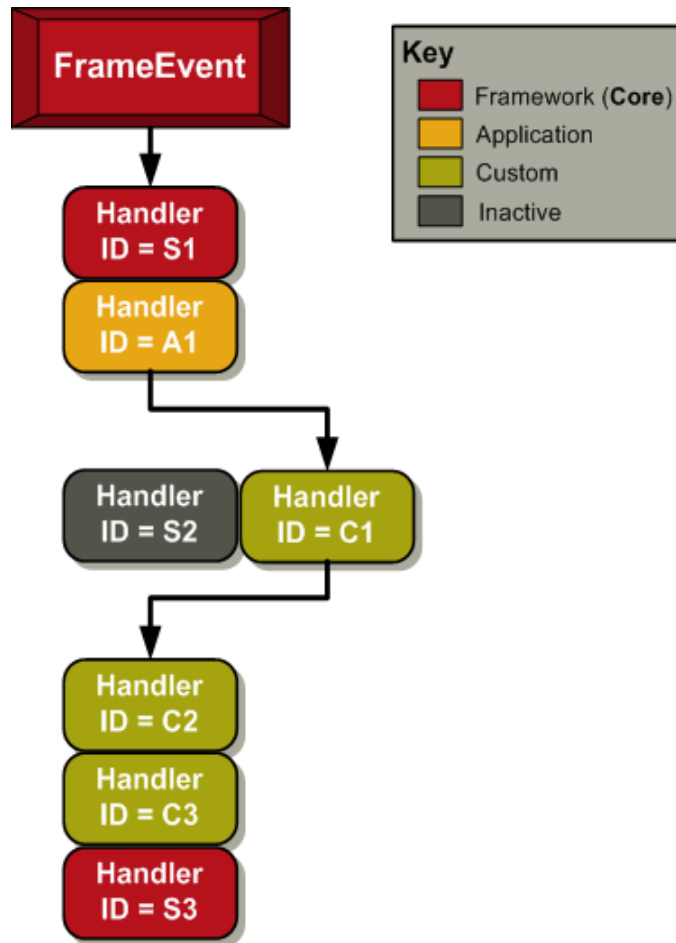
Dealing with obsolete handlers

If you are an Infor or business partner developer, you must not delete handler metadata that has been exported and made available to other (downstream) metadata developers. This is because those developers might have their own metadata associated with your metadata, by means of the **Keep With** feature, before your next release.

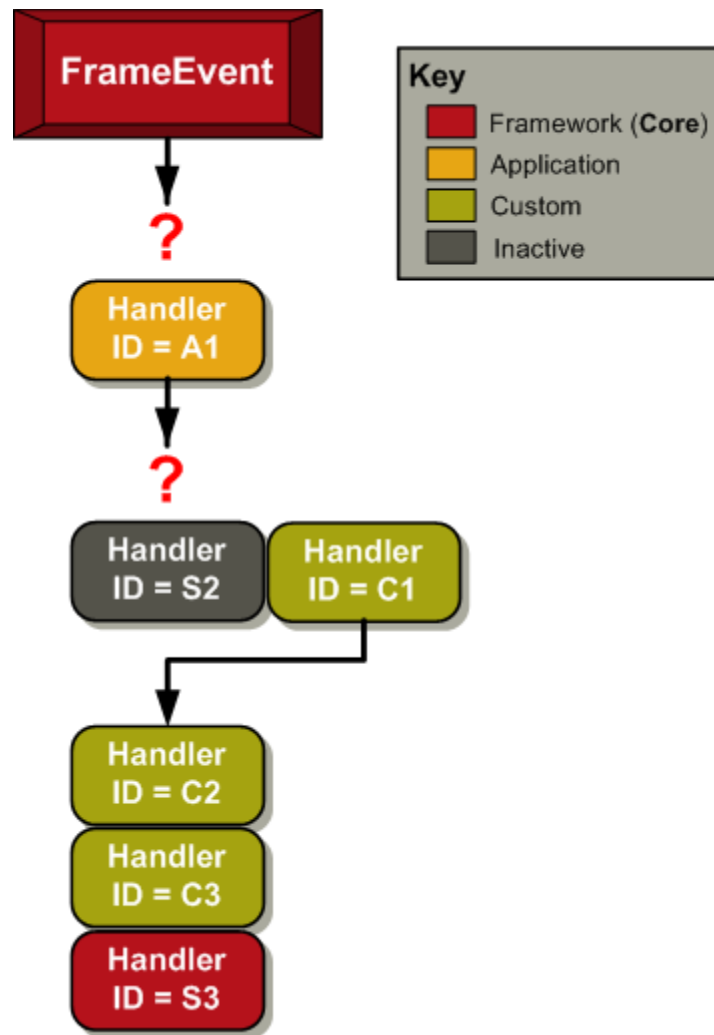
If your exported metadata removes a handler that is referenced by their handler's **Keep With** field, a broken link occurs during import. This forces the App Metadata Sync or App Metadata Transport utility to relegate the downstream developer's handler to "free floating" status, meaning its sequence is less controllable. This is especially true if you—or an upstream developer whose handlers are referenced by your handlers' **Keep With** field—later resequence your handlers.

Therefore, when a handler is no longer needed, on the **Event Handlers** form you should mark it as obsolete, by selecting the check box labeled **Obsolete**. This effectively and permanently deactivates the handler but leaves it in the sequence, so that associations with it do not get destroyed.

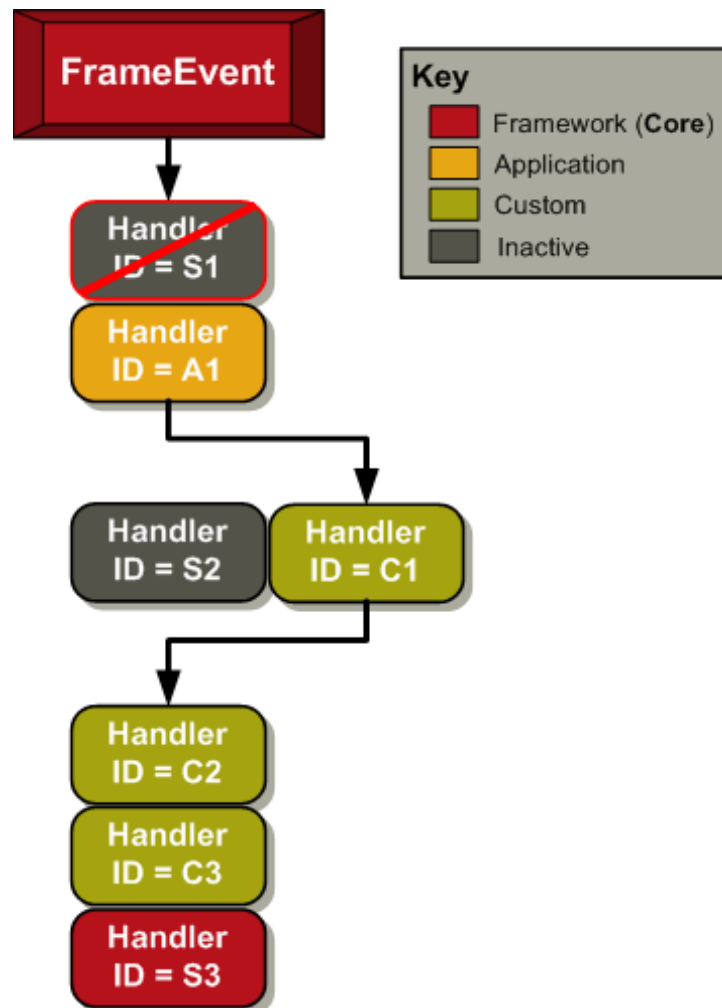
For example, consider this sequence of handlers:



Suppose, after initially releasing this version of the metadata, Infor decides not to use the handler S1 anymore. Because a business partner has associated handler A1 with this handler, if Infor simply deletes handler S1, a broken link occurs the next time the business partner or a customer who uses the business partner's add-on product tries to install the upgraded metadata. Handler A1 is relegated to a "free floating" status (as in this diagram), and unexpected consequences can result after subsequent resequencing or synchronizing operations.



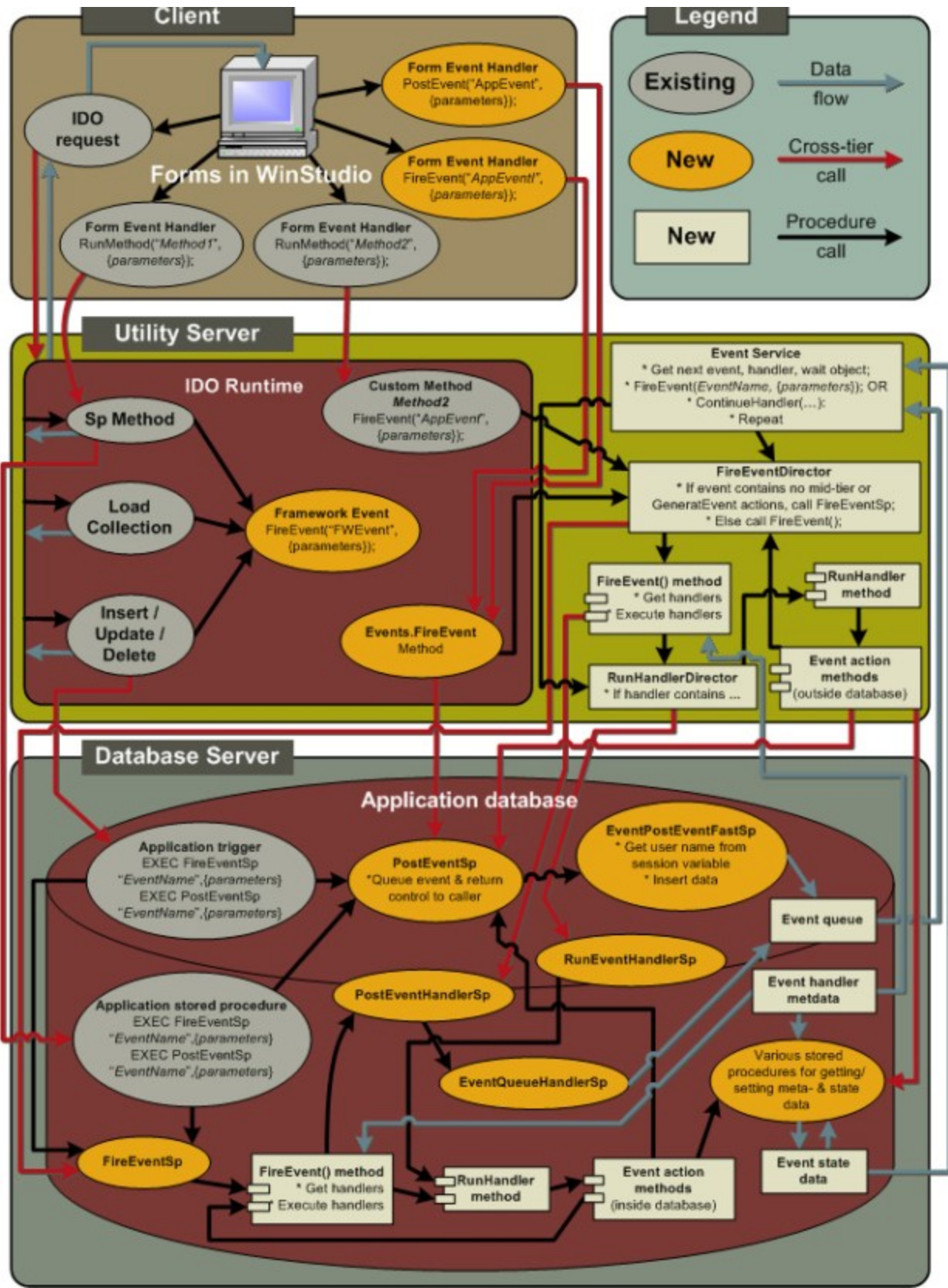
So, rather than delete handler S1, Infor would mark it as Obsolete before the next release. Then, when the update is installed, the reference to handler S1 is not broken, though the system ignores it when handling this event, as in the following diagram:



End of example

Appendix F: Event flow options

The flow diagram on the following page illustrates many of the possible ways an event can be generated and the types of flows that can result. This diagram highlights the differences between functionality with and without the Application Event System.



Appendix G: Sample scenarios



Scenario 1: Voting for various choices

In this scenario, we need several managers at the same level to approve an engineering change, by means of a response to a message. So, we must send a notification that prompts the managers for a response. If at least two of the managers send responses approving the change, we then approve the requested change in the application.

Considerations:

- Assume that global constants were created for EngineeringMgr, ProjectMgr, and ProgramMgr.
- Use the IdoOnItemUpdate framework event.

To accomplish this scenario:

1. Create and save the event handler with these settings:
 - o Event Name = IdoOnItemUpdate
 - o Applies to Objects = SLECNs
 - o Description = ECN Approval
2. Create one action, which sends a prompt to the managers requesting a response:
 - o Action Sequence = 10
 - o Action Type = Prompt
 - Use the **Event Action Prompt** form and associated **Event Action Parameter** forms to create these parameters:
 - TO = GC(EngineeringMgr) + ';' + GC(ProjectMgr) + ';' + GC(ProgramMgr)
 - SUBJECT = SUBSTITUTE("Need approval for engineering change {0}, {1}", P("EcnNum"), P("ReasonCodeDescription"))
 - CATEGORY = "Engineering"
 - BODY = "Please review the proposed engineering change on the Variables tab. Then use the Approve or Reject buttons on the Response tab to register your response. "
 - SAVEMESSAGE = FALSE
 - QUESTION = To approve or reject, click the buttons below.
 - CHOICES = 1,sApprove, 0,sReject
 - VOTINGRULE = Minimum Count Preferred Choice
 - PREFERREDCHOICE = 1
 - MINIMUM = 2
 - FILTERFORM = EngineeringChangeNotices
 - FILTER = SUBSTITUTE("ECNNum={0}", FP("ECNNum"))
3. Save the action.
 - o If you have done everything correctly, your syntax for this action step should look like this:

```
TO(" + GC(EngineeringMgr) + ';' + GC(ProjectMgr) + ';' + GC(ProgramMgr))
CATEGORY("Engineering")
SUBJECT(SUBSTITUTE("Need approval for engineering change {0}, {1}", P("EcnNum"),
P("ReasonCodeDescription")))
BODY("Please review the proposed engineering change on the Variables tab. Then use
```

the Approve or Reject buttons on the Response tab to register your response.")
 SAVEMESSAGE(FALSE)
 QUESTION("To approve or reject, click the buttons below.")
 CHOICES("1,sApprove,0,sReject") VOTINGRULE(MinimumCountPreferredChoice)
 MINIMUM(2)
 PREFCHOICE("1")
 FILTERFORM("EngineeringChangeNotices") FILTER(SUBSTITUTE("ECNNum={0}",
 FP("ECNNum")))

4. Create the second action, which tells the system how to respond if approval is not granted:
 - o Action Sequence = 20
 - o Action Type = Fail
 - This action type ends handler execution with an error status. This effectively aborts the process and prevents the ECN from being changed in the database.
 - Starting with the **Event Action Fail** form, use the associated forms to create these parameters:
 - CONDITION(VOTINGRESULT(10) = "0")
 - RESULT("The ECN change request was rejected by the managers.")
5. Save the action and close the **Event Actions** form.
6. Return to the **Event Handlers** form and select the **Suspend** check box.
7. Save the handler.
8. Discard the cached metadata.

To test the event handler:

1. Open the **Engineering Change Notices** form and update an existing ECN. Save it. After you save the ECN, when the Engineering Change Notices refreshes the display, the record should be disabled for updating. It remains read-only until/unless it has been approved.
2. (Optional) With the **Engineering Change Notices** form selected, from the **Actions** menu, select **View Event Status**. This opens the **Event Status** form. Navigate to the last row and verify that the status for this event is Running.
3. Open the **Inbox** form for the individual designated as the Engineering Manager and verify that the message was received and that the Response tab displays the question and choice buttons. Click the button labeled Yes.
4. Open the **Inbox** form for the individual designated as the Project Manager and verify that the message was received and that the Response tab displays the question and choice buttons. Click the button labeled Yes.
5. Refresh the collection on the **Engineering Change Notices** form and verify that the ECN now displays normally (read/write) and shows your changes. As soon as two managers vote for the preferred choice, voting is closed and the change is approved. The third manager's vote is not needed.

You can also do a second test, clicking the **No** button to reject the request by all three managers. In this case, when you refresh the **Engineering Change Notices** form, the ECN record displays normally but your changes are gone.

Points to note and remember

When creating this kind of event handler, keep these points in mind:

- When creating a message that requires a response from the recipient (usually a Prompt action type), you must mark the handler so that it suspends when executed. This means that it is also automatically marked as a synchronous handler.
- Because these event handlers must be suspended, pending the managers' responses, the Framework Event Service must be enabled for the configuration in which you are logged on.



Scenario 2: Localizing message contents by translating captions in a purchase request

This scenario uses strings in captions so the text can be read by users in different countries.

Metadata setup:

These strings must exist in the Forms database Strings tables:

Strings.Name	Strings.String	SpainString.String
sItem	Item	Prod
sWhse	Whse	Whse
sPoitemApprovalQuestion	Do you approve of purchasing %1 %2 of [%3: %4] for delivery to [%5: %6]	Usted aprueba de comprar %1 %2 del %3 "%4" para la entrega al %5 "%6"
sPoitemApproval	Purchase Approval	Aprobación de Comprar el Artículo
sOrderApproval	Order Approval	Aprobación del Documento

Event message category:

In the **Event Message Categories** form, set up the Category FORMAT(sOrder Approval), with the description Approval of an Order.

Event action parameters:

Create an event action that includes these parameters:

```
QUESTION (
    CLIENTSUBSTITUTE(
        "sPoitemApprovalQuestion",
        P(QtyOrdered),
        P(UMDesc),
        "STRINGS(sItem)",
        P(Item), "STRINGS(sWhse)",
        P(Whse)
    )
)
```

```

)
SUBJECT(
    CLIENTSUBSTITUTE(
        P(PoitemApproval)
    )
)

CATEGORY("FORMAT(sOrder Approval)")
...

```

After the event action execution, the full components of the message in the database are:

- EventMessage.Question
- FORMAT(sPoltemApprovalQuestion, ~LIT~(100.0), ~LIT~(Metric Tons), STRINGS(sltem),~LIT~(5" screw\x03 chrome\x04hex head\x05), STRINGS(sWhse), ~LIT~(MAIN))
- EventMessage.Subject
- FORMAT(sPoltemApproval)
- EventMessage.Category
- FORMAT(sOrderApproval)

Results (English):

An English-speaking user who refreshes the Inbox sees this message:

Component	Text
Question	Do you approve of purchasing 100.0 Metric Tons of [Item: 5" screw, chrome (hex head)] for delivery to [Whse: MAIN]
Subject	Purchase Approval
Category	Order Approval

Results (Spanish):

A Spanish-speaking user who refreshes the Inbox sees this message:

Component	Text
Question	Usted aprueba de comprar 100.0 Metric Tons del Prod "5" screw, chrome (hex head)" para la entrega al Alm "MAIN"
Subject	Aprobación de Comprar el Artículo
Category	Aprobación del Documento



Scenario 3: Cross-site event firing - Adding a message to another site's inbox

This scenario illustrates the general requirements used to set up cross-site event firing. Our example adds a message to another site's Inbox form.

- An event handler called GenericNotify is available in every target site. You can fire this event to perform the "remote" work. This event has one handler with one action that performs a Notify, using event parameters for To, Subject, Category, and Body.
- A stored procedure called dbo.FireGenericNotifySp is available in every target site. You can use this stored procedure to fire the event. This stored procedure accepts the T-SQL parameter @To, @Subject, @Category and @Body, and it fires the GenericNotify event, passing in the information that the event needs.

Note: You could do the same thing using a hand-coded IDO method that calls `Mongoose.EventSystem.EventHandlers.FireApplicationEvent()`, in case the event needs to perform IDO-level actions. However, for this scenario, using the stored procedure is easier.

- Where you want to add a message to another site's inbox, your event handler includes a Dispatch IDO Request action, providing these parameters:

Parameter	Description
URL()	URL of an IDO Request Service that serves the target site Sample value: <code>http://MyUtilityServer/IDORequestService/RequestService.aspx</code>
CONFIGNAME()	This parameter is the name of a configuration whose application database contains the target site. (Mongoose requires a configuration that is named after each site, so you can use the site name here.) Sample value: <code>E(Siteld)</code>
IDOREQUEST()	This parameter invokes "SP!" (or the name of an IDO) plus the name of the stored procedure (in this case <code>FireGenericNotifySp</code>), passing in the event parameters as method parameters to the "method." Sample value (shown as XML to allow pasting into the form): <code>SUBSTITUTE('<RequestHeader Type="Invoke"> <InitiatorType /> <InitiatorName /> <SourceName /> <TargetName /> <RequestData> <Name>SP!</Name> <Method>FireGenericNotifySp</Method></code>

Parameter	Description
	<pre> <Parameters> <Parameter>{0}</Parameter> <Parameter>{1}</ Parameter> <Parameter>{2}</Parameter> <Parameter>{3}</ Parameter> <Parameter ByRef="Y" /> </Parameters> </RequestData> </RequestHeader>', E(ToParm), E(SubjectParm), E(CategoryParm), E(BodyParm)) </pre>

- If you have multiple intranets that do not serve all sites, and you want the event to fire across intranets, you may need to set up an event global constant for each intranet (or site) URL, and select one of those at runtime.
- The sample action shown here assumes that the originating user has the same password on the source and target sites, which allows it to log in for the invoke. If this may not be the case, you can instead set up a generic remote user, with permissions only to perform this one invoke, and specify the USERNAME() and PASSWORD() parameters on the Dispatch IDO Request action. Alternatively, you can adjust the USELOCALPASSWORD() parameter.
- If you want to perform a different type of remote work, you can set up your own events with handlers and actions to address the work you want to do. Then you can create stored procedures to fire those events, and finally call those stored procedures using Dispatch IDO Request actions in the appropriate existing handlers.